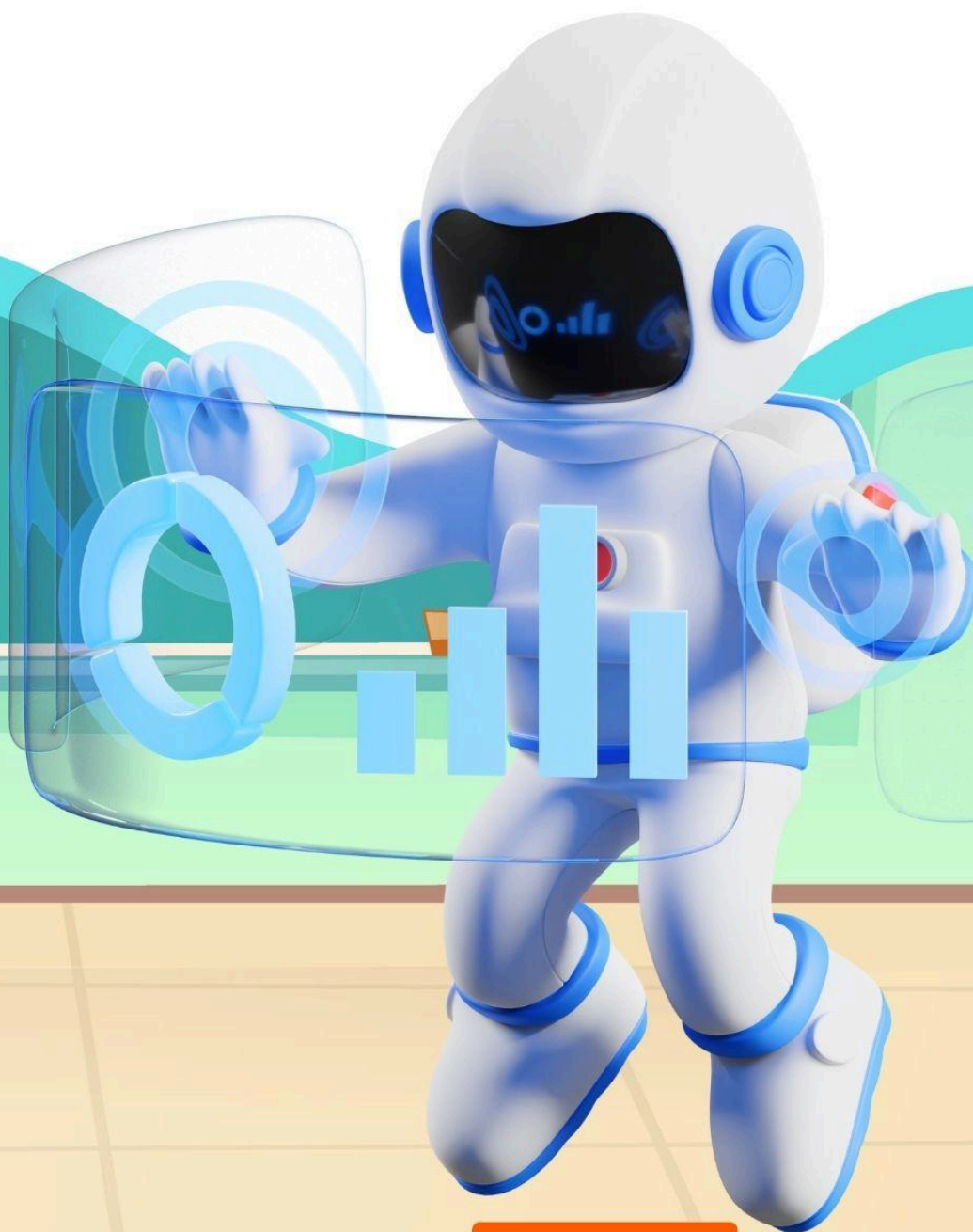


LLM & Retrieval-Augmented Generation (RAG)

untuk SMK dan umum



2025

Kelas 11 Semester 2

LLM & Retrieval-Augmented Generation (RAG)

Aolia Ikhwanudin
Onno W. Purbo

Institut Teknologi Tangerang Selatan
OnnoCenter
2025

Lisensi & Catatan Karya

Karya ini dilisensikan di bawah lisensi **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

Artinya, **siapa pun bebas untuk menggunakan, menyalin, membagikan, dan mengadaptasi** materi ini, **dengan syarat**:

- **Memberikan atribusi yang sesuai** (menyebut sumber asli),
- **Menyertakan lisensi yang sama** jika dimodifikasi atau dikembangkan lebih lanjut.

Lihat detail lisensi di sini <https://creativecommons.org/licenses/by-nd/4.0/legalcode.id>

Desain Cover: Irwan Siswanto

Disclaimer

Materi pembelajaran ini dibuat dengan **dana swadaya masyarakat Indonesia** dan **kontribusi sukarela dari para dosen di Institut Teknologi Tangerang Selatan (ITTS)**.

Karya ini didistribusikan secara bebas untuk mendukung pendidikan digital dan kecerdasan buatan di kalangan pelajar Indonesia.

Kami **memohon maaf** jika terdapat kekurangan dalam isi maupun penyajian materi ini. Kritik dan saran sangat kami harapkan untuk terus menyempurnakan karya serupa di masa depan.

Kata Pengantar

Buku ini disusun sebagai wujud nyata kepedulian sivitas akademika Institut Teknologi Tangerang Selatan (ITTS) terhadap perkembangan pendidikan kejuruan di Indonesia, khususnya bagi siswa Sekolah Menengah Kejuruan (SMK). Di tengah kemajuan teknologi yang begitu pesat, kami terdorong untuk menghadirkan materi pembelajaran **yang menantang, aplikatif, dan relevan dengan dunia industri digital**, agar siswa dapat mencapai **kemampuan maksimalnya** dalam memahami dan membangun sistem *Artificial Intelligence (AI)*.

Perlu kami sampaikan bahwa **materi dalam buku ini menuntut tingkat pemahaman teknis yang cukup tinggi**, terutama karena pembahasannya **sangat kental dengan praktik *coding* menggunakan bahasa Python** untuk membuat dan melatih model AI secara mandiri. Oleh karena itu, **tidak semua siswa, sekolah, maupun guru mungkin dapat langsung menyerap seluruh isi buku ini secara optimal**.

Bagi pembaca yang masih merasa kesulitan mengikuti langkah-langkah teknis yang disajikan, **kami sangat menyarankan untuk terlebih dahulu mempelajari buku “AI untuk SMA”**, yang disusun dengan pendekatan yang lebih **sederhana dan bertahap** sebelum melanjutkan ke tingkat SMK ini.

Penyusunan dan penerbitan buku ini dilakukan secara mandiri dengan semangat berbagi pengetahuan terbuka. Kami mengucapkan terima kasih kepada berbagai pihak yang telah mendukung dan memberi masukan dalam proses penulisan. Kritik dan saran yang membangun sangat kami harapkan agar edisi berikutnya dapat semakin baik dan mudah diakses oleh seluruh siswa Indonesia.

Kami berharap buku ini dapat menjadi **jembatan antara teori dan praktik AI**, sekaligus menginspirasi siswa SMK untuk **berpikir kreatif, logis, dan bertanggung jawab dalam menggunakan teknologi**. Dengan rendah hati, kami mohon doa dan dukungan agar karya sederhana ini dapat memberi manfaat luas dan menjadi amal jariyah bagi semua pihak yang terlibat.

Tangerang Selatan, Oktober 2025

Penulis

Daftar Isi

Disclaimer.....	2
Kata Pengantar.....	3
Daftar Isi.....	4
BAB 1: Pengenalan LLM Open-Source (Mistral, LLaMA 3 via Ollama).....	2
Tujuan Pembelajaran.....	2
Peta Konsep Sederhana.....	2
Apersepsi: Tugas Mendadak, Internet Lemot!.....	2
Penjelasan Teori.....	2
Konsep Teori dengan Analogi.....	2
Instalasi Ollama + Open-WebUI di Ubuntu 24.04 (Docker).....	3
Kasus Ilustrasi.....	4
Soal Latihan & Pembahasan.....	6
Fun Facts!.....	8
Tips Belajar atau Menghafal Cepat.....	8
Aktivitas Siswa.....	8
Rangkuman.....	11
Latihan Soal.....	12
Tugas Proyek.....	15
BAB 2: Arsitektur LLM (Transformer, Embedding, Tokenisasi).....	19
Tujuan Pembelajaran.....	19
Peta Konsep Sederhana.....	19
Apersepsi: Bot Game yang Tiba-tiba Pintar!.....	19
Penjelasan Teori.....	19
1. Tokenisasi (Tokenization).....	20
2. Embedding (Vektor Kata).....	20
3. Transformer (Attention Mechanism).....	20
Kasus Ilustrasi.....	21
Soal Latihan & Pembahasan.....	24
Fun Facts!.....	28
Tips Belajar/Menghafal Cepat.....	28
Aktivitas Siswa.....	28
Rangkuman.....	33
Latihan Soal.....	33
Tugas Proyek.....	37
BAB 3: Konsep RAG (Retriever, Index, Generator).....	43
Tujuan Pembelajaran.....	43
Peta Konsep Sederhana.....	43
Apersepsi: AI yang Gagal Jawab Soal Ulangan!.....	43
Penjelasan Teori.....	44
Kenapa Kita Butuh RAG?.....	44
Konsep RAG dengan Bahasa Sederhana.....	44

Tiga Komponen Penting RAG.....	44
Ilustrasi.....	44
Kenapa RAG Penting?.....	45
Hubungannya dengan Machine Learning.....	45
Kasus Ilustrasi.....	45
Soal Latihan & Pembahasan.....	48
Fun Facts!.....	50
Tips Belajar/Menghafal Cepat.....	50
Aktivitas Siswa.....	51
Rangkuman.....	54
Latihan Soal.....	54
Tugas Proyek.....	58
BAB 4: Praktik RAG dengan Python + FAISS/Chroma.....	61
Tujuan Pembelajaran.....	61
Peta Konsep Sederhana.....	61
Apersepsi: Kamu Mau Cari Jawaban Ulangan!.....	61
Penjelasan Teori.....	62
1. Persiapan di Ubuntu 24.04.....	62
2. Instalasi Library FAISS dan Chroma.....	62
3. Konsep Inti yang Harus Dipahami.....	63
4. Contoh Membuat RAG Sederhana dengan Python + Chroma.....	63
5. Versi Cepat dengan FAISS.....	64
6. Seru, Bukan?.....	65
Rangkuman.....	65
Tantangan untuk Siswa SMK.....	65
Kasus Ilustrasi.....	65
Mini-Checklist Teknis (Ubuntu 24.04).....	69
Soal Latihan & Pembahasan.....	70
Fun Facts!.....	73
Tips Belajar atau Menghafal Cepat.....	73
Aktivitas Siswa.....	73
Rangkuman.....	80
Latihan Soal.....	80
Lampiran Kode Ringkas (opsional dipakai di atas).....	84
Tugas Proyek.....	85
BAB 5: Integrasi RAG ke OpenWebUI untuk QnA Lokal.....	92
Tujuan Pembelajaran.....	92
Peta Konsep Sederhana.....	92
Apersepsi: AI yang Gagal Jawab Soal Ulangan!.....	92
Penjelasan Teori.....	93
Kenapa Penting?.....	93
Komponen Utama.....	93
Analogi Seru.....	94
Kaitan dengan Teknologi Open Source.....	94

Cara Instalasi di Ubuntu 24.04.....	94
Kesimpulan.....	95
Kasus Ilustrasi.....	96
Soal Latihan & Pembahasan.....	97
Fun Facts!.....	99
Tips Belajar atau Menghafal Cepat.....	99
Aktivitas Siswa.....	100
Rangkuman.....	102
Latihan Soal.....	103
Tugas Proyek.....	106
BAB 6: Etika & Keamanan Data dalam RAG.....	110
Tujuan Pembelajaran.....	110
Peta Konsep Sederhana.....	110
Apersepsi: Bot yang Bocorkan Rahasia Tim!.....	110
Penjelasan Teori.....	111
1) Kenapa Etika & Keamanan Itu Wajib?.....	111
2) Tiga Pilar Utama.....	111
3) Peta Bahaya (Threat Model) Sederhana.....	111
4) Prinsip “Aman Secara Bawaan” (Safe by Default).....	112
5) Aturan Emas (Golden Rules) untuk Siswa SMK.....	112
6) Red Flag vs Green Flag (Cepat Hafal).....	112
Kesimpulan:.....	113
Kasus Ilustrasi.....	113
Soal Latihan & Pembahasan.....	116
Fun Facts!.....	118
Tips Belajar/Menghafal Cepat.....	118
Aktivitas Siswa.....	118
Rangkuman.....	121
Latihan Soal.....	121
Tugas Proyek.....	125
BAB 7: Optimasi Penggunaan RAG di Komputer Sekolah (CPU/GPU Lokal).....	128
Tujuan Pembelajaran.....	128
Peta Konsep Sederhana.....	128
Apersepsi: Game Keren Tapi Lag!.....	128
Penjelasan Teori.....	129
1) Apa itu “Optimasi” di RAG?.....	129
2) Peran Perangkat Keras (Kayak Tim Balap).....	129
3) Memilih Model (Kecil vs Besar).....	129
4) Trik “Ngirit” Wajib Tahu.....	129
5) Optimasi Retriever (FAISS/Chroma).....	130
6) Optimasi Generator (LLM di Ollama + OpenWebUI).....	130
7) Batas Aman di Komputer Sekolah.....	130
8) Cara Ukur “Sudah Optimal atau Belum”.....	130
Checklist Cepat (Biar “Ngebut” Tanpa Drama).....	130

Ilustrasi Mini (Ringan, tapi Ngena).....	131
Bonus: Skema Alur Otak RAG (Singkat).....	131
Kasus Ilustrasi.....	131
Soal Latihan & Pembahasan.....	136
Fun Facts!.....	140
Tips Belajar atau Menghafal Cepat.....	140
Aktivitas Siswa.....	140
Tips kecil biar makin ngebut.....	145
Rangkuman.....	145
Latihan Soal.....	145
Tugas Proyek.....	148
BAB 8: Proyek – Chatbot Sekolah Berbasis Dokumen Kurikulum.....	155
Tujuan Pembelajaran.....	155
Peta Konsep Sederhana.....	155
Apersepsi: Kamu Mau Tahu Respon Penonton Film!.....	155
Penjelasan Teori.....	155
Kenapa Chatbot?.....	156
Langkah Besar dalam Proyek Ini.....	156
Teori yang Harus Dipahami.....	156
Analogi Seru.....	156
Kasus Ilustrasi.....	157
Soal Latihan & Pembahasan.....	159
Fun Facts!.....	161
Tips Belajar atau Menghafal Cepat.....	161
Aktivitas Siswa.....	162
Rangkuman.....	165
Latihan Soal.....	165
Tugas Proyek.....	169
Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.....	172

BAB 1: Pengenalan LLM Open-Source (Mistral, LLaMA 3 via Ollama)

Pernah bayangin punya asisten pintar sekeren ChatGPT, tapi dia tinggal di laptop kalian dan bisa diajak ngobrol kapan saja, bahkan saat kuota habis? Di bab ini, kita akan mewujudkan itu! Kita bakal kenalan sama **Ollama**, sebuah tool ajaib yang memungkinkan kita menjalankan model AI super canggih seperti **Mistral** dan **LLaMA 3** secara gratis. Ini adalah pintu gerbang menuju karier keren di bidang kecerdasan buatan!

Tujuan Pembelajaran

Setelah bab ini, kalian bakal bisa:

- Memahami apa itu **Ollama** dan mengapa **LLM open-source** itu penting.
- Menginstal dan menjalankan Ollama di komputer kalian.
- Mengunduh berbagai model LLM (seperti **Mistral** dan **LLaMA 3**).
- Berinteraksi dengan model AI lokal kalian untuk berbagai kebutuhan.

Peta Konsep Sederhana

Bayangin ini seperti punya mobil impian.

- **ChatGPT (LLM Online):** Mobil keren, tapi cuma bisa kalian sewa (langganan internet) dan ada aturan-aturannya.
- **Ollama (Personal Garage):** Ini garasi pribadi kalian. Bebas mau ditaruh mobil (model AI) apa aja.
- **LLaMA 3 & Mistral (Mobil Impian):** Ini adalah mobil-mobil AI open-source yang bisa kalian unduh, modifikasi, dan pakai sesuka hati di garasi pribadi kalian.

Apersepsi: Tugas Mendadak, Internet Lemot!

Kamu lagi ngerjain tugas, butuh ide buat presentasi, dan mau minta bantuan AI. Tiba-tiba, internet di rumah lemot banget. Mau buka ChatGPT, loadingnya lama. Mau tanya Google, hasilnya kurang pas. Panik, kan?

Nah, bayangkan kalau kalian punya asisten pintar di laptop sendiri. Gak perlu internet, gak perlu nunggu loading. Dia bisa langsung kasih ide dan jawaban dalam hitungan detik. Itulah kekuatan **Ollama**! Ia bikin kita jadi mandiri dan siap menghadapi tantangan apa pun.

Penjelasan Teori

Konsep Teori dengan Analogi

- **ChatGPT (Online):** Kayak mobil mewah yang disewa. Keren, tapi harus bayar kuota dan ikut aturan pemiliknya.

- **Ollama (Offline di Laptop):** Kayak garasi pribadi. Kamu bisa simpan mobil (model AI) apapun, bebas.
- **Mistral & LLaMA 3 (Mobil Gratisan):** Mobil impian yang open-source. Bisa di-*download*, dipakai, bahkan dimodifikasi.

Kenapa pilih yang *open-source*?

1. **Privasi Aman** – Data gak keluar ke server lain.
2. **Cepat** – Gak butuh internet, jadi respon instan.
3. **Gratis** – Gak ada biaya bulanan.
4. **Bebas Modifikasi** – Cocok buat siswa kreatif yang suka ngoprek.

Instalasi Ollama + Open-WebUI di Ubuntu 24.04 (Docker)

Tenang, ini bukan hal rumit kok. Kita pakai *open-source tools* biar gampang dan rapi.

Langkah Instalasi:

- **Siapkan Server Ubuntu 24.04**
 - Bisa di laptop, PC, atau server sekolah.
- **Pasang Docker & Docker Compose**
 - Docker itu kayak wadah ajaib buat jalankan aplikasi.
 - Perintahnya:

```
sudo apt update
sudo apt install docker.io docker-compose -y
```

- **Buat File docker-compose.yml**
Isinya kayak resep makanan. Kita bilang ke Docker: jalankan Ollama + Open-WebUI:

```
version: '3.9'
services:
  ollama:
    image: ollama/ollama
    container_name: ollama
    volumes:
      - ./ollama:/root/.ollama
    ports:
      - "11434:11434"
    restart: always

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    ports:
      - "3000:8080"
    volumes:
      - ./open-webui:/app/backend/data
    restart: always
```

- **Jalankan dengan Docker Compose**

```
docker-compose up -d
```

- **Coba Akses Web**

- Buka browser → ketik <http://localhost:3000>.
- Tadaaa 🎉! Kamu bisa ngobrol dengan AI lewat tampilan web.

Kasus Ilustrasi

Kasus 1: Membantu Tugas Sekolah

Situasi: Kamu diminta bikin 5 ide proyek untuk jurusan TKJ (Teknik Komputer dan Jaringan).

Ilustrasi: Daripada bingung, kamu tanya ke AI di laptopmu.

Pseudocode:

```
jalankan ollama dengan model llama3
ajukan pertanyaan: "Berikan 5 ide proyek akhir untuk jurusan
TKJ"
AI memberikan jawaban berupa daftar ide
tampilkan ide di layar
```

Kasus 2: Debug Kode Python

Situasi: Kode Python kamu error, gak tahu salahnya di mana.

Ilustrasi: Tempel kode di Ollama, lalu minta bantuannya.

Pseudocode:

```
jalankan ollama dengan model mistral
copy kode python yang error
paste ke dalam chat ollama
ajukan pertanyaan: "Tolong perbaiki kode ini"
AI memberi saran perbaikan kode
```

Kasus 3: Membuat Ringkasan Buku

Situasi: Kamu harus presentasi buku setebal 200 halaman.

Ilustrasi: Kamu salin teks bab penting, lalu minta ringkasannya ke AI.

Pseudocode:

```
jalankan ollama dengan model llama3
paste teks buku bab 3
ajukan pertanyaan: "Ringkas isi bab ini dalam 3 paragraf
singkat"
AI menampilkan ringkasan
```

Kasus 4: Membuat Jadwal Harian Seimbang

Situasi: Kamu suka gaming, tapi juga harus belajar.

Ilustrasi: Kamu minta AI bikin jadwal yang seimbang antara belajar, olahraga, dan main game.

Pseudocode:

```
jalankan ollama dengan model mistral
ajukan pertanyaan: "Buatkan jadwal harian untuk belajar,
olahraga, dan gaming"
AI menampilkan tabel jadwal harian
```

Kasus 5: Generator Kode Otomatis

Situasi: Kamu diminta guru untuk bikin program Python sederhana menghitung luas segitiga.

Ilustrasi: Kamu tanya ke AI, lalu langsung dapat kode siap pakai.

Pseudocode:

```
jalankan ollama dengan model llama3
ajukan pertanyaan: "Buatkan kode python untuk menghitung luas
segitiga"
AI memberikan kode python
salin kode lalu jalankan di python
```

Catatan

- Ollama bisa dipakai untuk **ide proyek**, **debug kode**, **ringkas buku**, **atur jadwal**, sampai **buat kode otomatis**.
- Semua bisa jalan **offline**, aman, gratis, dan cepat.
- Dengan contoh ini, kamu jadi tahu bahwa AI lokal itu bukan cuma teori, tapi bisa bantu kegiatan sehari-hari di sekolah.

Soal Latihan & Pembahasan

Soal 1: Dasar Ollama

Pertanyaan:

Perintah apa yang digunakan untuk mengunduh model *LLaMA 3* di Ollama?

Pseudocode:

```
buka terminal
ketik perintah: ollama pull llama3
tunggu hingga proses selesai
```

Jawaban:

```
ollama pull llama3
```

Pembahasan:

Perintah **pull** dipakai untuk menarik (download) model dari internet ke komputer lokal. Tanpa **pull**, kamu tidak bisa menjalankan model.

Soal 2: Keunggulan LLM Offline

Pertanyaan:

Sebutkan satu keunggulan utama menjalankan *LLM offline* dengan Ollama dibanding ChatGPT online.

Pseudocode:

```
jika LLM offline:
    data tetap di komputer (aman)
    akses cepat tanpa internet
```

Jawaban:

Privasi lebih terjamin karena data tetap di komputer lokal.

Pembahasan:

LLM offline tidak mengirim data ke server lain. Jadi, data kamu aman, cepat, dan bisa digunakan tanpa internet.

Soal 3: Error Handling

Pertanyaan:

Apa yang terjadi jika kamu mengetik perintah **ollama run mistral** padahal kamu belum pernah mengunduh model *Mistral*?

Pseudocode:

```
jika model mistral belum ada:
    ollama otomatis mencoba pull model mistral
```

```
jika sukses → model berjalan  
jika gagal → muncul error
```

Jawaban:

Ollama akan otomatis mencoba mengunduh model *Mistral* terlebih dahulu.

Pembahasan:

Jika model belum ada di komputer, Ollama langsung mendownload. Baru setelah selesai, model bisa dipakai.

Soal 4: Coding dengan AI

Pertanyaan:

Kamu meminta AI untuk membuat kode Python menghitung luas persegi panjang dengan panjang = 10, lebar = 5. Hasil output yang benar adalah?

Pseudocode:

```
panjang = 10  
lebar = 5  
luas = panjang * lebar  
print(luas)
```

Jawaban:

50

Pembahasan:

Luas persegi panjang = panjang × lebar = $10 \times 5 = 50$.

Soal 5: Batas Kemampuan LLM Offline

Pertanyaan:

Berikan satu contoh pertanyaan yang **tidak bisa dijawab** oleh LLM offline seperti LLaMA 3 atau Mistral.

Pseudocode:

```
jika pertanyaan butuh data real-time:  
    LLM offline tidak bisa menjawab
```

Jawaban:

“Apa skor pertandingan sepak bola hari ini?”

Pembahasan:

LLM offline tidak terhubung ke internet, jadi dia tidak bisa menjawab pertanyaan yang butuh data *real-time* seperti berita terbaru atau skor pertandingan.

Catatan

- Soal 1 menguji perintah dasar.
- Soal 2 melatih pemahaman konsep privasi.
- Soal 3 mengajarkan logika error dan auto-download.
- Soal 4 mengaitkan LLM dengan coding Python.
- Soal 5 menguji batas kemampuan LLM offline.

Dengan latihan ini, siswa SMK bisa paham teori + langsung lihat aplikasinya lewat *pseudocode*.

Fun Facts!

- Ukuran model LLM itu bisa sangat besar, lho! Model-model terbesar bisa berukuran lebih dari 100 GB.
- **Ollama** dikembangkan untuk mempermudah siapa saja, bahkan yang tidak punya komputer super canggih, untuk mencoba teknologi AI terbaru.
- Ada pekerjaan baru yang muncul berkat teknologi ini, namanya **prompt engineer**. Tugasnya adalah menulis "pertanyaan" yang paling efektif agar AI memberikan jawaban terbaik.

Tips Belajar atau Menghafal Cepat

- **Langsung Coba!** Belajar AI paling efektif adalah dengan praktik. Jangan takut mencoba, kok, gak akan merusak komputer.
- **Mulai dari Model Kecil:** Unduh model yang ukurannya lebih kecil dulu (misal **gemma** atau **tinydolphin**) agar prosesnya lebih cepat.
- **Buat Catatan Sendiri:** Tulis di buku, "LLM ini jago buat nulis cerita," atau "Model itu jago buat coding."

Aktivitas Siswa

Aktivitas 1: Instalasi & Tarik Model LLM

Tujuan: Belajar mengunduh model LLM dari Ollama.

Langkah:

1. Buka **Terminal** di Ubuntu.
2. Ketik perintah untuk menarik (*pull*) model.
3. Tunggu sampai proses selesai.

Pseudocode:

```
buka terminal
ketik: ollama pull llama3
jika selesai → model siap digunakan
```

Contoh:

```
ollama pull llama3
```

Aktivitas 2: Interaksi Teks dengan AI

Tujuan: Coba ngobrol dengan AI lokal.

Langkah:

1. Jalankan model di Terminal.
2. Tulis pertanyaan sederhana.
3. Lihat jawaban dari AI.

Pseudocode:

```
jalankan model llama3
ajukan pertanyaan: "Buat cerita pendek tentang robot dan
manusia"
tampilkan jawaban AI
```

Contoh:

```
ollama run llama3
```

(lalu ketik pertanyaan di layar)

Aktivitas 3: Generator Kode Python

Tujuan: Minta AI membuat program Python sederhana.

Langkah:

1. Jalankan model LLM.
2. Ajukan prompt untuk membuat kode.
3. Salin kode lalu jalankan di Python.

Pseudocode:

```
jalankan model mistral
ajukan pertanyaan: "Buat kode python untuk menghitung luas
persegi panjang"
AI memberi kode → jalankan di python
```

Contoh Python Script:

```
panjang = 10
lebar = 5
luas = panjang * lebar
print("Luas persegi panjang:", luas)
```

Aktivitas 4: Eksperimen Data dengan Pandas

Tujuan: Belajar analisis data sederhana.

Langkah:

1. Buat file CSV kecil berisi data siswa.
2. Buka file dengan Pandas.
3. Hitung rata-rata nilai.

Pseudocode:

```
import pandas
baca file csv
hitung rata-rata kolom nilai
tampilkan hasil
```

Contoh Python Script:

```
import pandas as pd

data = pd.DataFrame({
    "Nama": ["Ani", "Budi", "Cici"],
    "Nilai": [80, 90, 70]
})

print("Rata-rata nilai:", data["Nilai"].mean())
```

Aktivitas 5: Membuat Neural Network Sederhana

Tujuan: Mengenal *machine learning* dengan Keras.

Langkah:

1. Import library Keras.
2. Buat model sederhana dengan satu layer.
3. Jalankan model untuk melatih data dummy.

Pseudocode:

```
import keras
buat model sequential
tambahkan layer dense
kompilasi model
latih dengan data dummy
```

Contoh Python Script:

```
from keras.models import Sequential
from keras.layers import Dense
import numpy as np

# Data dummy
x = np.array([[0], [1], [2], [3]])
```

```

y = np.array([[0], [1], [2], [3]])

# Model sederhana
model = Sequential()
model.add(Dense(1, input_dim=1))

model.compile(optimizer='sgd', loss='mse')
model.fit(x, y, epochs=10)

```

Catatan

- **Aktivitas 1:** Belajar *pull model* dari Ollama.
- **Aktivitas 2:** Ngobrol dengan AI lokal.
- **Aktivitas 3:** Gunakan AI untuk buat kode Python.
- **Aktivitas 4:** Analisis data dengan Pandas.
- **Aktivitas 5:** Coba neural network sederhana dengan Keras.

Dengan aktivitas ini, siswa SMK bukan cuma baca teori, tapi langsung **praktek**. Seru, menantang, dan pastinya bikin tambah paham! 🚀

Rangkuman

Di bab ini, kita sudah mengenal **Ollama** sebagai tool *open-source* yang bisa menjalankan *Large Language Model (LLM)* seperti **Mistral** dan **LLaMA 3** langsung di komputer kita. Dengan Ollama, kita punya **laboratorium AI pribadi** tanpa harus bergantung pada internet atau layanan berbayar. Ini membuat kita lebih **mandiri, cepat, dan aman** saat menggunakan teknologi AI.

Selain itu, kita belajar bahwa **LLM offline** punya banyak keuntungan: data tetap **privat**, respon lebih **cepat**, dan bebas dipakai tanpa biaya. Kita juga bisa menggunakannya untuk berbagai kebutuhan, mulai dari **membantu tugas sekolah, menulis kode Python, merangkum buku, sampai mengatur jadwal harian**. Semua ini membuktikan bahwa teknologi *open-source* bisa diakses siapa saja, termasuk siswa SMK.

Terakhir, bab ini mengajarkan bahwa memahami dan mencoba langsung itu penting. **Coba instal, jalankan, dan eksplorasi Ollama**, karena dengan praktik kita bisa lebih cepat paham. Ingat, *AI* bukan hanya tool bantu, tapi juga bisa jadi **teman belajar, partner kreatif, bahkan inspirasi untuk karier masa depan**. Dengan semangat bereksperimen, kalian siap melangkah ke dunia kecerdasan buatan yang seru dan penuh peluang. 🚀

Latihan Soal

A. Benar/Salah (5 Soal)

1. Ollama bisa menjalankan *LLM* secara offline di laptop.
Jawaban: Benar
2. *LLaMA 3* dan *Mistral* adalah model *open-source*.
Jawaban: Benar
3. Untuk memakai Ollama wajib koneksi internet setiap kali *run*.
Jawaban: Salah
4. Keunggulan LLM offline adalah privasi data lebih terjaga.
Jawaban: Benar
5. Perintah `ollama pull` dipakai untuk menjalankan model.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Perintah untuk mengunduh *LLaMA 3* di Ollama adalah...
A. `ollama run llama3`
B. `ollama pull llama3`
C. `ollama start llama3`
D. `ollama get llama3`
Jawaban: B
2. Manfaat utama LLM offline:
A. Selalu terbaru real-time
B. Butuh langganan bulanan
C. Privasi dan cepat
D. Hanya jalan di *cloud*
Jawaban: C
3. Jika menjalankan `ollama run mistral` dan model belum ada, yang terjadi:
A. Gagal permanen
B. Otomatis *pull* lalu jalan
C. Menghapus cache
D. Meminta *license key*
Jawaban: B
4. *Open-WebUI* berfungsi sebagai...
A. *Compiler* Python
B. Antarmuka web untuk ngobrol dengan LLM lokal
C. Pengganti sistem operasi

D. Editor video

Jawaban: B

5. Perintah untuk memulai kontainer Docker secara *detached*:

A. `docker run -it`

B. `docker stop`

C. `docker-compose up -d`

D. `docker compose logs`

Jawaban: C

6. Analogi yang tepat: **Ollama = garasi pribadi, model AI = ...**

A. Sopir

B. Mobil

C. Jalan tol

D. Bensin

Jawaban: B

7. File konfigurasi *docker-compose* umumnya bernama...

A. `compose.json`

B. `dockerfile`

C. `docker-compose.yml`

D. `docker.run`

Jawaban: C

8. Library Python untuk *dataframe*:

A. `keras`

B. `pandas`

C. `torchvision`

D. `flask`

Jawaban: B

9. Contoh prompt yang sesuai untuk LLM offline:

A. "Skor bola menit ini?"

B. "Ringkas teks ini jadi 3 poin."

C. "Buka situs berita X."

D. "Unduh file dari internet."

Jawaban: B

10. Yang bukan keuntungan *open-source*:

A. Bisa dipelajari

B. Bisa dimodifikasi

C. Gratis pakai

D. Tertutup dan rahasia

Jawaban: D

C. Isian Singkat (5 Soal)

1. Tulis perintah *Terminal* untuk mengunduh model **mistral**.

Jawaban: `ollama pull mistral`

2. Tulis perintah *Terminal* untuk menjalankan **llama3**.

Jawaban: `ollama run llama3`

3. Sebutkan satu alasan memilih LLM offline.

Jawaban: Privasi

4. Nama antarmuka web *open-source* untuk Ollama yang populer.

Jawaban: Open-WebUI

5. Isi *Python* satu baris untuk mencetak "Halo AI".

Jawaban: `print("Halo AI")`

D. Soal Eksplorasi (3 Soal)

Soal 1: Studi Kasus Privasi:

Jelaskan skenario di sekolahmu ketika memakai LLM offline lebih aman dibanding layanan online. Sebutkan data apa yang ingin kamu jaga.

Kunci jawaban (contoh):

Mengolah draf surat pribadi siswa/guru di laptop sekolah agar tidak keluar jaringan; data yang dijaga: nama, NIS, nilai, nomor kontak.

Soal 2: Perbandingan Model:

Buat rencana uji sederhana untuk membandingkan LLaMA 3 dan Mistral pada tugas "ringkas paragraf". Sertakan 3 langkah uji dan metrik singkat.

Kunci jawaban (contoh):

Langkah:

1. siapkan 5 paragraf
2. minta ringkasan 3 kalimat
3. nilai keterbacaan.

Metrik: panjang ringkasan, jumlah fakta benar, kemudahan dibaca.

Soal 3: Mini Otomasi:

Tuliskan ide workflow "tempel teks → ringkas → simpan ke file" memakai pseudocode sederhana.

Kunci jawaban (contoh):

```
input teks
kirim prompt ke llama3: "ringkas 3 kalimat"
terima hasil
simpan ke "ringkasan.txt"
```

Tugas Proyek

Proyek 1: Kamus Mini Bahasa Gaul dengan AI

Tugas: Buatlah kamus mini dari 5 kata gaul, lalu minta AI lokal (LLaMA 3/Mistral) untuk menjelaskan artinya.

Langkah:

1. Jalankan Ollama.
2. Masukkan daftar kata gaul.
3. Minta AI menjelaskan arti kata satu per satu.

Pseudocode:

```
jalankan ollama run llama3
input: ["mager", "gabut", "bucin", "santuy", "gokil"]
for setiap kata:
    tanya AI "Apa arti kata <kata> dalam bahasa gaul?"
    tampilkan jawaban AI
```

Contoh Output:

- *Mager* = Malas gerak.
- *Gabut* = Gak ada kerjaan.
- *Bucin* = Budak cinta.

Proyek 2: Ringkas Artikel Panjang Jadi 3 Kalimat

Tugas: Salin artikel berita dari internet (copy teks saja, bukan link). Minta AI membuat ringkasan hanya 3 kalimat.

Langkah:

1. Cari teks artikel.
2. Jalankan model di Ollama.
3. Tempel teks artikel → beri prompt untuk diringkas.

Pseudocode:

```
jalankan ollama run mistral
input: artikel panjang
tanya: "Ringkas artikel ini dalam 3 kalimat saja"
tampilkan hasil ringkasan
```

Contoh Hasil:

“Artikel ini membahas pentingnya menjaga kesehatan mental. Banyak anak muda mengalami stres karena sekolah dan pekerjaan. Solusinya adalah olahraga, tidur cukup, dan dukungan sosial.”

Proyek 3: Generator Soal Pilihan Ganda

Tugas: Minta AI membuat 5 soal pilihan ganda tentang “Jaringan Komputer” lengkap dengan jawabannya.

Langkah:

1. Jalankan Ollama.
2. Ajukan prompt untuk membuat soal.
3. Simpan hasil soal ke file teks.

Pseudocode:

```
jalankan ollama run llama3
tanya: "Buatkan 5 soal pilihan ganda tentang Jaringan Komputer
beserta jawabannya"
simpan output ke file soal.txt
```

Contoh Soal:

1. Apa kepanjangan dari LAN?
 - a. Local Area Network 
 - b. Long Area Network
 - c. Light Access Node
 - d. Linked Access Net

Proyek 4: Analisis Sentimen Sederhana Komentar

Tugas: Buat program Python sederhana untuk menganalisis komentar apakah positif atau negatif menggunakan *rule-based* (bukan AI dulu).

Langkah:

1. Buat daftar kata positif dan negatif.
2. Bandingkan kata dalam komentar dengan daftar.
3. Tentukan sentimen.

Pseudocode:

```
buat list kata_positif = ["bagus", "hebat", "keren"]
buat list kata_negatif = ["jelek", "buruk", "parah"]

input komentar
if ada kata_positif:
    hasil = "Positif"
else if ada kata_negatif:
    hasil = "Negatif"
else:
    hasil = "Netral"
```

Contoh Python Script:

```
positif = ["bagus", "hebat", "keren"]
negatif = ["jelek", "buruk", "parah"]

komentar = "Pelajaran ini keren banget"
if any(word in komentar for word in positif):
    print("Sentimen: Positif")
elif any(word in komentar for word in negatif):
    print("Sentimen: Negatif")
else:
    print("Sentimen: Netral")
```

Proyek 5: Buat Jadwal Otomatis dengan Python

Tugas: Buat program Python yang menampilkan jadwal belajar harian siswa (pagi = belajar, siang = olahraga, malam = gaming).

Langkah:

1. Buat dictionary berisi jadwal.
2. Cetak jadwal sesuai urutan waktu.

Pseudocode:

```
buat dictionary jadwal
isikan {"Pagi": "Belajar", "Siang": "Olahraga", "Malam":
"Gaming"}
for setiap item:
    print waktu + kegiatan
```

Contoh Python Script:

```
jadwal = {
    "Pagi": "Belajar",
    "Siang": "Olahraga",
    "Malam": "Gaming"
}

for waktu, kegiatan in jadwal.items():
    print(waktu, ":", kegiatan)
```

Output:

```
Pagi : Belajar
Siang : Olahraga
Malam : Gaming
```

Catatan

- **Proyek 1:** Kamus Gaul Lokal.

- **Proyek 2:** Ringkas Artikel Panjang.
- **Proyek 3:** Generator Soal Pilihan Ganda.
- **Proyek 4:** Analisis Sentimen Sederhana.
- **Proyek 5:** Jadwal Harian Otomatis.

Semua proyek ini mudah tapi tetap **menantang**. Siswa bisa latihan *logical thinking*, coding Python, sampai kreatif membuat soal dan kamus gaul.

BAB 2: Arsitektur LLM (Transformer, Embedding, Tokenisasi)

Selamat datang kembali. Setelah di bab sebelumnya kita berhasil "mengundang" AI canggih ke laptop kita, sekarang saatnya kita bongkar rahasianya! Kita akan masuk ke "**otak**" dari AI, yaitu arsitektur **Large Language Model (LLM)**. Ini adalah materi yang bakal bikin kalian paham kenapa AI bisa secerdas itu. Siap jadi insinyur AI?

Tujuan Pembelajaran

- Memahami arsitektur dasar LLM.
- Mengerti proses **tokenisasi** dan **embedding** dalam mengolah teks.
- Mengenali peran utama arsitektur **Transformer**.
- Menghubungkan semua konsep ini untuk memahami cara kerja AI.

Peta Konsep Sederhana

Bayangin kita mau merakit robot super cerdas dari balok LEGO.

- **Kata-kata (Teks)**: Balok-balok LEGO dengan bentuk dan warna yang campur aduk. Ini data mentah kita.
- **Tokenisasi (Memilah Balok)**: Memilah balok-balok itu berdasarkan warna dan bentuknya, lalu memberinya nomor. Setiap kata jadi satu balok.
- **Embedding (Memberi Kode Rahasia)**: Memberi setiap balok sebuah "kode rahasia" yang berisi angka. Balok yang mirip (misal: "makan" dan "minum") punya kode yang mirip.
- **Transformer (Merakit Robot)**: Sebuah mesin ajaib yang bisa merakit semua balok itu dengan sangat cepat, memperhatikan hubungan antarbalok agar jadi robot yang sempurna.

Apersepsi: Bot Game yang Tiba-tiba Pintar!

Pernah main game dan ketemu NPC (Non-Player Character) yang bisa ngobrol dan jawab pertanyaanmu dengan sangat nyambung? Itu bukan sihir, tapi arsitektur **Transformer** di balik LLM!

Dulu, bot cuma bisa jawab dengan kalimat yang sudah diprogram. Kalau kamu tanya, "Di mana pintu masuknya?" dia akan jawab, "Saya tidak mengerti." Tapi sekarang, berkat arsitektur LLM, dia bisa mengerti konteksnya dan menjawab, "Pintu masuk ada di sebelah utara, di samping pohon besar."

Penjelasan Teori

Di bab sebelumnya, kita sudah berhasil “mengundang” AI canggih masuk ke laptop kita. Sekarang kita naik level. Kita bakal **membedah otak AI**, yaitu **arsitektur LLM (Large Language Model)**.

Bayangin kalian jadi *engineer* robot masa depan.

Robot ini bisa ngobrol, paham bahasa manusia, bahkan kasih jawaban yang masuk akal.

Nah, semua itu bisa terjadi berkat 3 komponen ajaib:

- **Tokenisasi**
- **Embedding**
- **Transformer**

1. Tokenisasi (Tokenization)

- **Apa itu?** Kalimat dipecah jadi potongan kecil (*token*). Bisa kata, sub-kata, atau tanda baca.
- **Contoh:** "Saya suka makan bakso." → ['Saya', 'suka', 'makan', 'bakso', '.']
- **Kenapa penting?** Komputer cuma ngerti angka, jadi tokenisasi bikin teks lebih gampang diproses.

2. Embedding (Vektor Kata)

- **Apa itu?** Setiap token diubah jadi daftar angka (*vector*).
- **Analogi:** Kata *enak* $\approx$ *lezat* → punya angka yang mirip. Kata *buruk* → beda jauh.
- **Kenapa penting?** Ini cara AI “mengerti” arti dan hubungan antar kata.

3. Transformer (Attention Mechanism)

- **Apa itu?** Mesin canggih yang bisa membaca semua kata sekaligus, bukan satu-satu.
- **Analogi:** "Dia melempar bola ke dia."
Transformer bisa sadar bahwa “dia” pertama dan “dia” kedua itu beda orang.
- **Kenapa penting?** AI jadi bisa ngerti konteks, bukan sekadar hafalan.

Kasus Ilustrasi

Kasus 1: Sistem Peringkas Teks Otomatis

Cerita: Kalian dapat berita sekolah yang panjang. Kalian butuh ringkasan 2–3 kalimat.

Tujuan: Meringkas teks, tetap akurat dan mudah dibaca.

Alur LLM:

- *Tokenisasi*: pecah kalimat jadi token.
- *Embedding*: ubah token jadi angka (vektor).
- *Transformer*: pakai *attention* untuk pilih bagian penting → buat ringkasan.

Pseudocode:

```
input: teks_berita
tokens = tokenize(teks_berita)
vectors = embed(tokens)
summary = transformer.summarize(vectors, max_sentences=3)
output: summary
```

Starter code (Python, langkah awal):

```
teks = "Pengumuman lomba kebersihan kelas... (panjang)"
tokens = teks.split() # tokenisasi sederhana
print("Jumlah token:", len(tokens))
# Ringkasan bisa dilakukan memakai model LLM lokal via
Ollama/OpenWebUI.
```

Kasus 2: Filter Komentar di Media Sosial

Cerita: OSIS ingin menyaring komentar: negatif / netral / positif.

Tujuan: Klasifikasi sentimen komentar.

Alur LLM:

- *Tokenisasi*: pecah tiap komentar.
- *Embedding*: representasi angka; “bagus”, “keren” saling dekat.
- *Transformer*: pahami konteks → prediksi label sentimen.

Pseudocode:

```
input: daftar_komentar
for komentar in daftar_komentar:
    t = tokenize(komentar)
    v = embed(t)
    label = transformer.classify(v, labels=["neg", "neu", "pos"])
    simpan(label)
laporkan_ringkasan_label()
```

Starter code (Python, dummy awal):

```
komentar = ["Keren banget!", "Biasa saja", "Kurang rapi nih"]
for k in komentar:
    print(k.lower().split()) # ganti dengan tokenizer lebih baik
nanti
```

Catatan: Untuk proyek nyata, bisa *fine-tune* model kecil di **Keras** atau pakai LLM lokal lewat **Ollama**.

Kasus 3: Bot di E-Commerce (Koperasi Sekolah)

Cerita: Koperasi ingin bot jawab stok/harga seragam dan tool tulis.

Tujuan: Jawab pertanyaan natural berdasarkan katalog produk.

Alur LLM:

- *Tokenisasi*: pertanyaan user.
- *Embedding*: pertanyaan & deskripsi produk ke vektor.
- *Transformer*: pahami maksud (“ada ukuran L?”) → ambil konteks produk → jawab.

Pseudocode:

```
input: pertanyaan, katalog
q_tok = tokenize(pertanyaan)
q_vec = embed(q_tok)
prod_vecs = embed(deskripsi_produk_katalog)
best = cosine_similarity(q_vec, prod_vecs).argmax()
jawaban = transformer.generate(context=katalog[best],
question=pertanyaan)
output: jawaban
```

Starter code (Python, pendek):

```
pertanyaan = "Ada seragam ukuran L?"
if "seragam" in pertanyaan.lower() and "l" in pertanyaan.lower():
    print("Seragam ukuran L tersedia. Stok: 5. Harga: 120k.")
# Versi Transformer akan lebih tangguh untuk variasi kalimat.
```

Kasus 4: “Mendiagnosis” Gejala (Edu-Health)

Cerita: Siswa menuliskan gejala: “demam 3 hari, batuk kering”.

Tujuan: Edukasi pengelompokan gejala (bukan diagnosis medis).

Alur LLM:

- *Tokenisasi*: gejala jadi token.
- *Embedding*: istilah kesehatan ke vektor.
- *Transformer*: pahami pola gejala → kategorikan → beri saran edukasi.

Pseudocode:

```
input: teks_gejala
t = tokenize(teks_gejala)
v = embed(t)
kategori = transformer.classify(v,
labels=["ringan", "flu_like", "butuh_pemeriksaan"])
if kategori == "butuh_pemeriksaan":
    saran = "Segera konsultasi tenaga kesehatan."
else:
    saran = "Istirahat, minum air, pantau suhu."
output: kategori, saran
```

Starter code (Python, logika sederhana):

```
g = "demam 3 hari dan batuk kering"
if ("demam" in g) and ("batuk" in g):
    print("Kategori: flu_like | Saran: istirahat & cek suhu.
Jika memburuk, periksa.")
```

Disclaimer: Ini **bukan** tool diagnosis. Hanya untuk edukasi.

Kasus 5: Autocompletion Teks di HP

Cerita: Saat mengetik, HP menebak kata berikutnya.

Tujuan: Prediksi kata selanjutnya berbasis konteks.

Alur LLM:

- *Tokenisasi*: teks yang sudah diketik.
- *Embedding*: urutan kata ke vektor.
- *Transformer: self-attention* ke kata sebelumnya → prediksi *next token*.

Pseudocode:

```
input: teks_sejauh_ini
tok = tokenize(teks_sejauh_ini)
vec = embed(tok)
next_word = transformer.predict_next(vec)
tampilkan_saran(next_word, k=3)
```

Starter code (Python, dummy kamus kecil):

```
konteks = "Aku mau"
kamus = {"aku mau": ["makan", "belajar", "tidur"]}
print("Saran:", kamus.get(konteks.lower(), ["(latih model
dulu)"])[ :3])
```

Toolkit *Open Source* yang disarankan

- **Python + Pandas** → olah teks/CSV.

- **NLTK / regex** → *tokenisasi* dasar.
- **scikit-learn / Gensim** → *embedding* klasik (TF-IDF, Word2Vec).
- **Keras (TensorFlow)** → *neural network* kecil, *text classification*.
- **Orange Data Mining** → eksplorasi tanpa banyak kode (Corpus, Word Embeddings, Scatter Plot).
- **Ollama + OpenWebUI** → jalankan *LLM* lokal/offline untuk *summarize*, *QA*, dst.

Ingat Cepat

- **Tokenisasi** = teks → potongan kecil (*token*).
- **Embedding** = token → angka yang “mengerti” makna.
- **Transformer** = *attention* → paham konteks → jawaban lebih pintar.

Kalau mau, saya bisa lanjutkan jadi **lembar kerja** per kasus (input contoh, langkah detail, *rubric* penilaian, dan *starter notebook* Python) biar langsung siap dipakai di kelas.

Soal Latihan & Pembahasan

Soal 1: Soal Tokenisasi: Pisahkan Kata & Tanda Baca

Pertanyaan:

Lakukan *tokenisasi* pada kalimat:

"Buku ini sangat bagus, saya suka sekali!"

Tunjukkan **token** termasuk tanda baca.

Jawaban:

```
['Buku', 'ini', 'sangat', 'bagus', ',', 'saya', 'suka', 'sekali', '!']
```

Pembahasan (sederhana):

- *Tokenisasi* memecah teks menjadi potongan kecil bernama *token*.
- Tanda baca penting dipertahankan karena punya makna (misal “!” menunjukkan emosi).
- `split()` saja tidak cukup, karena biasanya menghilangkan tanda baca. Pakai *regex* atau *NLTK*.

Pseudocode:

```
function tokenize(teks):
    pola = r"\w+|[\^w\s]"    # kata atau tanda baca
    tokens = regex_findall(pola, teks)
    return tokens

Open source hint: di Python bisa pakai re.findall()
atau nltk.word_tokenize().
```

Soal 2: Soal Embedding: Kata Mana yang Paling Mirip?

Pertanyaan:

Diberi *embedding* 2 dimensi (sederhana):

- enak = [0.8, 0.9]
- lezat = [0.78, 0.88]
- buruk = [0.1, 0.2]

a) Pasangan kata mana yang paling mirip?

b) Kenapa *embedding* lebih baik daripada kode angka urut (buku=1, pena=2, ...)?

Jawaban:

a) **enak** dan **lezat**.

b) Karena *embedding* menyimpan **kemiripan makna**. Kode urut tidak punya informasi makna.

Pembahasan (sederhana):

- *Embedding* memetakan kata menjadi vektor angka.
- Kata yang mirip makna → vektor posisinya **dekat**.
- *Cosine similarity* bisa dipakai untuk mengukur kedekatan.

Pseudocode (cek kedekatan kasar):

```
function cosine_similarity(a, b):  
    return dot(a,b) / (norm(a)*norm(b))  
  
sim_enak_lezat = cosine_similarity([0.8,0.9],[0.78,0.88])  
sim_enak_buruk = cosine_similarity([0.8,0.9],[0.1,0.2])  
sim_lezat_buruk = cosine_similarity([0.78,0.88],[0.1,0.2])  
return argmax([sim_enak_lezat, sim_enak_buruk, sim_lezat_buruk])
```

Open source hint: bisa pakai **NumPy** untuk **dot** dan **norm**.

Soal 3: Soal Attention: “Dia” yang Mana?

Pertanyaan:

Kalimat: “**Rina** memberi **Dina** buku karena dia lupa membawanya.”

Menurut *Transformer* dengan *attention*, “**dia**” lebih mungkin merujuk ke **Rina** atau **Dina**? Jelaskan singkat.

Jawaban:

Kemungkinan besar **Dina** (penerima buku) → “dia lupa membawa **buku**”.

Pembahasan (sederhana):

- *Transformer* memakai *self-attention*, melihat semua kata sekaligus.

- Kata “dia” akan memberi perhatian (bobot tinggi) ke kata-kata kunci di sekitar aksi: “memberi”, “buku”, serta ke entitas yang logis **lupa membawa buku** (penerima).
- Hasilnya, rujukan lebih konsisten ke **Dina**.

Pseudocode (sketsa perhatian):

```
tokens = ["Rina", "memberi", "Dina", "buku", "karena", "dia", "lupa", "membawanya", "."]

for i, t in enumerate(tokens):
    if t == "dia":
        weights = attention_to_all(tokens, index=i)
        # bobot besar ke ["Dina", "buku", "lupa", "membawanya"]
        referent = argmax_entity(weights)
        return referent    # -> "Dina"
```

Open source hint: konsep ini bisa didemokan dengan *attention visualization* pada model kecil di **Keras** atau via model LLM lokal (Ollama).

Soal 4: Soal Perbandingan: Bot *Keyword* vs Transformer

Pertanyaan:

Ada dua pertanyaan ke bot toko:

1. “Ada sepatu lari ukuran 42?”
2. “Sepatu buat jogging size 42 masih ready?”
Bot A pakai pencarian *keyword* sederhana. Bot B pakai *Transformer*.
Siapa yang lebih konsisten menjawab benar untuk **keduanya**? Mengapa?

Jawaban:

Bot B (Transformer) lebih konsisten benar.

Pembahasan (sederhana):

- *Keyword* sensitif terhadap kata tepat.
- *Transformer* memahami **makna** dan **parafrase** (“sepatu lari” ≈ “sepatu jogging”).
- Jadi variasi kalimat tetap dipahami.

Pseudocode (ilustrasi logika):

```
function jawab(pertanyaan, katalog, mode):
    if mode == "keyword":
        if "sepatu lari" in pertanyaan and "42" in pertanyaan:
            return stok(katalog, "sepatu lari", 42)
        else:
            return "Maaf tidak paham"
    if mode == "transformer":
        intent = infer_intent_with_transformer(pertanyaan)
```

```
size = extract_size_with_transformer(pertanyaan)
return stok(katalog, intent.kategori_sepatu, size)
```

Open source hint: untuk baseline, bisa gabungkan **scikit-learn** (TF-IDF) + klasifikasi sederhana; lalu bandingkan dengan LLM lokal via **Ollama/OpenWebUI**.

Soal 5: Soal Urutan Pipeline LLM: Susun yang Benar

Pertanyaan:

Susun langkah agar **klasifikasi komentar** bekerja dengan benar:

- A. *Transformer* melakukan klasifikasi sentimen.
- B. *Tokenisasi* memecah komentar jadi token.
- C. *Embedding* mengubah token jadi vektor angka.

Jawaban (urutan benar):

B → C → A

Pembahasan (sederhana):

- Komputer butuh bentuk angka.
- Maka urutan logis: **pecah teks (B) → jadi angka (C) → model memutuskan (A)**.

Pseudocode (ringkas):

```
function klasifikasi_komentar(teks):
    tokens = tokenize(teks)           # B
    vecs   = embed(tokens)           # C
    label  = transformer.classify(vecs,
    labels=["neg", "neu", "pos"])
                                     # A
    return label
```

Open source hint:

- Tokenisasi: **NLTK** / *regex*.
- Embedding: **scikit-learn** (TF-IDF) atau **Gensim** (Word2Vec).
- Model: **Keras** (NN sederhana) atau LLM lokal (**Ollama**).

Wrap-Up

- *Tokenisasi* = potong teks → **token**.
- *Embedding* = token → **angka bermakna**.
- *Transformer* = *attention* → **paham konteks**.

Kalau mau, saya bisa ubah paket soal ini jadi **lembar kerja (worksheet)** siap cetak + **starter notebook (.ipynb)** berisi kode Python minimal untuk tiap soal.

Fun Facts!

- Nama paper yang memperkenalkan **Transformer** adalah "**Attention is All You Need**", karena mekanisme perhatiannya adalah kunci dari teknologi ini.
- **LLaMA 3** itu punya versi yang sangat besar, lho! Bahkan ada yang ukurannya sampai puluhan GB.
- **Ollama** memudahkan kita untuk menjalankan model-model ini, yang dulu cuma bisa dijalankan di komputer super canggih.

Tips Belajar/Menghafal Cepat

- **Gambar Peta Konsep:** Gambar arsitektur LLM di buku catatanmu. Gambarkan bagaimana panah-panah menghubungkan Tokenisasi -> Embedding -> Transformer.
- **Gunakan Analogi:** Selalu ingat analogi LEGO atau merakit robot.
- **Coba Sendiri!** Tonton video tutorial di YouTube tentang cara kerja Transformer.

Aktivitas Siswa

Aktivitas 1: Tokenizer Mini + Penghitung Token

Tujuan: Membuat *tokenisasi* sederhana dan menghitung jumlah *token*.

tool: Python, **re** (bawaan), bisa lanjut **nltk** kalau mau.

Langkah:

1. Ambil satu kalimat dari kamu.
2. Pisahkan kata dan tanda baca.
3. Tampilkan daftar *token* dan jumlahnya.
4. Bandingkan hasil **split()** vs **regex**.

Pseudocode:

```
input kalimat
tokens_split = kalimat.split()
tokens_regex = regex_findall(r"\w+|[\^\w\s]", kalimat)
cetak tokens_split, len(tokens_split)
cetak tokens_regex, len(tokens_regex)
```

Python:

```
import re

kalimat = "Buku ini sangat bagus, saya suka sekali!"
tokens_split = kalimat.split()
tokens_regex = re.findall(r"\w+|[\^\w\s]", kalimat)

print("split():", tokens_split, "->", len(tokens_split))
```

```
print("regex :", tokens_regex, "->", len(tokens_regex))
```

Jawaban/Cek Hasil yang diharapkan:

- `split()` biasanya:
`['Buku', 'ini', 'sangat', 'bagus', ',', 'saya', 'suka', 'sekali!']`
(tanda baca menempel).
- *Regex* rapi:
`['Buku', 'ini', 'sangat', 'bagus', ',', ',', 'saya', 'suka', 'sekali', '!', '']`.
- Jelaskan kenapa *regex* lebih cocok untuk *tokenisasi*.

Aktivitas 2: Emoji-Aware Tokenizer (Bonus Tanda Baca)

Tujuan: Menjaga emoji/tanda baca sebagai *token* terpisah.

tool: Python, `re`.

Langkah:

1. Siapkan kalimat dengan emoji: "Mantap banget 😄, lanjut ga?".
2. Buat pola *regex* yang memisahkan kata, tanda baca, dan emoji.
3. Bandingkan dengan `split()`.

Pseudocode:

```
pola = r"[A-Za-z0-9_]+|[^\\w\\s]|[\U0001F600-\U0001F64F]"
tokens = regex_findall(pola, kalimat)
cetak tokens
```

Python:

```
import re

kalimat = "Mantap banget 😄, lanjut ga?"
pola = r"[A-Za-z0-9_]+|[^\\w\\s]|[\U0001F300-\U0001F6FF]"

tokens = re.findall(pola, kalimat)
print(tokens)
```

Jawaban/Cek Hasil:

Contoh keluaran: `['Mantap', 'banget', '😄', ',', ' ', 'lanjut', 'ga', '?']`.

Siswa paham: emoji adalah sinyal emosi → bisa pengaruh sentimen.

Aktivitas 3: Mini *Embedding*: One-Hot, TF-IDF, dan Kemiripan

Tujuan: Mengenal *embedding* dan mengukur kemiripan kalimat.

tool: Python, `pandas`, `scikit-learn` (TF-IDF).

Langkah:

1. Buat 3–5 kalimat singkat (positif/negatif/netral).
2. Ubah ke vektor TF-IDF (atau *one-hot*).
3. Hitung *cosine similarity* antar kalimat.
4. Temukan kalimat yang paling mirip.

Pseudocode:

```
input: daftar_kalimat
X = tfidf_vectorize(daftar_kalimat)
sim = cosine_similarity(X, X)
untuk tiap kalimat i:
    temukan j dengan sim terbesar (j != i)
laporkan pasangan paling mirip
```

Python:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

kalimat = [
    "Makanannya enak sekali",
    "Rasanya lezat dan enak",
    "Pelayanannya buruk",
    "Biasa saja"
]

vec = TfidfVectorizer()
X = vec.fit_transform(kalimat)
sim = cosine_similarity(X)

df = pd.DataFrame(sim, columns=range(len(kalimat)),
                  index=range(len(kalimat)))
print(df.round(2))

# Pasangan paling mirip untuk tiap baris
for i in range(len(kalimat)):
    j = sim[i].argsort()[-2] # indeks tertinggi selain diri sendiri
    print(f"Paling mirip dgn {i}: {j} | '{kalimat[i]}' ~ '{kalimat[j]}'")
```

Jawaban/Cek Hasil:

Kalimat 0 (“enak sekali”) biasanya paling mirip dengan 1 (“lezat dan enak”).
Siswa paham: *embedding* memetakan makna → bisa ukur kemiripan.

Aktivitas 4: *Attention* Mainan: Sorot Kata Penting

Tujuan: Mensimulasikan *attention* sederhana (bukan model asli), agar paham ide “fokus” ke kata penting.

tool: Python.

Langkah:

1. Ambil kalimat: "Aku mau belajar Python besok pagi di lab."
2. Tentukan kata kunci (*query*) misal: "belajar".
3. Beri skor tinggi pada kata yang dekat dengan *query* atau terkait (*Python*, *besok*).
4. Pilih 3 kata dengan skor tertinggi → itulah “fokus”.

Pseudocode:

```
tokens = tokenize(kalimat)
query = "belajar"
scores = zeros(len(tokens))
untuk i, t pada tokens:
    if t terkait query: tambah skor
    kurangi skor jika sangat jauh dari posisi query
top3 = ambil 3 skor tertinggi
cetak top3 dan kenapa dipilih
```

Python (heuristik sederhana):

```
kalimat = "Aku mau belajar Python besok pagi di lab."
tokens = kalimat.lower().split()
query = "belajar"
q_idx = tokens.index(query)

scores = []
for i, t in enumerate(tokens):
    # skor dasar: kebalikan jarak posisi
    dist = abs(i - q_idx) + 1
    s = 1 / dist
    # bonus kata terkait
    if t in ["python", "besok", "pagi", "lab"]:
        s += 0.5
    scores.append((t, round(s, 2)))

scores_sorted = sorted(scores, key=lambda x: x[1], reverse=True)
print("Attention top-3:", scores_sorted[:3])
```

Jawaban/Cek Hasil:

Top-3 biasanya: ['belajar', 'python', 'besok'/'pagi'].

Siswa paham: *attention* = memberi bobot fokus pada kata-kata kunci & konteks dekat.

Aktivitas 5: *Retriever* Mini: Jawab Pertanyaan dari Katalog Koperasi

Tujuan: Menjawab pertanyaan dengan mencari item paling relevan (mirip *retrieval* sebelum *transformer* menjawab).

tool: Python, **scikit-learn** (TF-IDF).

Langkah:

1. Buat mini katalog (nama produk + deskripsi).
2. Ambil pertanyaan user (mis. “Ada seragam ukuran L?”).
3. TF-IDF untuk katalog + pertanyaan.
4. Hitung *cosine similarity*, ambil produk paling mirip.
5. Cetak jawaban singkat.

Pseudocode:

```
katalog = [deskripsi1, deskripsi2, ...]
q = pertanyaan
X = tfidf([q] + katalog)
sim = cosine_similarity(X[0], X[1:])
idx = argmax(sim)
jawab = nama_produk[idx], info[idx]
cetak jawab
```

Python:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

produk = [
    ("Seragam Putih L", "Seragam sekolah warna putih ukuran L bahan katun"),
    ("Seragam Putih M", "Seragam sekolah warna putih ukuran M bahan katun"),
    ("Pulpen Hitam", "Pulpen tinta hitam halus untuk menulis")
]

pertanyaan = "Ada seragam ukuran L?"

docs = [pertanyaan] + [p[1] for p in produk]
vec = TfidfVectorizer()
X = vec.fit_transform(docs)

sim = cosine_similarity(X[0], X[1:]).flatten()
best = sim.argmax()
print("Pertanyaan:", pertanyaan)
print("Produk paling relevan:", produk[best][0])
```

Jawaban/Cek Hasil:

Model mini memilih **“Seragam Putih L”**.

Siswa paham: tahap *retrieval* mencari konteks → lalu *LLM/transformer* biasanya menyusun jawaban dari konteks itu.

Catatan

- *Tokenisasi* → potong teks jadi *token*.
- *Embedding* → token jadi angka bermakna (mirip = dekat).
- *Transformer + attention* → fokus, pahami konteks, jawab lebih pintar.

Rangkuman

LLM adalah **otak AI** yang paham bahasa. Jalurnya sederhana: **kata** → **tokenisasi** → **embedding** → **transformer**. *Tokenisasi* memecah kalimat jadi potongan kecil bernama **token**. *Embedding* mengubah token jadi **angka bermakna**. Kata yang mirip makna punya angka yang **dekat**. Lalu *transformer* memakai **attention** untuk **fokus** ke kata penting dan **mengerti konteks**. Hasilnya, AI bisa **menjawab dengan nyambung dan lebih pintar**.

Dengan paham **alur LLM**, kamu bukan cuma pakai AI, tapi **mengerti cara kerjanya**. Ini bekal untuk **projek nyata**: ringkas berita, filter komentar, bot produk, sampai *autocomplete*. Semua bisa dibangun dari tool **open source**: **Python, Pandas, Keras, neural network, scikit-learn, Orange DM**, dan **Ollama/OpenWebUI**. Intinya: **praktikkan urutan B → C → A** = *tokenisasi* (**B**), *embedding* (**C**), lalu *transformer* klasifikasi/generasi (**A**).

Mulai dari hal kecil. **1)** Buat *tokenizer* sederhana dengan **regex**. **2)** Coba *embedding* (TF-IDF) dan ukur **kemiripan** kalimat. **3)** Lihat ide *attention* dengan memberi **bobot** pada kata penting. **4)** Jalankan model lokal via **Ollama/OpenWebUI** untuk *summary* atau Q&A. Ingat: **attention is all you need**. Kuasai tiga kunci—**Tokenisasi, Embedding, Transformer**—dan kamu siap jadi **insinyur AI muda**!

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Tokenisasi* memecah teks menjadi potongan kecil yang disebut token.
Jawaban: Benar
2. *Embedding* adalah proses mengubah token menjadi gambar.
Jawaban: Salah
3. Pada *Transformer*, *attention* membantu model fokus pada kata penting.
Jawaban: Benar
4. *Embedding* memungkinkan kata yang mirip makna punya vektor yang berdekatan.
Jawaban: Benar
5. *Transformer* membaca kata satu per satu secara berurutan seperti RNN klasik.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Tujuan utama *tokenisasi* adalah...
 - A. Menghapus semua tanda baca
 - B. Mengubah teks jadi angka langsung
 - C. Memecah teks jadi token
 - D. Menerjemahkan bahasa asing
 Jawaban: C

2. Contoh keluaran *tokenisasi* untuk "Saya, belajar!" adalah...
 - A. ["Saya, ", "belajar!"]
 - B. ["Saya", ", ", "belajar", "!"]
 - C. ["Saya belajar"]
 - D. ["Saya", "belajar"]
 Jawaban: B

3. *Embedding* terbaik digambarkan sebagai...
 - A. Nomor urut kata
 - B. Daftar angka yang mewakili makna
 - C. Panjang kalimat
 - D. Kode ASCII
 Jawaban: B

4. Mekanisme kunci pada *Transformer* adalah...
 - A. Pooling
 - B. Convolution
 - C. *Attention*
 - D. Dropout
 Jawaban: C

5. Manakah yang **bukan** manfaat *embedding*?
 - A. Menangkap kemiripan makna kata
 - B. Mengurangi dimensi representasi
 - C. Menyimpan posisi kata otomatis
 - D. Memperbaiki performa model
 Jawaban: C

6. Urutan pipeline yang benar untuk klasifikasi teks:
 - A. *Embedding* → *Tokenisasi* → *Transformer*
 - B. *Tokenisasi* → *Embedding* → *Transformer*
 - C. *Transformer* → *Tokenisasi* → *Embedding*
 - D. *Tokenisasi* → *Transformer* → *Embedding*
 Jawaban: B

7. Pada *self-attention*, model...
 - A. Hanya melihat kata sebelumnya
 - B. Hanya melihat kata berikutnya
 - C. Melihat semua token secara bersamaan
 - D. Tidak melihat token sama sekali

Jawaban: C

8. Representasi berikut paling mungkin mirip vektor untuk kata... “lezat”:

- A. Dekat dengan “buruk”
- B. Dekat dengan “enak”
- C. Dekat dengan “pelan”
- D. Dekat dengan “meja”

Jawaban: B

9. Di bawah ini *tokenizer* sederhana di Python. Apa outputnya?

```
s = "Aku suka bakso."  
print(s.split())
```

- A. ['Aku', 'suka', 'bakso.']
- B. ['Aku', 'suka', 'bakso', '.']
- C. ['Aku suka bakso.']
- D. ['Aku_suka_bakso.']

Jawaban: A

10. Apa fungsi *positional encoding* dalam *Transformer*?

- A. Menghapus kata duplikat
- B. Memberi informasi urutan ke model
- C. Mengubah teks jadi gambar
- D. Mengurangi ukuran dataset

Jawaban: B

C. Isian Singkat (5 Soal)

1. Istilah untuk mengubah kata menjadi daftar angka disebut _____.

Jawaban: embedding

2. Mekanisme yang membuat model fokus pada kata penting disebut _____.

Jawaban: attention

3. Urutan sederhana pemrosesan LLM: tokenisasi → _____ → transformer.

Jawaban: embedding

4. Teknik untuk menangani kata baru di luar kosakata sering memakai token _____.

Jawaban:

5. Komponen yang memberi model info posisi token dalam urutan adalah _____.

Jawaban: positional encoding

D. Soal Eksplorasi (3 Soal)

Soal 1:

Rancang *tokenizer* mini yang memisahkan kata dan tanda baca untuk kalimat bertanda seru/emoji. Tulis *pseudocode* dan uji pada 2 kalimat.

Contoh jawaban eksplorasi:

Pseudocode:

```
function tok(s):  
    return regex_findall(r"\w+|[\^\w\s]|[\U0001F300-\U0001F6FF]",  
        lower(s))
```

Uji:

```
Input: "Mantap banget 😄, lanjut?" →  
["mantap", "banget", "😄", ",", "lanjut", "?"]  
Input: "Wow!!!" → ["wow", "!", "!", "!", ""]
```

Soal 2:

Bandingkan *split()* vs *regex tokenizer* pada kalimat dengan tanda baca. Tulis kode Python pendek dan simpulkan perbedaan.

Contoh jawaban eksplorasi:

Kode:

```
import re  
s = "Buku ini bagus, saya suka!"  
print("split :", s.split())  
print("regex :", re.findall(r"\w+|[\^\w\s]", s))
```

Kesimpulan: *split()* menempelkan tanda baca di kata terakhir, *regex* memisahkan tanda baca sebagai token sendiri → lebih rapi untuk *embedding*.

Soal 3:

Ukur kemiripan sederhana dua kalimat pendek dengan TF-IDF dan *cosine similarity*. Laporkan pasangan paling mirip.

Contoh jawaban eksplorasi:

Kode:

```
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.metrics.pairwise import cosine_similarity  
docs = ["Makanannya enak sekali", "Rasanya lezat", "Pelayanannya  
buruk"]  
X = TfidfVectorizer().fit_transform(docs)  
sim = cosine_similarity(X)  
pair = (0,1) # paling mirip  
print("Paling mirip:", docs[pair[0]], "~", docs[pair[1]])
```

Hasil: kalimat 0 dan 1 paling mirip; keduanya bernada positif tentang rasa.

Tugas Proyek

Proyek 1 — BPE-Lite: Subword Tokenizer mini

Tujuan: Membuat *tokenizer* sederhana ala BPE (merge pasangan huruf yang paling sering).

Data/tool: Python saja. Input bisa 5–10 kata bahasa Indonesia.

Langkah:

- Kumpulkan daftar kata (misal: makan, makanannya, minum, minumannya).
- Pecah jadi karakter + simbol akhir kata `</w>`.
- Hitung frekuensi pasangan karakter yang bersebelahan.
- *Merge* pasangan paling sering → jadi *subword* baru.
- Ulangi 5–10 kali. Pakai hasil *subword* untuk memecah kata.

Pseudocode:

```
vocab = {"makan</w>":3, "makanannya</w>":2, "minum</w>":3,
"minumannya</w>":1}
for iter in range(kali_merge):
    pairs = hitung_semua_pasangan(vocab) # contoh: ("ma","ka"),
    ("ka","n"), ...
    best = pasangan_dengan_freq_tertinggi(pairs)
    vocab = merge_pair(vocab, best) # gabungkan best jadi satu
token
tokenizer = ekstrak_daftar_subword_dari(vocab)
encode(kata) -> pecah kata dengan subword terpanjang yang cocok
```

Python (ringkas, konsep):

```
from collections import Counter

vocab = {"m a k a n </w>":3, "m a k a n a n n y a </w>":2,
        "m i n u m </w>":3, "m i n u m n y a </w>":1}

def get_stats(vocab):
    pairs = Counter()
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[(symbols[i], symbols[i+1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    bigram = ' '.join(pair)
    new_vocab = {}
    for word, freq in vocab.items():
```

```

        new_word = word.replace(bigram, ''.join(pair))
        new_vocab[new_word] = freq
    return new_vocab

for _ in range(5):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)

print("Vocab BPE-lite:")
for w in vocab:
    print(w, ":", vocab[w])

```

Kunci Jawaban (contoh):

- *Subword* yang muncul bisa seperti: **ma**, **kan**, **min**, **um**, **nya**, **</w>**.
- Kata “**makanannya**” mungkin terpecah jadi: **ma** | **kan** | **an** | **nya** | **</w>**.
- Nilai edukasi: *Subword* membantu *tokenisasi* kata baru yang belum pernah ada.

Proyek 2 — Vocab Builder + Penanganan OOV (<unk>)

Tujuan: Membangun kosakata dari korpus kecil, dengan *min_frequency* dan token **<unk>**.

Data/tool: Python + teks pendek (10–20 kalimat).

Langkah:

- *Tokenisasi* sederhana (kata + tanda baca).
- Hitung frekuensi kata.
- Simpan kata dengan frekuensi $\geq$ *min_freq* ke *vocab*; sisanya jadi **<unk>**.
- Tulis fungsi **encode()** untuk mengubah kalimat $\rightarrow$ id.

Pseudocode:

```

tokens = tokenize(korpus)
freq = hitung_frekuensi(tokens)
vocab = {w:idx for w, f in freq if f >= min_freq}
vocab["<unk>"] = id_khusus
function encode(kalimat):
    return [vocab.get(token, vocab["<unk>"]) for token in
            tokenize(kalimat)]

```

Python (singkat):

```

import re, collections

def tok(t):
    return re.findall(r"\w+|[\^\w\s]", t.lower())

```

```

korpus = ["Ini contoh kalimat.", "Kalimat kedua, contoh lagi!",
"Contoh saja."]
all_tok = []
for s in korpus:
    all_tok += tok(s)

freq = collections.Counter(all_tok)
min_freq = 2
vocab = {w:i for i, (w,f) in enumerate(freq.items()) if
f>=min_freq}
unk_id = len(vocab); vocab["<unk>"]=unk_id

def encode(s):
    return [vocab.get(w, unk_id) for w in tok(s)]

print("Vocab:", vocab)
print("Encode:", encode("Kalimat ini contoh baru.)) # kata tak
dikenal -> <unk>

```

Kunci Jawaban (contoh):

- Kata sering seperti **contoh**, **kalimat** masuk *vocab*; kata baru → **<unk>**.
- Nilai edukasi: OOV (Out-Of-Vocabulary) ditangani aman, tidak bikin error.

Proyek 3 — *Character Bigram Language Model* (Markov mini)

Tujuan: Membangun model sederhana yang memprediksi huruf berikutnya (gambaran *language model* sebelum *transformer*).

Data/tool: Python, string pendek (nama, kata Indonesia).

Langkah:

- Ambil teks pendek (misal daftar nama makanan).
- Hitung frekuensi *bigram* karakter (pasangan huruf berurutan).
- Dari suatu huruf, pilih huruf berikutnya paling sering.
- *Generate* teks 50 karakter.

Pseudocode:

```

text = gabung_semua_kata_dengan_spasi
count[char1,char2] += 1
function next_char(c):
    kandidat = count[c, *]
    pilih char2 dengan freq tertinggi
generate:
    c = mulai_dari_spasi
    for i in 1..50:
        c = next_char(c)
        tulis c

```

Python (ringkas):

```
from collections import defaultdict, Counter
import random

kata = ["bakso", "bubur", "sate", "soto", "sop", "bakmi"]
text = " ".join(kata)
pairs = defaultdict(Counter)

for a, b in zip(text, text[1:]):
    pairs[a][b] += 1

def next_char(c):
    if c not in pairs or not pairs[c]:
        return random.choice(list(pairs.keys()))
    return pairs[c].most_common(1)[0][0]

out = []
c = " "
for _ in range(60):
    c = next_char(c)
    out.append(c)

print("Generated:", "".join(out))
```

Kunci Jawaban (contoh):

- Model bisa menghasilkan urutan mirip kosakata asal: “soto sop bak...”.
- Nilai edukasi: *LM* dasar paham **pola lokal**, tapi belum paham **konteks panjang**—di sinilah *transformer* menang.

Proyek 4 — *Positional Encoding Visualizer* (Sine-Cosine)

Tujuan: Membuat *positional encoding* ala *transformer* dan melihat polanya.

Data/tool: Python, NumPy, Matplotlib.

Langkah:

- Buat fungsi `positional_encoding(max_len, d_model)` kecil (misal `max_len=32, d_model=8`).
- Isi nilai sin/cos sesuai rumus.
- Tampilkan matriks (heatmap) agar siswa melihat pola gelombang.

Pseudocode:

```
function pos_enc(L, D):
    PE = zeros(L, D)
    for pos in 0..L-1:
        for i in 0..D-1:
            if i genap:
                PE[pos,i] = sin(pos / (10000^(i/D)))
```

```

        else:
            PE[pos,i] = cos(pos / (10000^(i/D)))
    return PE
plot_heatmap(PE)

```

Python (inti):

```

import numpy as np
import matplotlib.pyplot as plt

def pos_enc(L=32, D=8):
    PE = np.zeros((L, D))
    for pos in range(L):
        for i in range(D):
            div = np.power(10000, 2*(i//2)/D)
            if i % 2 == 0:
                PE[pos, i] = np.sin(pos / div)
            else:
                PE[pos, i] = np.cos(pos / div)
    return PE

PE = pos_enc(32, 8)
plt.imshow(PE, aspect='auto')
plt.title("Positional Encoding (L=32, D=8)")
plt.xlabel("dimensi"); plt.ylabel("posisi")
plt.show()

```

Kunci Jawaban (contoh):

- Terlihat pola gelombang berbeda tiap dimensi.
- Nilai edukasi: *transformer* tahu **urutan** tanpa *RNN*, lewat sin-cos posisi.

Proyek 5 — *Token Budget Estimator* + Peringatan Panjang

Tujuan: Menghitung *jumlah token* kira-kira dan memberi peringatan jika teks terlalu panjang.

Data/tool: Python. (Pendekatan sederhana: *whitespace+punctuation tokenizer*).

Langkah:

- Tulis fungsi `tokenize_approx()` (kata + tanda baca terpisah).
- Hitung jumlah token.
- Jika > ambang (misal 256), beri peringatan: "Ringkas teks."
- (Opsional) Hitung rata-rata panjang kalimat (pakai `. ! ?`).

Pseudocode:

```

function tokenize_approx(teks):
    return regex_findall(r"\w+|[\^\w\s]", teks)
n = len(tokens)
if n > 256: print("Panjang! ringkas dulu.")

```

```
rata_kalimat = total_token / jumlah_kalimat
laporkan(n, rata_kalimat)
```

Python:

```
import re
def tokenize_approx(s):
    return re.findall(r"\w+|[\^\w\s]", s)

teks = "Tulis paragrafmu di sini. Pastikan informatif! Tambah
beberapa kalimat?"
toks = tokenize_approx(teks)
n = len(toks)

jml_kal = max(1, len(re.findall(r"[.!?]", teks)))
rata = n / jml_kal

print("Total token kira-kira:", n)
print("Rata token/kalimat:", round(rata,2))
if n > 256:
    print("Peringatan: Teks panjang. Ringkas dulu ya.")
```

Kunci Jawaban (contoh):

- Menampilkan **Total token** dan **Rata token/kalimat**.
- Nilai edukasi: paham **batas konteks** model & pentingnya **ringkas** sebelum diproses *LLM*.

Catatan

- **Proyek 1–2:** paham *tokenisasi*, *subword*, dan *OOV*.
- **Proyek 3:** lihat batas model lama (Markov) *vs transformer*.
- **Proyek 4:** kenal *positional encoding* yang bikin model paham urutan.
- **Proyek 5:** sadar *token budget*—teks yang terlalu panjang bikin model kerja berat.

BAB 3: Konsep RAG (Retriever, Index, Generator)

Pernah nggak kalian ngobrol sama AI, terus dia jawabnya ngawur atau bohong? Itu namanya **halusinasi**. Nah, di bab ini, kita bakal belajar cara bikin AI jadi super jujur dan akurat! Caranya pakai teknik yang namanya **RAG (Retrieval-Augmented Generation)**. Ini skill yang paling dicari di perusahaan, lho, karena bisa bikin AI jadi lebih pintar dari yang lain!

Tujuan Pembelajaran

- Memahami konsep **RAG** dan mengapa ia penting.
- Mengenal tiga komponen utama RAG: **Retriever, Index, dan Generator**.
- Menjelaskan bagaimana RAG membuat AI menjadi lebih akurat.
- Mengaplikasikan konsep RAG untuk membuat chatbot cerdas dari dokumen pribadi.

Peta Konsep Sederhana

Bayangin ini seperti kalian mau mengerjakan soal ujian.

- **Pertanyaan Ujian (Prompt Pengguna)**: Ini adalah pertanyaan yang kalian tanyakan ke AI.
- **Buku Pelajaran (Dokumen Pribadi)**: Ini adalah file-file (PDF, Word, TXT) yang kalian punya.
- **Retriever (Si Pencari)**: Ini adalah "otak" yang tugasnya mencari bab atau halaman yang paling relevan dari buku pelajaran kalian.
- **Index (Daftar Isi)**: Ini adalah sistem yang membuat pencarian di buku jadi super cepat.
- **Generator (Si Penulis Jawaban)**: Ini adalah model AI-nya, yang tugasnya membaca halaman yang ditemukan oleh Retriever dan menulis jawaban yang akurat.

Apersepsi: AI yang Gagal Jawab Soal Ulangan!

Kamu lagi belajar untuk ulangan fisika. Kamu buka **Ollama** dan tanya, "Jelaskan rumus gaya dalam bab 3!"

- **AI Biasa**: Dia cuma jawab rumus umum yang dia tahu dari internet, belum tentu dari bab 3 buku pelajaranmu.
- **AI dengan RAG**: Kamu sudah upload buku pelajaranmu ke sistem. Saat kamu tanya, si AI akan **mencari** (Retriever) bab 3, **membaca** (Generator) isinya, lalu **menjawab** dengan akurat sesuai buku kalian.

Nah, ini dia rahasia AI yang bisa kasih jawaban yang super spesifik dan terpercaya!

Penjelasan Teori

Kenapa Kita Butuh RAG?

Pernah nggak kalian ngobrol sama *chatbot AI* terus jawabannya ngawur?

Contoh: kalian tanya tentang *rumus gaya di bab 3 buku fisika*, tapi si AI malah kasih jawaban asal dari internet. Nah, itu namanya **halusinasi AI**.

Masalah ini bisa diselesaikan dengan teknik **RAG (Retrieval-Augmented Generation)**. Dengan RAG, AI bisa “baca” dokumen kita sendiri dulu, lalu kasih jawaban yang akurat sesuai isi dokumen itu.

Konsep RAG dengan Bahasa Sederhana

RAG bisa kita pahami sebagai kombinasi dua hal:

- **Mencari (Retrieval)**
- **Menjawab (Generation)**

Kalau AI biasa cuma mengandalkan *neural network* dari pelatihan umumnya, RAG menambahkan satu langkah penting: **AI harus cari dulu data relevan di dokumen kita**.

Tiga Komponen Penting RAG

Bayangin kalian lagi ujian, ya.

1. **Retriever (Si Pencari Pintar)**
 - Tugasnya: mencari bagian dokumen yang paling cocok dengan pertanyaan.
 - Analogi: kayak tombol *Ctrl+F* di laptop, tapi lebih pintar karena dia paham makna kalimat, bukan sekadar kata kunci.
2. **Index (Daftar Isi Super Cepat)**
 - Tugasnya: menyusun semua dokumen agar gampang dicari.
 - Analogi: seperti *daftar isi* atau katalog perpustakaan, biar kita nggak perlu baca semua buku, cukup loncat ke bagian penting.
3. **Generator (Si Penulis Jawaban)**
 - Tugasnya: membaca isi yang ditemukan Retriever, lalu menulis jawaban yang rapi dan akurat.
 - Analogi: kayak temanmu yang rajin dan bisa langsung bikin rangkuman jelas dari catatan.

Ilustrasi

- **Pertanyaan Ulangan** → “Apa itu hukum Newton?”
- **Retriever** → Cari halaman di buku fisika tentang hukum Newton.
- **Index** → Membantu Retriever supaya cepat ketemu.

- **Generator** → Menuliskan jawaban yang sesuai buku: *Hukum Newton I: benda akan tetap diam atau bergerak lurus beraturan jika tidak ada gaya.*

Kalau tanpa RAG?

AI bisa ngawur: “Newton adalah merek kue.” 😅

Kenapa RAG Penting?

- **Mengurangi Halusinasi AI** → AI nggak asal jawab, tapi berdasarkan dokumen nyata.
- **Akurat & Spesifik** → Jawaban bisa disesuaikan dengan kebutuhanmu.
- **Dipakai di Dunia Kerja** → Banyak perusahaan pakai RAG buat chatbot internal, misalnya Google, Amazon.
- **Bisa Pakai Open Source** → Python, Pandas, Keras, FAISS, atau Chroma bisa dipakai gratis untuk bikin sistem RAG.

Hubungannya dengan Machine Learning

- **Machine Learning** mengajari komputer dari data.
- **Neural Network** jadi otak AI yang bisa ngerti teks.
- **RAG** bikin AI tambah kuat, karena *neural network* ini diberi “bahan bacaan” dari dokumen terbaru.
- Hasilnya? AI jadi **pintar sekaligus jujur**.

Kasus Ilustrasi

Kasus 1: Chatbot Tata Tertib Sekolah

Kalian sering bingung aturan “bawa HP boleh nggak”?

Dengan RAG, bot jawab langsung dari dokumen resmi sekolah. Akurat. Nggak halu.

Ilustrasi singkat

- Dokumen: PDF “Tata Tertib Sekolah 2025”.
- Pertanyaan: “Apakah boleh membawa HP ke kelas?”
- *Retriever*: cari paragraf tentang HP.
- *Generator*: merangkum jadi jawaban singkat.

Pseudocode

```
# CASE 1: Tata Tertib Sekolah
docs = load("tata_tertib_sekolah.pdf")
index = build_index(docs)           # *index* pakai FAISS/Chroma
query = "Apakah boleh membawa HP ke kelas?"
hits = retriever(index, query)       # *retriever*
answer = generator(hits, query)       # *generator* (model
open-source via Ollama)
print(answer)
```

Kasus 2: Asisten Panduan Game (Boss Final)

Kalian punya *fan-made wiki* game.

Bot menjawab strategi boss final. Tepat. Sesuai teks panduan.

Ilustrasi singkat

- Dokumen: TXT/MD “Panduan Game—Strategi Boss Final”.
- Pertanyaan: “Langkah cepat kalahkan Boss X?”
- *Retriever*: ambil bagian “Boss X”.
- *Generator*: tulis langkah-langkah singkat.

Pseudocode

```
# CASE 2: Panduan Game
docs = load_folder("panduan_game/")      # .txt, .md
index = build_index(docs)
query = "Bagaimana langkah cepat mengalahkan Boss X?"
hits = retriever(index, query)
steps = generator(hits, "Rangkum jadi 5 langkah.")
print(steps)
```

Kasus 3: Chatbot Resep Keluarga (Nasi Goreng)

Resep nenek sudah di-scan? Simpan di RAG.

Tanya bahan. Bot jawab dari resep asli. Mantap buat anak kos.

Ilustrasi singkat

- Dokumen: “Resep_Nenek_NasiGoreng.pdf”.
- Pertanyaan: “Apa saja bahan nasi goreng?”
- *Retriever*: cari judul “Nasi Goreng”.
- *Generator*: daftar bahan, ringkas.

Pseudocode

```
# CASE 3: Resep Keluarga
docs = load(["Resep_Nenek_NasiGoreng.pdf", "Resep_Soto.txt"])
index = build_index(docs)
query = "Apa saja bahan untuk nasi goreng?"
hits = retriever(index, query)
answer = generator(hits, "Tampilkan daftar bahan saja.")
print(answer)
```

Kasus 4: Asisten Proyek Kelas (Deadline & Tugas)

Tim kalian punya banyak file: proposal, jadwal, hasil riset.
Bot menjawab soal *deadline* dan *scope*. Nggak perlu cari manual.

Ilustrasi singkat

- Dokumen: "Timeline_Proyek.docx", "Proposal.pdf".
- Pertanyaan: "Kapan deadline final report?"
- *Retriever*: temukan tabel jadwal.
- *Generator*: jawab tanggal + catatan singkat.

Pseudocode

```
# CASE 4: Proyek Kelas
docs = load(["Timeline_Proyek.docx", "Proposal.pdf",
            "Notulen_Rapat.txt"])
index = build_index(docs)
query = "Kapan deadline final report proyek ini?"
hits = retriever(index, query)
answer = generator(hits, "Jawab satu kalimat, sertakan tanggal.")
print(answer)
```

Kasus 5: Rangkuman Dokumen Teknis (Robotik)

Manual robotik tebal banget. Capek baca.
RAG bantu cari bagian "kalibrasi sensor" lalu rangkum langkahnya.

Ilustrasi singkat

- Dokumen: "Manual_Robotik.pdf".
- Pertanyaan: "Bagaimana langkah kalibrasi sensor jarak?"
- *Retriever*: ambil sub-bab *calibration*.
- *Generator*: 5–7 langkah terstruktur.

Pseudocode

```
# CASE 5: Rangkuman Teknis
docs = load("Manual_Robotik.pdf")
index = build_index(docs)
query = "Langkah-langkah kalibrasi sensor jarak?"
hits = retriever(index, query)
summary = generator(hits, "Rangkum sebagai langkah bernomor
(maks 7).")
print(summary)
```

Catatan

- *Index* bisa dibuat dengan **FAISS** atau **Chroma** (dua-duanya *open source*).
- *Retriever* mencari potongan teks paling relevan.
- *Generator* bisa pakai model *open source* via **Ollama** (contoh: Llama 3, Mistral).

- *Pipeline* bisa jalan di **Python**. Data bantu diolah dengan **Pandas**.
- Kalau mau gabung ke proyek AI yang lebih lanjut, bisa hubungkan ke model *neural network* atau *Keras* untuk evaluasi jawaban (opsional)

Soal Latihan & Pembahasan

Soal 1: Mengenal Komponen RAG

Pertanyaan:

Apa saja 3 komponen utama dari RAG, dan apa fungsinya?

Jawaban & Pembahasan:

- Retriever → *mencari informasi* dari dokumen.
- Index → *menyusun dokumen* biar gampang dicari.
- Generator → *menulis jawaban* berdasarkan dokumen yang ditemukan.

Pseudocode:

```
# Soal 1
komponen_RAG = ["Retriever", "Index", "Generator"]

for k in komponen_RAG:
    print(k)
```

Soal 2: Mengapa Index Penting?

Pertanyaan:

Kenapa *index* itu penting dalam sistem RAG?

Jawaban & Pembahasan:

Karena *index* membuat proses pencarian jadi cepat.

Kalau dokumen banyak dan panjang, tanpa *index* AI akan lambat mencari.

Pseudocode:

```
# Soal 2
docs = load("banyak_dokumen.pdf")
index = build_index(docs) # index membantu pencarian lebih cepat
query = "Apa itu hukum Newton?"
hasil = retriever(index, query)
print(hasil)
```

Soal 3: Bedanya AI Biasa dan AI RAG

Pertanyaan:

Jika kamu bertanya pada AI biasa “Apa resep nasi goreng ibu?”, apa kemungkinan jawabannya? Bagaimana jika pakai RAG?

Jawaban & Pembahasan:

- AI biasa: kemungkinan mengarang atau *halusinasi*.
- AI dengan RAG: mencari resep dari dokumen asli, lalu jawab sesuai data.

Pseudocode:

```
# Soal 3
query = "Apa resep nasi goreng ibu?"

# AI biasa
print("Kemungkinan jawaban ngawur...")

# AI dengan RAG
docs = load("resep_keluarga.txt")
index = build_index(docs)
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Soal 4: Urutan Proses RAG

Pertanyaan:

Lengkapi urutan proses ini:

Pertanyaan Pengguna → Retriever → Dokumen Relevan → _____ → Jawaban

Jawaban & Pembahasan:

Yang kosong adalah **Generator**.

Karena generator-lah yang membuat jawaban berdasarkan hasil pencarian.

Pseudocode:

```
# Soal 4
urutan = ["Pertanyaan Pengguna", "Retriever", "Dokumen Relevan",
"Generator", "Jawaban"]
print(" -> ".join(urutan))
```

Soal 5: Studi Kasus Proyek Kelas

Pertanyaan:

Kamu punya dokumen proyek dengan jadwal deadline.

Jika kamu tanya ke chatbot: *"Kapan deadline proyek ini?"*, bagaimana cara kerja RAG?

Jawaban & Pembahasan:

1. Index menyimpan dokumen proyek.
2. Retriever mencari bagian "Jadwal".
3. Generator membaca bagian itu dan menjawab: *"Deadline proyek tanggal 20 Desember."*

Pseudocode:

```
# Soal 5
docs = load("dokumen_proyek.pdf")
index = build_index(docs)

query = "Kapan deadline proyek ini?"
hits = retriever(index, query)
answer = generator(hits, query)

print(answer)
```

Jadi, lewat soal-soal ini, kalian bisa paham:

- RAG itu seperti tim kerja → *Retriever* cari, *Index* simpan rapi, *Generator* jawab.
- Dengan teknologi open source seperti Python, Pandas, Keras, dan neural network, kalian bisa bikin chatbot pintar yang nggak halu.

Fun Facts!

- **RAG** adalah salah satu alasan kenapa AI modern bisa menjawab pertanyaan dengan sangat akurat dan terperinci.
- Perusahaan besar seperti Google dan Amazon menggunakan RAG untuk membuat chatbot internal mereka.
- **OpenWebUI** punya fitur RAG bawaan. Kalian bisa langsung upload dokumen dan mencobanya!

Tips Belajar/Menghafal Cepat

- **Gunakan Analogi:** Selalu ingat analogi **perpustakaan** atau **mesin pencari**.

- **Praktek di OpenWebUI:** Ini cara paling gampang. Cari file PDF atau TXT di komputer kalian dan coba fitur RAG-nya.
- **Diskusi Kasus:** Coba diskusikan dengan teman, "Bagaimana kita bisa pakai RAG untuk proyek sekolah?"

Aktivitas Siswa

Aktivitas 1: Buat Dokumen Fakta Pribadi

Tujuan: Melatih penggunaan dokumen sederhana di RAG.

Langkah-langkah:

1. Buat file teks bernama `hobi.txt`.
2. Isi dengan 10 fakta tentang hobi kalian (contoh: Mobile Legends, Esports, Musik).
3. Simpan file tersebut di folder kerja kalian.
4. Upload ke OpenWebUI → fitur RAG.
5. Tanyakan pertanyaan tentang isi dokumen ke AI.

Pseudocode:

```
docs = load("hobi.txt")
index = build_index(docs)
query = "Apa fakta menarik tentang Mobile Legends?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Contoh Python:

```
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain.text_splitter import CharacterTextSplitter

# Load dokumen
with open("hobi.txt", "r") as f:
    text = f.read()

splitter = CharacterTextSplitter(chunk_size=200,
chunk_overlap=50)
chunks = splitter.split_text(text)

# Index dokumen
embeddings = HuggingFaceEmbeddings()
db = FAISS.from_texts(chunks, embeddings)

# Query
query = "Apa fakta menarik tentang Mobile Legends?"
```

```
docs = db.similarity_search(query)
print(docs[0].page_content)
```

Aktivitas 2: Chatbot Aturan Sekolah

Tujuan: Membuat bot yang menjawab pertanyaan aturan sekolah.

Langkah-langkah:

1. Cari dokumen “Tata Tertib Sekolah” dalam format **.pdf** atau **.txt**.
2. Masukkan ke dalam sistem RAG.
3. Ajukan pertanyaan: “Apakah boleh membawa HP ke kelas?”
4. AI harus menjawab dari isi dokumen, bukan karangan.

Pseudocode:

```
docs = load("tata_tertib.txt")
index = build_index(docs)
query = "Apakah boleh membawa HP ke kelas?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Aktivitas 3: Resep Keluarga Digital

Tujuan: Membuat bot untuk membaca resep keluarga.

Langkah-langkah:

1. Scan atau tulis ulang resep makanan keluarga ke file **resep.txt**.
2. Upload file ke sistem RAG.
3. Ajukan pertanyaan seperti: “Apa bahan untuk nasi goreng?”
4. AI akan membaca resep lalu jawab.

Pseudocode:

```
docs = load("resep.txt")
index = build_index(docs)
query = "Apa bahan untuk nasi goreng?"
hits = retriever(index, query)
answer = generator(hits, "Tuliskan hanya daftar bahan.")
print(answer)
```

Aktivitas 4: Proyek Kelas dengan RAG

Tujuan: Membantu kelompok memahami deadline proyek.

Langkah-langkah:

1. Buat file **proyek.txt** berisi timeline tugas.

2. Simpan dokumen ke sistem.
3. Tanyakan: "Kapan deadline final report proyek ini?"
4. Periksa apakah AI menjawab sesuai timeline.

Pseudocode:

```
docs = load("proyek.txt")
index = build_index(docs)
query = "Kapan deadline final report proyek ini?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Aktivitas 5: Rangkuman Dokumen Teknis

Tujuan: Melatih AI untuk membuat rangkuman dokumen panjang.

Langkah-langkah:

1. Buat atau unduh dokumen panjang, misalnya "Manual Robotik.pdf".
2. Upload ke sistem RAG.
3. Ajukan pertanyaan: "Bagaimana langkah kalibrasi sensor jarak?"
4. Lihat bagaimana AI merangkum isi dokumen dalam poin-poin singkat.

Pseudocode:

```
docs = load("manual_robotik.pdf")
index = build_index(docs)
query = "Bagaimana langkah kalibrasi sensor jarak?"
hits = retriever(index, query)
answer = generator(hits, "Rangkum dalam 5 langkah singkat.")
print(answer)
```

Catatan

- Lewat 5 aktivitas ini, kalian sudah **latihan langsung** dengan konsep RAG.
- Semua pakai teknologi *open source* → Python, Pandas, FAISS/Chroma, HuggingFace, OpenWebUI.
- Dengan RAG, kalian bisa bikin chatbot yang **tidak halusinasi**, tapi jawab **spesifik** dari dokumen.

Rangkuman

RAG bikin AI jadi lebih **jujur** dan **akurat**. Caranya sederhana: AI **mencari dulu** informasi yang pas, baru **menjawab**. Tiga bagiannya adalah **Retriever**, **Index**, dan **Generator**. *Retriever* itu si pencari; *Index* itu daftar isi super cepat; *Generator* itu penulis jawaban. Tanpa RAG, AI bisa **halusinasi**. Dengan RAG, AI menjawab sesuai **dokumen kita sendiri**—bukan ngarang.

Alurnya gampang diingat: **Pertanyaan → Retriever → Index bantu cepat → Potongan dokumen → Generator → Jawaban akurat**. Bayangin seperti ujian: kamu tanya, lalu cari halaman yang tepat, terus tulis jawaban singkat dan benar. Inilah *retrieval + generation* yang bekerja bareng. Hasilnya: jawaban jadi **spesifik, ringkas, dan relevan**.

Semua bisa kalian praktikkan pakai **teknologi open source**: **Python, Pandas, FAISS/Chroma** untuk *indexing*, **Ollama/OpenWebUI** untuk model, dan kalau perlu **Keras** dengan *neural network* untuk eksperimen lanjut. Intinya: **RAG = Cari dulu, simpan rapi, jawab tepat**. Kuasai tiga komponen ini, maka kalian pegang **skill penting industri** buat bikin *chatbot* kelas, rangkuman dokumen, sampai asisten belajar yang anti-*halu*.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. RAG membantu AI mengurangi *halusinasi*.
Jawaban: Benar
2. *Retriever* berfungsi menyimpan dokumen agar lebih cepat dicari.
Jawaban: Salah
3. *Index* mirip seperti daftar isi di buku.
Jawaban: Benar
4. *Generator* menulis jawaban berdasarkan informasi yang ditemukan.
Jawaban: Benar
5. AI biasa tanpa RAG bisa selalu menjawab benar.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Apa kepanjangan RAG?
 - a. Random Access Generator
 - b. Retrieval-Augmented Generation
 - c. Rapid AI Generation
 - d. Recursive AI Graph

Jawaban: b

2. Komponen RAG yang bertugas mencari dokumen relevan adalah...
- a. Index
 - b. Retriever
 - c. Generator
 - d. Prompt

Jawaban: b

3. Mengapa RAG penting?
- a. Membuat jawaban lebih cepat
 - b. Mengurangi *halusinasi* AI
 - c. Mengubah bahasa dokumen
 - d. Menambah warna tampilan

Jawaban: b

4. Index bekerja mirip dengan...
- a. Daftar isi buku
 - b. Halaman depan
 - c. Gambar ilustrasi
 - d. Sampul buku

Jawaban: a

5. Jika AI tanpa RAG ditanya resep keluarga, kemungkinan jawabannya...
- a. Sangat akurat
 - b. Bisa mengarang (*halusinasi*)
 - c. Tidak menjawab
 - d. Selalu salah

Jawaban: b

6. Generator akan membaca...
- a. Pertanyaan pengguna
 - b. Hasil pencarian *Retriever*
 - c. Isi Index
 - d. Semua dokumen sekaligus

Jawaban: b

7. Apa analogi *Retriever* yang benar?
- a. Tukang ketik jawaban
 - b. Mesin pencari pintar
 - c. Penyusun daftar isi
 - d. Pembuat soal

Jawaban: b

8. Urutan proses yang benar dalam RAG adalah...
- a. Index → Retriever → Generator → Pertanyaan
 - b. Pertanyaan → Index → Retriever → Generator → Jawaban
 - c. Pertanyaan → Retriever → Generator → Index

d. Pertanyaan → Retriever → Dokumen → Generator → Jawaban

Jawaban: d

9. OpenWebUI dengan Ollama bisa digunakan untuk...

- a. Menulis jawaban tanpa dokumen
- b. Mencoba fitur RAG dengan upload file
- c. Membuat grafik otomatis
- d. Menerjemahkan bahasa

Jawaban: b

10. Perusahaan besar seperti Google menggunakan RAG untuk...

- a. Membuat iklan
- b. Chatbot internal yang akurat
- c. Mendesain logo
- d. Mengelola server

Jawaban: b

C. Isian Singkat (5 Soal)

1. Tiga komponen utama RAG adalah _____, _____, dan _____.

Jawaban: Retriever, Index, Generator

2. Index dalam RAG berfungsi seperti _____ di dalam buku.

Jawaban: daftar isi

3. Proses RAG dimulai dari pertanyaan pengguna, lalu dilanjutkan oleh _____.

Jawaban: Retriever

4. Generator menghasilkan _____ berdasarkan dokumen relevan.

Jawaban: jawaban

5. AI tanpa RAG bisa memberi jawaban salah karena _____.

Jawaban: halusinasi

D. Soal Eksplorasi (3 Soal)

Soal 1:

Eksplorasi Chatbot Aturan Sekolah

Buatlah chatbot sederhana dengan RAG yang bisa menjawab pertanyaan aturan sekolah. Misalnya: "Apakah boleh bawa HP ke sekolah?".

Contoh Jawaban:

- Buat file `aturan.txt` berisi peraturan sekolah.
- Bangun index dari file.
- Retriever mencari bagian "HP".

- Generator menjawab: *“Menurut aturan, siswa tidak boleh membawa HP.”*

Pseudocode:

```
docs = load("aturan.txt")
index = build_index(docs)
query = "Apakah boleh bawa HP ke sekolah?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Soal 2:

Eksplorasi Kamus Gaul

Bayangkan kalian membuat chatbot RAG untuk menjelaskan bahasa gaul. Misalnya ketika ditanya “Apa arti kata *gabut*?”.

Contoh Jawaban:

- Buat file **kamus_gaul.txt** dengan daftar kata.
- Index → retriever → generator.
- Output: “Gabut = gak ada kerjaan / bosan.”

Pseudocode:

```
docs = load("kamus_gaul.txt")
index = build_index(docs)
query = "Apa arti kata gabut?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Soal 3:

Eksplorasi Rangkuman Resep

Cobalah membuat bot yang bisa merangkum resep keluarga. Misalnya: “Bagaimana cara membuat nasi goreng singkat?”.

Contoh Jawaban:

- Buat file **resep.txt** berisi resep detail.
- Retriever cari bagian nasi goreng.
- Generator meringkas jadi: “Nasi, telur, kecap, bawang. Tumis semua, sajikan.”

Python Sederhana:

```
docs = load("resep.txt")
index = build_index(docs)
query = "Bagaimana cara membuat nasi goreng singkat?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Tugas Proyek

Proyek 1: Kamus Istilah Anime

Soal:

Buat chatbot yang bisa menjawab arti istilah populer anime (misal: *senpai*, *sensei*, *shonen*).

Langkah Jawaban:

1. Buat file `kamus_anime.txt` berisi 10 istilah anime dan artinya.
2. Simpan file itu.
3. Masukkan ke sistem RAG.
4. Tanyakan: "Apa arti *senpai*?"

Pseudocode:

```
docs = load("kamus_anime.txt")
index = build_index(docs)
query = "Apa arti senpai?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Proyek 2: Jadwal Ekstrakurikuler

Soal:

Kamu diminta bikin chatbot yang bisa menjawab jadwal ekstrakurikuler di sekolah.

Langkah Jawaban:

1. Buat file `jadwal_ekskul.txt` berisi jadwal ekstrakurikuler (basket, musik, pramuka, futsal).
2. Masukkan file ke RAG.
3. Tanyakan: "Kapan ekstrakurikuler basket diadakan?"

Pseudocode:

```
docs = load("jadwal_ekskul.txt")
index = build_index(docs)
query = "Kapan ekstrakurikuler basket diadakan?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)
```

Python Sederhana:

```

with open("jadwal_ekskul.txt", "w") as f:
    f.write("Basket: Senin jam 15.00\nMusik: Rabu jam 14.00\nPramuka: Jumat jam 13.00")

query = "Kapan ekskul basket diadakan?"
# Output: "Ekskul basket diadakan hari Senin jam 15.00"

```

Proyek 3: Tips Belajar Efektif

Soal:

Buat bot yang bisa kasih tips belajar sesuai dokumen buatanmu.

Langkah Jawaban:

1. Buat file `tips_belajar.txt` berisi 8–10 tips belajar.
2. Simpan file itu.
3. Upload ke RAG.
4. Tanyakan: “Bagaimana cara belajar agar tidak cepat bosan?”

Pseudocode:

```

docs = load("tips_belajar.txt")
index = build_index(docs)
query = "Bagaimana cara belajar agar tidak cepat bosan?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)

```

Proyek 4: Panduan Instalasi Python

Soal:

Buat bot yang bisa menjawab cara instalasi Python di Windows/Linux.

Langkah Jawaban:

1. Buat file `instalasi_python.txt` berisi langkah instalasi di Windows & Linux.
2. Masukkan ke sistem RAG.
3. Tanyakan: “Bagaimana cara instalasi Python di Windows?”

Pseudocode:

```

docs = load("instalasi_python.txt")
index = build_index(docs)
query = "Bagaimana cara instalasi Python di Windows?"
hits = retriever(index, query)
answer = generator(hits, query)
print(answer)

```

Python Sederhana:

```
query = "Bagaimana cara instalasi Python di Windows?"  
# Output dari RAG:  
# 1. Download installer dari python.org  
# 2. Jalankan installer  
# 3. Centang "Add Python to PATH"  
# 4. Klik Install
```

Proyek 5: Koleksi Lirik Lagu Favorit

Soal:

Buat chatbot yang bisa menjawab potongan lirik dari lagu favorit kalian.

Langkah Jawaban:

1. Buat file `lagu_favorit.txt` berisi 5–6 potongan lirik.
2. Masukkan ke sistem RAG.
3. Tanyakan: "Apa lirik dari lagu X di bagian reff?"

Pseudocode:

```
docs = load("lagu_favorit.txt")  
index = build_index(docs)  
query = "Apa lirik lagu X di bagian reff?"  
hits = retriever(index, query)  
answer = generator(hits, query)  
print(answer)
```

Catatan

- Semua proyek di atas melatih kalian untuk **menggunakan RAG dengan data sederhana**.
- Kalian bisa pakai dokumen buatan sendiri, lalu ajak AI menjawab pertanyaan.
- Kunci keberhasilan: **buat dokumen** → **index** → **retriever** → **generator** → **jawaban akurat**.

BAB 4: Praktik RAG dengan Python + FAISS/Chroma

Siap jadi ilmuwan super? Setelah kemarin kita kenalan sama konsep **RAG**, sekarang saatnya kita praktikkan! Kita akan bangun sebuah "otak" AI yang bisa menjawab pertanyaan dari dokumen kalian sendiri. Kerennya, kita bakal pakai Python dan sebuah teknologi super cepat bernama **FAISS** atau **Chroma**. Ini adalah skill yang paling dicari di perusahaan, lho!

Tujuan Pembelajaran

- Mengimplementasikan sistem **RAG** sederhana dengan Python.
- Menggunakan **FAISS** atau **Chroma** sebagai database vektor.
- Membangun **chatbot** yang bisa menjawab pertanyaan dari dokumen pribadi.
- Menghubungkan semua materi yang sudah dipelajari (Python, LLM, RAG) dalam satu proyek.

Peta Konsep Sederhana

Bayangkan ini seperti membuat asisten perpustakaan pribadi.

- **Tumpukan Buku (Dokumen)**: Ini adalah file-file kalian (PDF, Word, TXT).
- **Meja Kerja (Python)**: Tempat kita menulis kode untuk mengatur semua.
- **Katalog Kartu Super Cepat (Chroma/FAISS)**: Ini adalah **database** yang menyimpan "makna" dari setiap kalimat dalam dokumen, jadi mencarinya super cepat.
- **Pustakawan Cerdas (LLM)**: AI kita yang akan membaca jawaban dari katalog dan memberikannya kepadamu.

Apersepsi: Kamu Mau Cari Jawaban Ulangan!

Besok ada ulangan. Kamu punya catatan materi yang banyak banget. Pusing kan kalau harus cari satu-satu? Kamu cuma ingin tahu, "Apa rumus kecepatan rata-rata?"

- **Cara Lama**: Membuka semua file satu per satu, **Ctrl+F**, lalu mencari kata **kecepatan**. Tapi kalau catatannya pakai kata **laju**, kamu tidak akan menemukannya.
- **Cara Keren (dengan RAG)**: Kamu sudah mengindeks semua catatanmu ke dalam **Chroma**. Kamu tinggal tanya, "Apa rumus **laju** rata-rata?" Si AI akan tahu kalau **laju** dan **kecepatan** itu mirip, lalu langsung memberi jawaban dari catatanmu!

Penjelasan Teori

1. Persiapan di Ubuntu 24.04

Sebelum mulai bikin *AI otak super*, kita perlu siapkan toolnya dulu. Ibarat mau masak — kamu harus punya dapurnya dulu.

Langkah Persiapan:

1. Pastikan Python Sudah Ada

Cek dengan perintah:

```
python3 --version
```

Kalau belum ada, install:

```
sudo apt install python3 python3-pip -y
```

2. Buat Folder Proyek

Biar rapi, buat satu folder khusus:

```
mkdir proyek-rag  
cd proyek-rag
```

3. Buat Virtual Environment (opsional tapi keren)

Supaya semua library terpisah:

```
python3 -m venv env  
source env/bin/activate
```

2. Instalasi *Library* FAISS dan Chroma

Sekarang waktunya install otak-otak AI kita!

Kita butuh library khusus untuk membuat database vektor.

Instalasi Chroma (lebih mudah untuk pemula)

Chroma cocok untuk proyek RAG kecil dan menengah.

```
pip install chromadb  
pip install sentence-transformers
```

Catatan: sentence-transformers digunakan untuk mengubah teks menjadi angka (embedding).

Instalasi FAISS (untuk yang suka kecepatan tinggi)

FAISS itu buatan Meta (Facebook), sangat cepat buat data besar.

```
pip install faiss-cpu  
pip install sentence-transformers
```

Kalau kamu pakai laptop dengan GPU NVIDIA dan CUDA sudah aktif:

```
pip install faiss-gpu
```

Tips: FAISS dan Chroma sama-sama bagus!

Bedanya: FAISS lebih cepat, Chroma lebih mudah dipakai.

3. Konsep Inti yang Harus Dipahami

Sebelum ngoding, yuk pahami dulu konsep penting di balik RAG.

Bayangin RAG seperti sistem “tanya-jawab cerdas”.

- **Langkah 1 – Muat Dokumen:**
Kita ambil teks dari file (catatan, PDF, atau dokumen pelajaran).
- **Langkah 2 – Ubah ke Bentuk Angka (Vektor):**
Dengan bantuan *model embedding*, kalimat diubah jadi angka.
Misalnya kalimat “CPU adalah otak komputer” jadi deretan angka seperti `[0.23, -0.55, 0.77, ...]`.
- **Langkah 3 – Simpan ke Database:**
Semua vektor ini disimpan di **FAISS** atau **Chroma**.
- **Langkah 4 – Bertanya:**
Saat kamu tanya “Apa itu CPU?”, pertanyaanmu juga diubah jadi vektor.
- **Langkah 5 – Cari yang Paling Mirip:**
FAISS/Chroma akan mencari vektor paling mirip dengan pertanyaanmu.
- **Langkah 6 – Jawab dengan Cerdas:**
Hasil pencarian dikirim ke *model AI* seperti *LLM*, lalu dijawab dengan bahasa alami.

4. Contoh Membuat RAG Sederhana dengan Python + Chroma

Kita akan bikin sistem sederhana, supaya AI bisa jawab pertanyaan dari dokumen lokal.

Langkah-langkah:

Buat file Python

```
nano rag_chroma.py
```

Tulis kode berikut:

```
import chromadb
from sentence_transformers import SentenceTransformer

# Inisialisasi Chroma
client = chromadb.Client()
collection = client.create_collection("catatan_sekolah")

# Gunakan model open source untuk ubah teks jadi vektor
model = SentenceTransformer('all-MiniLM-L6-v2')
```

```
# Dokumen contoh
docs = ["CPU adalah otak komputer",
        "GPU digunakan untuk mempercepat proses grafis",
        "RAM menyimpan data sementara"]

# Ubah jadi vektor dan simpan ke Chroma
embeddings = model.encode(docs)
collection.add(documents=docs, embeddings=embeddings, ids=["1", "2",
"3"])

# Query pengguna
query = "fungsi dari GPU"
query_vector = model.encode([query])
hasil = collection.query(query_embeddings=query_vector, n_results=1)

print("Pertanyaan:", query)
print("Jawaban:", hasil['documents'][0])
```

Jalankan program

```
python3 rag_chroma.py
```

Hasilnya:

```
Pertanyaan: fungsi dari GPU
Jawaban: ['GPU digunakan untuk mempercepat proses grafis']
```

AI-mu berhasil menjawab dari dokumen sendiri!

5. Versi Cepat dengan FAISS

Kalau kamu ingin coba FAISS, berikut versi super singkatnya:

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# Dokumen
docs = ["AI membantu manusia", "Python adalah bahasa pemrograman",
        "Data penting untuk AI"]
model = SentenceTransformer('all-MiniLM-L6-v2')
vectors = model.encode(docs)

# Buat index FAISS
dimension = vectors.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(vectors)

# Query
query = "bahasa yang dipakai AI"
q_vector = model.encode([query])
```

```
D, I = index.search(q_vector, k=1)
print("Pertanyaan:", query)
print("Jawaban:", docs[I[0][0]])
```

FAISS lebih cepat, tapi sedikit lebih teknis dibanding Chroma.

6. Seru, Bukan?

Sekarang kamu sudah bisa bikin *AI kecil* sendiri di laptop kamu.

Semua pakai **tool open source** — Python, FAISS, Chroma, Pandas, dan Keras.

Dengan langkah-langkah sederhana, kamu sudah belajar konsep dasar **AI Retrieval** yang digunakan oleh perusahaan besar seperti Google, Meta, dan OpenAI!

Rangkuman

- Gunakan **Python** di Ubuntu 24.04 sebagai platform utama.
- Install *library* open source: **chromadb**, **faiss**, dan **sentence-transformers**.
- Ubah teks jadi angka (*vektor*) supaya komputer bisa memahami maknanya.
- Simpan dan cari vektor dengan **Chroma** atau **FAISS**.
- Bangun sistem RAG yang bisa menjawab pertanyaan dari dokumen sendiri.

Tantangan untuk Siswa SMK

Coba buat RAG yang bisa menjawab pertanyaan dari **dokumen kurikulum SMK**.

Misalnya:

“Berapa jam pelajaran produktif kelas XI TKJ?”

“Apa tujuan mata pelajaran Jaringan Komputer?”

Kalau berhasil, berarti kamu sudah naik level dari *siswa biasa* jadi **AI Engineer muda**!

Kasus Ilustrasi

Kasus 1: Chatbot Aturan Sekolah

Ilustrasi: Kamu sering bingung “Boleh bawa HP gak?” atau “Seragam hari Jumat apa?”. Daripada buka PDF panjang, bikin **chatbot** yang jawab dari dokumen tata tertib.

Tujuan: Menjawab pertanyaan siswa sesuai isi dokumen sekolah.

Bahan: PDF/TXT tata tertib → diekstrak jadi teks.

Pseudocode (Chroma):

```
# PSEUDOCODE: Chatbot Aturan Sekolah (Chroma)
mulai
    impor python_lib: chroma, sentence_transformers, pandas
    # bisa hasil ekstrak PDF
```

```

dokumen = baca_berkas("tatatertib.txt")
potong_dokumen_per_kalimat(dokumen) -> kalimat_list

model_embed = load_model("all-MiniLM-L6-v2")    # open source
embedding
vektor_kalimat = model_embed.encode(kalimat_list)

chroma = buat_client_chroma()
koleksi = chroma.buat_koleksi(nama="aturan_sekolah")
koleksi.tambah(documents=kalimat_list,
embeddings=vektor_kalimat, ids=auto_nomor())

loop:
    pertanyaan = input_pengguna()
    vektor_tanya = model_embed.encode(pertanyaan)
    hasil = koleksi.query(embedding=vektor_tanya, n=3)    # ambil
3 paling mirip

    jawaban = rangkum_dari(hasil.documents)    # pakai logika
sederhana / *LLM open source*
    cetak(jawaban)
selesai

```

Catatan: *Retriever* = Chroma, *Generator* bisa sederhana (merangkai kalimat) atau *LLM open source* kecil.

Kasus 2: Personal Study Assistant (Ringkas Catatan)

Ilustrasi: Menjelang ulangan, kamu mau tanya: “Inti Bab 3 apa?” AI mengutip dari catatanmu.

Tujuan: Ringkas materi dari catatan pelajaran pribadi.

Bahan: TXT/CSV catatan pelajaran, bisa diolah dengan **pandas**.

Pseudocode (FAISS):

```

# PSEUDOCODE: Study Assistant (FAISS)
mulai
    impor python_lib: faiss, sentence_transformers, pandas
    data = pandas.read_csv("catatan_pelajaran.csv")    # kolom:
"kalimat"
    kalimat_list = data["kalimat"].to_list()

    model_embed = load_model("all-MiniLM-L6-v2")
    mat_vektor = model_embed.encode(kalimat_list)    # bentuk: [n,
dim]

    index = faiss.IndexFlatL2(dimensi(mat_vektor))
    index.add(mat_vektor)

    loop:
        pertanyaan = input_pengguna()

```

```

v_tanya = model_embed.encode([pertanyaan])
jarak, idx = index.search(v_tanya, k=5)

kandidat = ambil(kalimat_list, idx)
ringkasan = ringkas(kandidat) # logika ringkas/*LLM open
source*
cetak(ringkasan)
selesai

```

Tip: Kalau materi panjang, simpan juga metadata (bab/halaman) supaya kutipan jelas.

Kasus 3: Pencari Resep Keluarga (Rendang, Soto, dst.)

Ilustrasi: Kamu punya kumpulan resep keluarga. Tanyakan “Bahan rendang apa?” AI ambil dari dokumen resep sendiri.

Tujuan: Menemukan daftar bahan/step dari dokumen resep.

Bahan: Folder TXT/MD resep. Bisa parsing dengan Python.

Pseudocode (Chroma + *prompting* sederhana):

```

# PSEUDOCODE: Resep Keluarga (Chroma)
mulai
    impor python_lib: chroma, sentence_transformers, glob, os
    file_list = glob("resep/*.txt")
    semua_kalimat = []

    untuk setiap file dalam file_list:
        teks = baca_berkas(file)
        kalimat = pecah_per_kalimat(teks)
        semua_kalimat.tambah(kalimat)

    model = load_model("all-MiniLM-L6-v2")
    emb = model.encode(semua_kalimat)

    db = buat_client_chroma()
    col = db.buat_koleksi("resep_keluarga")
    col.tambah(documents=semua_kalimat, embeddings=emb,
ids=auto_nomor())

    pertanyaan = "Apa bahan untuk rendang?"
    vq = model.encode([pertanyaan])
    hasil = col.query(query_embeddings=vq, n_results=5)

    jawaban = format_list_bahan(hasil.documents)    # ekstrak
*keyword* bahan
    cetak(jawaban)
selesai

```

Trik: Tambahkan fungsi `format_list_bahan()` yang mencari pola “Bahan:” atau bullet list.

Kasus 4: Bot Panduan Game (Tips & Item Langka)

Ilustrasi: Kamu simpan panduan *game* (build, lokasi item, boss). Tanyakan “Cara dapat item X?” AI mengutip dari panduanmu.

Tujuan: Q&A cepat dari koleksi panduan.

Bahan: File TXT/HTML yang sudah dibersihkan jadi teks.

Pseudocode (FAISS, fokus *retrieval* cepat):

```
# PSEUDOCODE: Panduan Game (FAISS)
mulai
    impor python_lib: faiss, sentence_transformers
    panduan = baca_berkas("panduan_game_bersih.txt")
    potongan = split_paragraf(panduan) # potong per paragraf

    model = load_model("all-MiniLM-L6-v2")
    V = model.encode(potongan)

    idx = faiss.IndexFlatIP(dimensi(V)) # IP = inner product (cocok untuk *cosine*
normalisasi)
    V_norm = normalisasi(V) # opsional: normalisasi untuk *cosine similarity*
    idx.add(V_norm)

    tanya = input("Pertanyaan game: ")
    q = model.encode([tanya])
    q_norm = normalisasi(q)
    D, I = idx.search(q_norm, k=3)

    kandidat = ambil(potongan, I)
    jawaban = gabungkan_dengan_kutipan(kandidat) # tampilkan paragraf + sumber
baris
    cetak(jawaban)
selesai
```

Upgrade: Simpan *tag* (map, boss, item) di **pandas DataFrame** untuk filter cepat sebelum *retrieval*.

Kasus 5: Penulis Laporan Otomatis Proyek (Ringkas Data Mentah)

Ilustrasi: Timmu punya catatan harian proyek (CSV). Kamu ingin tanya: “Ringkas progres minggu ini”.

Tujuan: Ambil paragraf relevan, lalu buat ringkasan pendek.

Bahan: CSV/Markdown laporan harian.

Pseudocode (Chroma + pandas + *template* sederhana):

```
# PSEUDOCODE: Laporan Otomatis (Chroma)
mulai
    impor python_lib: pandas, chroma, sentence_transformers
    df = pandas.read_csv("log_proyek.csv") # kolom:
tanggal, ringkasan, isu, aksi
```

```

    kalimat = gabung_ke_paragraf(df)                                # satukan jadi
    paragraf-paragraf

    embedder = load_model("all-MiniLM-L6-v2")
    E = embedder.encode(kalimat)

    c = buat_client_chroma()
    col = c.buat_koleksi("log_proyek")
    col.tambah(documents=kalimat, embeddings=E, ids=auto_nomor())

    pertanyaan = "Ringkas progres minggu ini dan isu utama"
    q = embedder.encode([pertanyaan])
    hits = col.query(query_embeddings=q, n_results=6)

    poin = ekstrak_poin(hits.documents)                             # ambil bullet
    penting

    laporan = format_template("""
        # Laporan Progres (Minggu Ini)
        - Pekerjaan utama: {p1}
        - Isu kritis: {p2}
        - Aksi berikutnya: {p3}
    """, poin)

    cetak(laporan)
    selesai

```

Catatan: Untuk *generator*, cukup *template filling* dulu. Kalau ingin lebih natural, pakai *LLM open source* kecil.

Tips

- Mulai dari **1–2 dokumen** dulu.
- Gunakan *print()* di tiap tahap: *load* → *embed* → *index* → *query*.
- Simpan **ID dan metadata** (judul, sumber) agar jawaban punya **kutipan**.
- Ingat konsep: *retriever* (cari) dan *generator* (menyusun jawaban).
- *Benchmark* kecil: coba ganti **Chroma** ↔ **FAISS**, bandingkan kecepatan & kemudahan.

Mini-Checklist Teknis (Ubuntu 24.04)

- `python3 --version` → pastikan siap.
- (Opsional) `python3 -m venv env && source env/bin/activate`
- **Chroma:** `pip install chromadb sentence-transformers`
- **FAISS (CPU):** `pip install faiss-cpu sentence-transformers`
- **pandas:** `pip install pandas`
- (Opsional) **Keras/TensorFlow** untuk eksperimen *neural network*: `pip install keras tensorflow`

Dengan 5 kasus ini, kamu sudah “merakit” *RAG mini* yang beneran siap dipakai: **chatbot sekolah, asisten belajar, resep keluarga, bot panduan game, dan penulis laporan otomatis**. Langkahnya jelas, toolnya *open source*, dan kamu tinggal eksekusi.

Soal Latihan & Pembahasan

Soal 1: Urutan Pipeline RAG yang Benar

Soal (konsep):

Kamu membangun RAG. Urutkan langkah berikut agar sistem bekerja benar:

- A. Simpan vektor ke **Chroma/FAISS** (*indexing*)
- B. Ubah dokumen & pertanyaan jadi *embedding*
- C. Ambil dokumen mirip (*retriever*) lalu susun jawaban (*generator*)
- D. Muat dokumen dari file

Jawaban: D → B → A → C

Pembahasan (singkat):

- Muat data dulu (D).
- Ubah teks ke angka (*embedding*) (B).
- Simpan vektor ke *vector database* (A).
- Saat tanya, *retriever* ambil yang mirip, *generator* susun jawaban (C).

Pseudocode:

```
mulai
    dokumen = load_files()
    emb_doc = embed(dokumen)                # B
    index = create_vector_db()
    index.add(emb_doc)                      # A
    pertanyaan = input_user()
    emb_q = embed([pertanyaan])             # B (untuk query)
    kandidat = index.search(emb_q, k=3)      # C (retriever)
    jawaban = generate_answer(kandidat)     # C (generator)
    cetak(jawaban)
selesai
```

Soal 2: Perbaiki *Bug*: “Tidak Ada *Embedding* Saat Menambah ke Chroma”

Soal (debug):

Kode kamu menambah dokumen ke Chroma tanpa *embedding*, lalu *query* selalu kosong. Apa perbaikan minimal?

Jawaban:

Hitung *embedding* dengan model lalu kirim `embeddings=` saat `collection.add(...)`.

Pembahasan:

Chroma/FAISS bekerja dengan angka (*embedding*). Tanpa itu, *similarity search* gagal.

Pseudocode (sebelum & sesudah):

```
# SALAH (sebelum)
collection.add(documents=docs, ids=auto_id())

# BENAR (sesudah)
model = load_model("all-MiniLM-L6-v2")
emb = model.encode(docs)
collection.add(documents=docs, embeddings=emb, ids=auto_id())
```

Soal 3: FAISS Error: “Dimension Mismatch”**Soal (teknis):**

Kamu buat FAISS index dengan dimensi 384, tapi model *embedding* menghasilkan vektor dimensi 768. Apa solusinya?

Jawaban:

Samakan dimensi: buat index FAISS dengan `dimension = vectors.shape[1]` dari hasil *embedding*.

Pembahasan:

FAISS butuh dimensi index = dimensi vektor. Ambil dari data supaya konsisten.

Pseudocode:

```
vectors = embed(dokumen)                # shape: [N, D]
D = shape(vectors)[1]                   # ambil dimensi
sebenarnya
index = faiss.IndexFlatL2(D)             # buat index sesuai D
index.add(vectors)
q = embed([pertanyaan])
dist, idx = index.search(q, k=3)
```

Soal 4: Pilih top-k yang Tepat untuk Retriever**Soal (desain):**

Kamu bingung pilih *k* saat mencari kemiripan. Jika *k* terlalu kecil → jawaban kurang konteks. Jika *k* terlalu besar → jawaban melebar/berisik. Untuk dokumen ringkas per topik, berapa *k* awal yang aman?

Jawaban:

Mulai dari *k* = 3, lalu uji (tuning) sesuai kualitas dokumen.

Pembahasan:

k=3 sering cukup memberi konteks tanpa terlalu bising. Nanti bisa naikturunkan sesuai evaluasi.

Pseudocode (tuning sederhana):

```
k = 3
loop percobaan dari daftar_k = [2,3,5]:
    hasil = index.search(emb_q, k)
    skor = evaluasi(hasil, metrik="relevansi")
    simpan(k, skor)
k_terbaik = argmax(skor)
pakai(k_terbaik)
```

Soal 5: Tambah Kutipan & *Metadata* agar Jawaban Terpercaya

Soal (praktik baik):

Jawaban chatbot kamu benar, tapi guru ingin lihat sumber (file & baris). Bagaimana menyimpan dan menampilkan *metadata*?

Jawaban:

Simpan **ids + metadata** (misal: `{"file": "aturan.pdf", "hal": 12}`) saat *indexing*, lalu tampilkan saat *retrieval*.

Pembahasan:

Transparansi penting. *Metadata* membuat jawaban bisa diaudit dan dipelajari ulang.

Pseudocode:

```
# saat indexing
for tiap potongan in potong(dokumen):
    emb = embed([potongan.teks])
    collection.add(
        documents=[potongan.teks],
        embeddings=emb,
        ids=[potongan.id],
        metadatas=[{"file": potongan.file, "hal": potongan.hal}]
    )

# saat menjawab
hasil = collection.query(query_embeddings=emb_q, n_results=3)
for i in range(len(hasil.documents)):
    tampilkan(hasil.documents[i])
    tampilkan("Sumber:", hasil.metadatas[i]["file"], "Hal:",
        hasil.metadatas[i]["hal"])
gabungkan_jadi_jawaban(hasil.documents, kutipan=hasil.metadatas)
```

Catatan

- RAG butuh urutan rapi: **Load** → **Embed** → **Index** → **Retrieve** → **Generate**.
- Chroma/FAISS wajib pakai *embedding* (angka), bukan teks mentah.
- Dimensi FAISS harus sama dengan dimensi vektor dari model.
- Mulai **top-k=3**, lalu *tuning*.
- Simpan **ID + metadata** supaya jawaban punya **kutipan** yang jelas.

Fun Facts!

- **FAISS** adalah library yang dibuat oleh **Meta** (perusahaan Facebook) untuk melakukan pencarian vektor yang super cepat.
- Teknologi **RAG** adalah solusi paling ampuh untuk melawan **halusinasi** AI. Banyak perusahaan menggunakannya untuk membuat AI yang bisa dipercaya.
- **ChromaDB** itu bisa jalan di laptop kita dan juga di internet. Jadi, bisa dipakai buat proyek kecil dan proyek besar.

Tips Belajar atau Menghafal Cepat

- **Pikirkan Analogi**: Selalu ingat analogi **katalog perpustakaan**.
- **Mulai dari Satu Dokumen**: Jangan langsung indeks ratusan dokumen. Mulai dari satu file teks pendek dulu.
- **Gunakan `print()`**: Selalu gunakan `print()` di setiap langkah untuk melihat hasilnya.

Aktivitas Siswa

Aktivitas 1 — “Hello Retrieval!” (Chroma paling dasar)

Goal: Bisa *load* → *embed* → *index* → *query* dokumen pendek.

Apa yang kamu pelajari: Alur RAG dasar dan cara kerja *vector search*.

Langkah/Tahap

- Buat folder proyek.
- Siapkan 3–5 kalimat pengetahuan dasar (TXT langsung di kode).
- Simpan ke **Chroma**, lalu tanya 1 pertanyaan.
- Lihat dokumen mana yang paling mirip.

Pseudocode

```
mulai
    siapkan_kalimat = ["CPU itu otak komputer", "GPU mempercepat
    grafis", "RAM menyimpan data sementara"]
    model = load_embedding_model("all-MiniLM-L6-v2")
    emb = model.encode(siapkan_kalimat)

    chroma = buat_client_chroma()
    col = chroma.buat_koleksi("hello_retrieval")
    col.tambah(documents=siapkan_kalimat, embeddings=emb,
    ids=auto_nomor())

    tanya = "fungsi GPU?"
    emb_q = model.encode([tanya])
    hasil = col.query(query_embeddings=emb_q, n_results=1)
    tampilkan(hasil.documents, hasil.ids)
```

selesai

Contoh Python

```
# aktivitas1_hello_chroma.py
import chromadb
from sentence_transformers import SentenceTransformer

docs = [
    "CPU itu otak komputer",
    "GPU mempercepat grafis",
    "RAM menyimpan data sementara"
]

model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(docs)

client = chromadb.Client()
col = client.create_collection("hello_retrieval")
col.add(documents=docs, embeddings=emb, ids=["1", "2", "3"])

query = "fungsi GPU?"
q_emb = model.encode([query])
res = col.query(query_embeddings=q_emb, n_results=1)

print("Pertanyaan :", query)
print("Dokumen top:", res["documents"][0][0], "| ID:",
      res["ids"][0][0])
```

Aktivitas 2 — “Catatan Belajar Pintar” (pandas + Chroma + sinonim)

Goal: Index catatan dari **CSV** dan uji *synonym search* (contoh: “laju” ~ “kecepatan”).

Apa yang kamu pelajari: Integrasi **pandas** + *embedding*, bukti bahwa RAG melihat *makna*.

Langkah/Tahap

- Buat **catatan.csv** (kolom: **kalimat**).
- Baca dengan **pandas**.
- Index ke **Chroma**.
- Coba dua pertanyaan: “rumus *kecepatan* rata-rata?” dan “rumus *laju* rata-rata?”.

Pseudocode

```
mulai
    data = pandas.read_csv("catatan.csv")
    kalimat = data["kalimat"].to_list()
    model = load_embedding_model("all-MiniLM-L6-v2")
    emb = model.encode(kalimat)

    col = chroma.buat_koleksi("catatan_pelajaran")
```

```

col.tambah(documents=kalimat, embeddings=emb,
ids=auto_nomor())

untuk tanya in ["rumus kecepatan rata-rata", "rumus laju
rata-rata"]:
    q = model.encode([tanya])
    top = col.query(query_embeddings=q, n_results=2)
    tampilkan(tanya, top.documents)
selesai

```

Contoh Python

```

# aktivitas2_catatan_csv_chroma.py
import chromadb, pandas as pd
from sentence_transformers import SentenceTransformer

df = pd.DataFrame({"kalimat": [
    "Rumus kecepatan rata-rata adalah jarak dibagi waktu",
    "Laju rata-rata sering disamakan dengan kecepatan
rata-rata",
    "Percepatan adalah perubahan kecepatan per satuan waktu"
]})
df.to_csv("catatan.csv", index=False)

data = pd.read_csv("catatan.csv")
docs = data["kalimat"].tolist()

model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(docs)

client = chromadb.Client()
col = client.create_collection("catatan_pelajaran")
col.add(documents=docs, embeddings=emb, ids=[str(i) for i in
range(len(docs))])

for q in ["rumus kecepatan rata-rata", "rumus laju rata-rata"]:
    qe = model.encode([q])
    res = col.query(query_embeddings=qe, n_results=2)
    print("\nQ:", q)
    for d in res["documents"][0]:
        print("-", d)

```

Aktivitas 3 — “FAISS Cepat vs k-Value” (bandingkan k=1,3,5)

Goal: Pakai **FAISS** dan uji kualitas hasil dengan *top-k* berbeda.

Apa yang kamu pelajari: *Retrieval tuning* sederhana.

Langkah/Tahap

- Siapkan 6–8 kalimat terkait satu topik.
- Buat **FAISS index** (L2).
- Coba **k = 1, 3, 5** untuk pertanyaan yang sama.
- Bandingkan: mana yang paling relevan?

Pseudocode

```
mulai
    docs = kumpulkan_kalimat()
    model = load_embedding_model()
    V = model.encode(docs)
    D = dimensi(V)
    index = faiss.IndexFlatL2(D)
    index.add(V)

    tanya = "definisi kecepatan rata-rata"
    vq = model.encode([tanya])

    untuk k in [1,3,5]:
        Dm, Im = index.search(vq, k)
        kandidat = pilih(docs, Im[0])
        tampilkan(k, kandidat)
selesai
```

Contoh Python

```
# aktivitas3_faiss_k_compare.py
import faiss
from sentence_transformers import SentenceTransformer
import numpy as np

docs = [
    "Kecepatan rata-rata adalah jarak dibagi waktu tempuh",
    "Laju sering digunakan sebagai sinonim kecepatan",
    "Kecepatan sesaat adalah kecepatan pada satu waktu tertentu",
    "Percepatan adalah perubahan kecepatan per waktu",
    "Jarak = kecepatan x waktu",
    "Waktu = jarak / kecepatan"
]

model = SentenceTransformer("all-MiniLM-L6-v2")
V = model.encode(docs).astype("float32")
D = V.shape[1]
index = faiss.IndexFlatL2(D)
index.add(V)

q = "definisi kecepatan rata-rata"
vq = model.encode([q]).astype("float32")

for k in [1,3,5]:
    dist, idx = index.search(vq, k)
    print(f"\nTop-{k}:")
    for i in idx[0]:
        print("-", docs[i])
```

Aktivitas 4 — “Jawaban + Kutipan Sumber” (metadata di Chroma)

Goal: Tampilkan jawaban beserta **sumber** (file/halaman).

Apa yang kamu pelajari: Simpan **metadata** saat *indexing* untuk *auditability*.

Langkah/Tahap

- Buat 3 potongan teks dari 2 “file” berbeda (simulasi).
- Saat **add**, isi **metadatas**=[{"file": "aturan.pdf", "hal": 12}, ...].
- Saat **query**, tampilkan jawaban + sumber.

Pseudocode

```
mulai
    docs = ["Boleh bawa HP saat jam istirahat", "Seragam Jumat
    adalah batik", "Upacara hari Senin wajib"]
    meta = [{"file": "aturan.pdf", "hal": 10},
            {"file": "aturan.pdf", "hal": 12},
            {"file": "agenda.pdf", "hal": 4}]
    emb = embed(docs)

    col.add(documents=docs, embeddings=emb, ids=auto_id(),
    metadatas=meta)

    tanya = "kapan boleh bawa HP?"
    q = embed([tanya])
    hits = col.query(query_embeddings=q, n_results=2)

    tampilkan(hits.documents, hits.metadatas) # sertakan sumber
selesai
```

Contoh Python

```
# aktivitas4_chroma_metadata.py
import chromadb
from sentence_transformers import SentenceTransformer

docs = [
    "Boleh bawa HP saat jam istirahat",
    "Seragam hari Jumat adalah batik",
    "Upacara hari Senin wajib diikuti"
]
metas = [
    {"file": "aturan.pdf", "hal": 10},
    {"file": "aturan.pdf", "hal": 12},
    {"file": "agenda.pdf", "hal": 4}
]

model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(docs)

client = chromadb.Client()
col = client.create_collection("aturan_sumber")
```

```
col.add(documents=docs, embeddings=emb, ids=["a", "b", "c"],
        metadatas=metas)

q = "kapan boleh bawa HP?"
qe = model.encode([q])
res = col.query(query_embeddings=qe, n_results=2)

print("Pertanyaan:", q)
for d, m in zip(res["documents"][0], res["metadatas"][0]):
    print("-", d, "| Sumber:", m["file"], "Hal:", m["hal"])
```

Aktivitas 5 — “Mini Chatbot Loop + Rerank Sederhana”

Goal: Buat *chat loop* kecil: tanya → ambil 5 dokumen → *rerank* cepat → tampilkan 2 terbaik + kutipan.

Apa yang kamu pelajari: Alur chatbot mini, *reranking* skor sederhana (tanpa model besar).

Catatan: Sebutkan *neural network/Keras* sebagai opsi lanjut (opsional).

Langkah/Tahap

- Index dokumen ringkas.
- Saat *query*, ambil **top-5**.
- *Rerank* manual: skor +1 jika mengandung kata kunci, atau hitung *cosine* lagi.
- Tampilkan 2 terbaik.
- **Opsional lanjut:** pakai *neural network* kecil (Keras) untuk *learn-to-rank* (tidak wajib).

Pseudocode

```
mulai
    index_chroma(dokumen)
    loop:
        tanya = input()
        kandidat5 = retrieve_top_k(tanya, k=5)
        skor = rerank_sederhana(kandidat5, kata_kunci=tokens(tanya))
        top2 = pilih_berdasarkan_skor(kandidat5, skor, n=2)
        tampilkan_jawaban(top2, sertakan_sumber=True)
        selesai_jika("quit")
    selesai
```

Contoh Python

```
# aktivitas5_chat_loop_rerank.py
import chromadb, re
from sentence_transformers import SentenceTransformer

docs = [
    "Prakerin kelas XI dimulai bulan Januari",
    "Jadwal upacara hari Senin pukul 07.00",
    "Jam istirahat pukul 10.00 dan 12.00",
    "Peraturan membawa HP hanya saat guru mengizinkan",
    "Kegiatan ekstrakurikuler dimulai pukul 15.30"
```

```

]
metas = [
    {"file": "info_sekolah.txt", "baris": 1},
    {"file": "info_sekolah.txt", "baris": 2},
    {"file": "info_sekolah.txt", "baris": 3},
    {"file": "tata_tertib.txt", "baris": 14},
    {"file": "ekskul.txt", "baris": 5}
]

model = SentenceTransformer("all-MiniLM-L6-v2")
emb = model.encode(docs)

client = chromadb.Client()
col = client.create_collection("chatbot_sekolah")
col.add(documents=docs, embeddings=emb, ids=[str(i) for i in
range(len(docs))], metadatas=metas)

def rerank_simple(candidates, query):
    # skor +1 untuk setiap kata kunci yang muncul
    tokens = re.findall(r"\w+", query.lower())
    scored = []
    for text, meta in candidates:
        s = 0
        low = text.lower()
        for t in tokens:
            if t in low:
                s += 1
        scored.append((s, text, meta))
    scored.sort(reverse=True, key=lambda x: x[0])
    return scored

print("Ketik pertanyaan (ketik 'quit' untuk selesai)")
while True:
    q = input("Q> ").strip()
    if q.lower() == "quit":
        break
    qe = model.encode([q])
    res = col.query(query_embeddings=qe, n_results=5)

    cand = list(zip(res["documents"][0], res["metadatas"][0]))
    reranked = rerank_simple(cand, q)[:2]

    print("- Jawaban Kandidat -")
    for score, text, meta in reranked:
        print(f"[skor {score}] {text} | Sumber: {meta['file']}
        baris {meta['baris']}")
    print()

```

Opsional Lanjut (sebutan neural network / Keras):

Kamu bisa melatih *reranker* kecil berbasis **Keras** dengan fitur sederhana (misal jumlah kecocokan kata, panjang teks, dll.). Ini *machine learning* dasar

untuk *learn-to-rank*. Tidak wajib untuk kelas ini, tapi seru untuk eksperimen lanjut.

Mini-Checklist

- `pip install chromadb faiss-cpu sentence-transformers pandas`
- Model *embedding*: "`all-MiniLM-L6-v2`" (open source, ringan).
- Simpan **metadata** untuk kutipan.
- *Debug tips*: cek dimensi FAISS, cek *embedding* saat `add`, dan tes `k` berbeda.

Rangkuman

Gambaran Besar. Kita membangun *RAG (Retriever-Augmented Generation)* agar AI menjawab dari dokumen kita sendiri, bukan asal tebak. Intinya: **teks diubah jadi angka** yang disebut *vector* memakai *embedding*. Lalu *vector* disimpan di **Chroma** atau **FAISS** supaya **pencarian makna jadi super cepat**. Saat kita bertanya, sistem **menemukan potongan paling mirip** lalu dirangkai jadi jawaban. Semua ini jalan di **Ubuntu 24.04** dengan **Python** dan tool *open source*.

Alur Kerja Inti. Urutan kerja yang rapi itu kunci: **muat dokumen** → **buat *embedding*** → **simpan di *vector database*** → ***retrieve*** → ***generate***. *Retriever* bertugas **mencari potongan paling relevan**, sedangkan *generator* **menyusun jawaban yang enak dibaca**. Kita bisa tambah **metadata & kutipan sumber** agar jawaban **terpercaya**. Untuk olah data tabel, pakai **pandas**. Untuk eksperimen model kecil, bisa coba *neural network* dengan **Keras** (*machine learning* tingkat lanjut, opsional).

Nilai Praktis. **FAISS cepat**, cocok saat data makin banyak; **Chroma mudah**, enak untuk mulai dan prototipe. Mulailah kecil: **1–2 dokumen dulu, uji *top-k* (mis. $k=3$)**, dan cek hasil pakai `print()` agar paham alurnya. **Simpan ID + sumber** supaya gampang diaudit dan diperbaiki. Dengan langkah sederhana ini, kamu sudah punya **asisten belajar pribadi yang akurat, cepat, dan aman**—semua pakai tool *open source* di laptopmu.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. RAG memakai *retriever* untuk mencari potongan teks yang relevan.
Jawaban: Benar
2. *Vector database* seperti Chroma/FAISS menyimpan teks mentah, bukan angka.
Jawaban: Salah

3. *Embedding* adalah cara mengubah kalimat menjadi deretan angka.

Jawaban: Benar

4. *sentence-transformers* adalah *open source*.

Jawaban: Benar

5. FAISS hanya bisa dipakai jika ada GPU.

Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Urutan paling tepat dalam RAG adalah...

- A. Generate → Retrieve → Index → Embed
- B. Retrieve → Embed → Index → Generate
- C. Load → Embed → Index → Retrieve/Generate
- D. Index → Load → Embed → Generate

Jawaban: C

2. Library untuk membuat *embedding* kalimat:

- A. pandas
- B. sentence-transformers
- C. matplotlib
- D. scikit-image

Jawaban: B

3. Perintah pip untuk Chroma:

- A. `pip install chroma`
- B. `pip install chromadb`
- C. `pip install chroma-db`
- D. `pip install chroma-py`

Jawaban: B

4. Tipe index FAISS yang sederhana untuk jarak L2:

- A. `IndexFlatL2`
- B. `IndexIVFPQ`
- C. `IndexHNSW`
- D. `IndexAnnoy`

Jawaban: A

5. Pada Chroma, menambah dokumen dan *embedding* dilakukan dengan:

- A. `collection.put()`
- B. `collection.save()`
- C. `collection.add()`
- D. `collection.push()`

Jawaban: C

6. Model *embedding* ringan yang sering dipakai:

- A. `gpt-4`
- B. `bert-large-uncased-whatever`
- C. `all-MiniLM-L6-v2`
- D. `resnet50`

Jawaban: C

7. Fungsi utama *retriever* dalam RAG:

- A. Membuat gambar
- B. Mencari potongan teks paling mirip
- C. Menyimpan file
- D. Mengubah PDF ke PNG

Jawaban: B

8. Tool Python untuk olah tabel CSV:

- A. keras
- B. `numpy.random`
- C. pandas
- D. pillow

Jawaban: C

9. Kelebihan FAISS:

- A. Antarmuka GUI
- B. Kecepatan *similarity search*
- C. Visualisasi 3D
- D. Konversi video

Jawaban: B

10. Mengapa *metadata* penting saat indexing?

- A. Untuk mengacak dokumen
- B. Untuk menyimpan sumber/asal potongan
- C. Untuk memperkecil ukuran file
- D. Untuk mempercantik tampilan

Jawaban: B

C. Isian Singkat (5 Soal)

1. Istilah untuk deretan angka yang mewakili kalimat adalah _____.

Jawaban: embedding

2. Dua *vector database* yang dipakai di bab ini: _____ dan _____.

Jawaban: FAISS, Chroma

3. Perintah membuat *virtual environment* di Python: `python3 -m _____ env`.

Jawaban: venv

4. Nama koleksi Chroma dibuat dengan `client._____("nama")`.

Jawaban: `create_collection`

5. Library Python untuk *neural network* yang disebut di bab ini: _____.

Jawaban: Keras

D. Soal Explorasi (3 Soal)

Soal 1:

Tulis skrip Python singkat yang: (a) membuat daftar 4 kalimat, (b) menghitung *embedding* dengan `sentence-transformers`, (c) melakukan pencarian kemiripan terdekat memakai FAISS, (d) menampilkan kalimat hasil teratas.

Contoh jawaban siswa:

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

docs = ["CPU otak komputer", "GPU untuk grafis", "RAM menyimpan
sementara", "SSD media penyimpanan cepat"]
m = SentenceTransformer("all-MiniLM-L6-v2")
V = m.encode(docs).astype("float32")

index = faiss.IndexFlatL2(V.shape[1]); index.add(V)

q = "peran GPU"
vq = m.encode([q]).astype("float32")
D, I = index.search(vq, 1)
print(docs[I[0][0]])
```

Soal 2:

Buat *pseudocode* RAG mini dengan Chroma yang juga menyimpan *metadata* {"file": "sumber.txt", "baris": N}. Query satu pertanyaan dan tampilkan dokumen + sumbernya.

Contoh jawaban siswa:

```
mulai
    daftar_kalimat = baca_per_baris("sumber.txt")
    meta = [{"file": "sumber.txt", "baris": i+1} untuk i dalam
range(len(daftar_kalimat))]
    model = load_embedding("all-MiniLM-L6-v2")
    E = model.encode(daftar_kalimat)

    chroma = buat_client_chroma()
    col = chroma.create_collection("mini_rag")
    col.add(documents=daftar_kalimat, embeddings=E, ids=auto_id(),
metadatas=meta)

    q = input_user()
    h = col.query(query_texts=[q], n_results=2)
```

```

        cetak(h.documents[0][0], h.metadatas[0][0]["file"],
h.metadatas[0][0]["baris"])
selesai

```

Soal 3:

Rancang *prompt* ringkas untuk *generator* yang merangkum 3 potong teks hasil *retrieval* menjadi jawaban 2–3 kalimat. Tulis *prompt* dan contoh pemanggilan fungsi Python yang menggabungkan tiga potongan teks sebelum dikirim ke *generator* (boleh *placeholder*).

Contoh jawaban siswa:

```

PROMPT:
"Rangkum jawaban dari konteks berikut. Tulis 2-3 kalimat, jelas,
dan mengutip istilah penting.
Konteks:
{KONTEN}
Jawaban:"

```

Python (gabung konteks):

```

kandidat = [doc1, doc2, doc3]
konten = "\n".join(kandidat)
prompt_final = PROMPT.replace("{KONTEN}", konten)
print(prompt_final) # dikirim ke generator open-source pilihan

```

Lampiran Kode Ringkas (opsional dipakai di atas)

Contoh Chroma minimal:

```

import chromadb
from sentence_transformers import SentenceTransformer

docs = ["Kecepatan = jarak/waktu", "Laju mirip kecepatan", "Percepatan
=  $\Delta v / \Delta t$ "]
m = SentenceTransformer("all-MiniLM-L6-v2")
emb = m.encode(docs)

cli = chromadb.Client()
col = cli.create_collection("fisika")
col.add(documents=docs, embeddings=emb, ids=["1", "2", "3"])

q = "rumus laju"
res = col.query(query_texts=[q], n_results=1)
print(res["documents"][0][0])

```

Contoh FAISS minimal:

```

from sentence_transformers import SentenceTransformer
import faiss, numpy as np

```

```
docs = ["AI membantu manusia", "Python bahasa pemrograman", "Data penting untuk AI"]
m = SentenceTransformer("all-MiniLM-L6-v2")
V = m.encode(docs).astype("float32")

idx = faiss.IndexFlatL2(V.shape[1]); idx.add(V)
q = "bahasa untuk AI"
vq = m.encode([q]).astype("float32")
D, I = idx.search(vq, 1)
print(docs[I[0][0]])
```

Tugas Proyek

Proyek 1: “Kamus Istilah TKJ Pintar” (CSV → Chroma)

Tujuan: Bangun RAG yang menjawab arti istilah TKJ: *IP Address, Subnet, DNS, VLAN, dst.*

Data: `kamus_tkj.csv` (kolom: `istilah`, `definisi`).

Langkah Singkat

- Kumpulkan ±30 istilah TKJ di CSV.
- Baca CSV pakai **pandas**. Gabungkan jadi kalimat “**ISTILAH: DEFINISI**”.
- Buat *embedding* → simpan ke **Chroma**.
- *Query* contoh: “Apa itu *subnet mask*?” “Bedanya *DNS* dan *DHCP*?”

Pseudocode

```
mulai
    data = pandas.read_csv("kamus_tkj.csv")
    kalimat = format("{istilah}: {definisi}", untuk setiap baris)
    model = load_embedding("all-MiniLM-L6-v2")
    E = model.encode(kalimat)

    chroma = buat_client_chroma()
    col = chroma.buat_koleksi("kamus_tkj")
    col.tambah(documents=kalimat, embeddings=E, ids=auto_id())

    pertanyaan = input_user()
    q = model.encode([pertanyaan])
    hits = col.query(query_embeddings=q, n_results=3)
    tampilkan(hits.documents, sertakan_id=True)
selesai
```

Contoh Python Mini

```
import chromadb, pandas as pd
from sentence_transformers import SentenceTransformer

df = pd.read_csv("kamus_tkj.csv") # kolom: istilah, definisi
docs = [f"{r.istilah}: {r.definisi}" for _, r in df.iterrows()]
```

```

m = SentenceTransformer("all-MiniLM-L6-v2")
emb = m.encode(docs)

cli = chromadb.Client()
col = cli.create_collection("kamus_tkj")
col.add(documents=docs, embeddings=emb, ids=[str(i) for i in
range(len(docs))])

q = "Apa itu subnet mask?"
qe = m.encode([q])
res = col.query(query_embeddings=qe, n_results=2)
print("Jawaban kandidat:", *res["documents"][0], sep="\n- ")

```

Kunci Jawaban (Contoh Output)

- **“Subnet mask:** pola angka untuk membagi jaringan menjadi beberapa subnet, menentukan bagian network dan host.”
- **“DNS vs DHCP:** DNS menerjemahkan nama domain ke IP; DHCP membagikan IP secara otomatis.”

Proyek 2: “Asisten Jadwal Kelas” (CSV → FAISS)

Tujuan: Jawab pertanyaan waktu/ruang pelajaran: “TKJ XI *Jaringan* jam berapa?”

Data: `jadwal.csv` (kolom: `kelas`, `mapel`, `hari`, `jam`, `ruang`, `guru`).

Langkah Singkat

- Susun kalimat per baris: “`KELAS X` `mapel Y` pada `HARI` jam `Z` di `RUANG R` oleh `GURU G`”.
- *Embedding* → **FAISS** (L2).
- *Query*: “Kapan TKJ XI Jaringan?” “Ruang *Basis Data* kelas XII?”

Pseudocode

```

mulai
    df = pandas.read_csv("jadwal.csv")
    docs = buat_kalimat_terstruktur(df)
    model = load_embedding()
    V = model.encode(docs)
    D = dim(V)

    index = faiss.IndexFlatL2(D)
    index.add(V)

    tanya = input_user()
    vq = model.encode([tanya])
    dist, idx = index.search(vq, k=3)
    tampilkan(docs[idx[0]])
selesai

```

Contoh Python Mini

```

import faiss, pandas as pd

```

```

from sentence_transformers import SentenceTransformer
import numpy as np

df = pd.read_csv("jadwal.csv")
docs = [f"{r.kelas} {r.mapel} {r.hari} {r.jam} {r.ruang}
{r.guru}" for _, r in df.iterrows()]

m = SentenceTransformer("all-MiniLM-L6-v2")
V = m.encode(docs).astype("float32")
index = faiss.IndexFlatL2(V.shape[1]); index.add(V)

q = "Kapan TKJ XI Jaringan?"
vq = m.encode([q]).astype("float32")
D, I = index.search(vq, 3)
print("Kandidat:")
for i in I[0]: print("-", docs[i])

```

Kunci Jawaban (Contoh Output)

- “TKJ XI Jaringan — Rabu 08.00–10.00, Lab Jaringan, Pak Andi.”

Proyek 3: “Pusat Bantuan Linux Offline” (TXT/MD → Chroma + Metadata)

Tujuan: Jawab “Bagaimana *chmod 755*?” “Cara cek IP di Linux?” dari *cheatsheet* lokal.

Data: folder `linux_help/` berisi `.md` atau `.txt`.

Langkah Singkat

- Baca semua file, potong per paragraf/point.
- Simpan `metadatas={"file":nama, "baris":n}` ke **Chroma**.
- *Query*: tampilkan jawaban + **sumber** (file & baris).

Pseudocode

```

mulai
    file_list = daftar_berkas("linux_help/*.md")
    potongan, meta = [], []
    untuk file dalam file_list:
        teks = baca(file)
        for (baris, paragraf) in potong(teks):
            potongan.tambah(paragraf)
            meta.tambah({"file":file, "baris":baris})

    E = embed(potongan)
    col.add(documents=potongan, embeddings=E, ids=auto_id(),
    metadatas=meta)

    q = input_user()
    h = col.query(query_texts=[q], n_results=3)
    tampilkan(h.documents + h.metadatas) # sertakan sumber
selesai

```

Contoh Python Mini

```
import chromadb, glob
from sentence_transformers import SentenceTransformer

chunks, metas = [], []
for path in glob.glob("linux_help/*.md"):
    with open(path, "r") as f:
        lines = f.readlines()
    for i, line in enumerate(lines, 1):
        if line.strip():
            chunks.append(line.strip())
            metas.append({"file": path, "baris": i})

m = SentenceTransformer("all-MiniLM-L6-v2")
emb = m.encode(chunks)

cli = chromadb.Client()
col = cli.create_collection("linux_help")
col.add(documents=chunks, embeddings=emb,
        ids=[str(i) for i in range(len(chunks))],
        metadatas=metas)

q = "cara chmod 755"
res = col.query(query_texts=[q], n_results=3)
for d, md in zip(res["documents"][0], res["metadatas"][0]):
    print("-", d, "| Sumber:", md["file"], "baris", md["baris"])
```

Kunci Jawaban (Contoh Output)

- “**chmod 755: `rwxr-xr-x`**, perintah: **`chmod 755 nama_file`** (Sumber: **`linux_help/permissions.md`**, baris 12).”

Proyek 4: “Asisten Katalog UMKM” (CSV Produk → Chroma + Filter keyword)

Tujuan: Jawab pertanyaan katalog: “Minuman < 10rb?” “Roti coklat tersedia?”

Data: **`produk_umkm.csv`** (kolom: **`nama, kategori, harga, stok, deskripsi`**).

Langkah Singkat

- Buat kalimat kaya info: “**`NAMA (KATEGORI) HARGA=... STOK=... DESKRIPSI`**”.
- Index ke **Chroma**.
- *Filter* sederhana: cari *keyword* harga/kategori sebelum *query* ke Chroma.

Pseudocode

```
mulai
    df = pandas.read_csv("produk_umkm.csv")
    docs = gabung_info(df)
    E = embed(docs)
    col.add(documents=docs, embeddings=E, ids=auto_id())
```

```

q = input_user()
kata_kunci = ekstrak_keyword(q) # contoh: kategori, batas
harga
kandidat = col.query(query_texts=[q], n_results=10)
hasil = filter_manual(kandidat.documents, kata_kunci)
tampilkan(top(hasil, n=3))
selesai

```

Contoh Python Mini

```

import chromadb, pandas as pd, re
from sentence_transformers import SentenceTransformer

df = pd.read_csv("produk_umkm.csv")
docs = [f"{r.nama} ({r.kategori}) harga={r.harga} stok={r.stok}.
{r.deskripsi}"
        for _, r in df.iterrows()]

m = SentenceTransformer("all-MiniLM-L6-v2")
emb = m.encode(docs)

cli = chromadb.Client()
col = cli.create_collection("umkm")

col.add(documents=docs, embeddings=emb, ids=[str(i) for i in
range(len(docs))])

q = "minuman di bawah 10000"
res = col.query(query_texts=[q], n_results=10)

def pass_price(text, limit=10000):
    m = re.search(r"harga=(\d+)", text)
    return m and int(m.group(1)) < limit

filtered = [d for d in res["documents"][0] if pass_price(d,
10000)]
print("Rekomendasi:", *filtered[:3], sep="\n- ")

```

Kunci Jawaban (Contoh Output)

- “Es Teh Jumbo (Minuman) harga=8000 stok=12. Segar dan manis.”
- “Susu Cokelat Botol (Minuman) harga=9500 stok=7. Praktis untuk bekal.”

Proyek 5: “Info Prakerin & Mitra Industri” (TXT → FAISS + Ringkas)

Tujuan: Jawab “Syarat prakerin TKJ?” “Kontak perusahaan A?” dari *brief*/brosur.

Data: *prakerin/* berisi *.txt requirements*, kontak HR, *SOP* singkat.

Langkah Singkat

- Gabungkan semua berkas jadi daftar paragraf.
- **FAISS** untuk *retrieve* top-5.

- **Ringkas 2–3 poin** (tanpa LLM: *template* sederhana).
- Tampilkan **kontak** jika ada (regex email/no HP).

Pseudocode

```

mulai
    potongan = baca_semua_paragraf("prakerin/*.txt")
    V = embed(potongan)
    index = faiss.IndexFlatL2(dim(V))
    index.add(V)

    tanya = input_user()
    q = embed([tanya])
    D, I = index.search(q, k=5)
    kandidat = pilih(potongan, I[0])

    ringkas = ambil_3_poin(kandidat)      # template: bullet
    ringkasan
    kontak = cari_kontak(kandidat)      # regex email/HP
    tampilkan(ringkas, kontak)
selesai

```

Contoh Python Mini

```

import faiss, glob, re
from sentence_transformers import SentenceTransformer
import numpy as np

paras = []
for p in glob.glob("prakerin/*.txt"):
    with open(p, "r") as f:
        for line in f:
            t = line.strip()
            if t: paras.append(t)

m = SentenceTransformer("all-MiniLM-L6-v2")
V = m.encode(paras).astype("float32")
index = faiss.IndexFlatL2(V.shape[1]); index.add(V)

q = "Syarat prakerin TKJ dan kontak HR"
vq = m.encode([q]).astype("float32")
D, I = index.search(vq, 5)
cand = [paras[i] for i in I[0]]

def ringkas3(lst): return ["- " + s[:120] for s in lst[:3]]
def kontak(lst):
    join = " ".join(lst)
    email = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+",
join)
    hp = re.findall(r"\b08\d{8,12}\b", join)
    return email[:2], hp[:2]

summary = ringkas3(cand)
em, hp = kontak(cand)
print(*summary, sep="\n")

```

```
print("Kontak email:", em, "| HP:", hp)
```

Kunci Jawaban (Contoh Output)

- **Ringkasan:**
 - “Surat pengantar sekolah & CV singkat diperlukan.”
 - “Durasi minimal 3 bulan, jadwal fleksibel.”
 - “Materi: jaringan dasar, dokumentasi, *troubleshooting*.”
- **Kontak:** `hr@mitraindustri.co.id`, `081234567890`.

Bonus Opsional (untuk yang penasaran *machine learning*/Keras)

- Jadikan **proyek 5** lebih pintar: latih *neural network* kecil (Keras) untuk *rerank* hasil *retrieve* berdasarkan *fitur sederhana* (jumlah *keyword*, panjang teks). Ini **opsional**, tidak wajib.

Checklist Umum (semua proyek)

- `pip install chromadb faiss-cpu sentence-transformers pandas`
- Model *embedding*: `all-MiniLM-L6-v2` (ringan, *open source*).
- Simpan **ID + metadata** (sumber file/baris) untuk **kutipan**.
- Uji `k = 1–5` agar jawaban tidak terlalu sempit/berisik.

BAB 5: Integrasi RAG ke OpenWebUI untuk QnA Lokal

Selamat! Sekarang kita akan satukan semua yang sudah kita pelajari: **Ollama**, **OpenWebUI**, dan **RAG**. Bayangin, kita bakal bikin **chatbot** super cerdas yang bisa jawab pertanyaan dari dokumen kalian sendiri, semua di satu tempat yang gampang dipakai. Ini adalah puncak dari semua materi kita, skill yang akan membuat kalian jadi incaran perusahaan teknologi!

Tujuan Pembelajaran

- Mengintegrasikan **Ollama** dan **OpenWebUI** untuk sistem **QnA** lokal.
- Menggunakan fitur **RAG** bawaan di **OpenWebUI** secara praktis.
- Membuat **chatbot** yang bisa menjawab pertanyaan berdasarkan dokumen pribadi.
- Menjelaskan mengapa **QnA** lokal itu lebih aman, cepat, dan akurat.

Peta Konsep Sederhana

Bayangkan kita mau bikin warung kopi.

- **Ollama (Mesin Kopi)**: Ini adalah tool utama kita, yang bisa membuat kopi (menjawab pertanyaan) super enak.
- **OpenWebUI (Tampilan Menu)**: Ini adalah menu dan kasir yang bikin pelanggan gampang pesan dan bayar.
- **Dokumen Pribadi (Biji Kopi Langka)**: Ini adalah biji kopi spesial yang cuma kita punya.
- **Integrasi RAG (Resep Khusus)**: Ini adalah cara kita menyajikan kopi dari biji spesial kita, membuat rasanya unik dan otentik.

Apersepsi: AI yang Gagal Jawab Soal Ulangan!

Kamu lagi belajar untuk ulangan fisika. Kamu buka **OpenWebUI** dan tanya, "Jelaskan rumus gaya dalam bab 3!"

- **Tanpa Integrasi RAG**: AI akan jawab rumus umum yang dia tahu dari internet. Belum tentu sesuai dengan buku pelajaranmu.
- **Dengan Integrasi RAG**: Kamu sudah unggah buku pelajaranmu ke sistem **RAG** di **OpenWebUI**. Saat kamu tanya, AI akan **mencari** bab 3, **membaca** isinya, lalu **menjawab** dengan akurat sesuai buku kalian.

Nah, ini dia rahasia AI yang bisa kasih jawaban yang super spesifik dan terpercaya!

Penjelasan Teori

Bayangin kamu punya robot pintar di laptop. Robot ini bisa jawab pertanyaan apa pun. Tapi ada masalah: kadang jawabannya ngawur, karena dia cuma nebak dari *pengetahuan umum*.

Nah, supaya robot ini jadi **lebih pintar dan akurat**, kita kasih kemampuan baru: **RAG (Retriever – Augmented Generation)**.

Dengan RAG, robot bisa **membaca dokumen pribadi kita**, lalu menjawab pertanyaan dengan tepat.

Kenapa Penting?

- Tanpa RAG → jawaban bisa salah atau tidak sesuai kebutuhanmu.
- Dengan RAG → jawaban sesuai isi dokumenmu, jadi **spesifik dan terpercaya**.
- Semua ini berjalan **lokal** di laptopmu, jadi data tetap aman dan tidak bocor.

Komponen Utama

Ollama

Ini adalah “**mesin otak AI**”-nya. Dia menjalankan model *LLM (Large Language Model)* seperti *LLaMA*, *Mistral*, *Gemma*, dan lain-lain. Semua bisa jalan **offline di Ubuntu 24.04**, tanpa butuh koneksi internet. Yang penting, semua berbasis **open source** dan **gratis**.

OpenWebUI

Ini adalah tampilan web-nya (seperti *chat interface*). Kamu tinggal buka browser, lalu **ketik pertanyaan**. Nggak perlu coding panjang-panjang. Jadi kayak pakai ChatGPT, tapi **semua di laptop kamu sendiri**.

Dokumen Pribadi

Bisa dari:

- PDF tugas sekolah,
- File Word laporan magang,
- Atau catatan kelas.

Dokumen ini nanti jadi “**bahan bacaan AI-mu**”.

RAG (Retriever–Augmented Generation)

Bagian ini adalah “**otak tambahan**” yang bikin AI bisa membaca dokumenmu. Caranya:

1. Dokumen di-*index* → diubah jadi angka-angka khusus (*vektor*).

2. Disimpan di *vector database* lokal (contohnya *ChromaDB*).
3. Waktu kamu nanya, pertanyaanmu juga diubah jadi vektor.
4. AI cari bagian dokumen yang paling cocok.
5. Baru bikin jawaban yang akurat, berdasarkan isi dokumen.

Analogi Seru

Bayangin kamu buka **warung kopi**:

- **Ollama** → mesin kopi (tool utama).
- **OpenWebUI** → menu kasir (tampilan supaya gampang dipesan).
- **Dokumen Pribadi** → biji kopi spesial yang cuma kamu punya.
- **RAG** → resep rahasia supaya kopi terasa unik.

Hasil akhirnya? Kopi enak dengan rasa khas → alias jawaban AI yang **spesifik, akurat, dan sesuai kebutuhanmu**.

Kaitan dengan Teknologi Open Source

Semuanya jalan di:

- **Ubuntu 24.04 LTS** (sistem operasi gratis dan aman),
- **Python** (bahasa pemrograman open source),
- **Pandas, Keras, TensorFlow, ChromaDB**, dan lain-lain (semua open source).

Cocok banget buat siswa SMK yang suka ngoprek dan belajar teknologi **tanpa biaya mahal**. Kamu bisa bikin chatbot sekolah sendiri, bahkan di lab yang nggak pakai internet.

Cara Instalasi di Ubuntu 24.04

Langkah-langkah berikut bisa kamu coba langsung di laptop sekolah atau lab SMK.

1. Siapkan Sistem

Pastikan Ubuntu kamu versi 24.04 LTS dan sudah update:

```
sudo apt update && sudo apt upgrade -y
```

2. Instal Ollama

Ollama adalah mesin AI-nya.

```
curl -fsSL https://ollama.com/install.sh | sh
```

Setelah selesai, jalankan:

```
ollama run llama3
```

Perintah ini akan download model open source bernama LLaMA 3.

Kalau berhasil, kamu bisa langsung ngobrol di terminal.

Contoh:

```
>>> Hello, how are you?
```

3. Instal OpenWebUI

Sekarang biar tampilannya lebih ramah, kita pakai OpenWebUI.
Jalankan langkah berikut:

```
# Instal dependensi
sudo apt install -y docker.io docker-compose

# Jalankan OpenWebUI lewat Docker
sudo docker run -d -p 3000:8080 \
  -v openwebui:/app/backend/data \
  --name openwebui \
  ghcr.io/open-webui/open-webui:main
```

Setelah itu, buka browser dan kunjungi:

```
http://localhost:3000
```

Tadaaa

Kamu sekarang punya ChatGPT lokal buatan sendiri, jalan di Ubuntu 24.04, tanpa internet, tanpa biaya!

Kesimpulan

Jadi, *RAG* adalah **jembatan pintar** antara otak AI (*Ollama*) dan tampilan web (*OpenWebUI*). Dengan menambahkan dokumen pribadi, kamu bisa bikin **chatbot sekolah** yang:

- Cerdas dan cepat,
- Aman karena berjalan lokal,
- Dan bisa kamu bangun sendiri dengan tool **open source** di Ubuntu 24.04.

Dengan konsep ini, siswa SMK bisa belajar:

- *AI + Data Processing + Linux + Open Source* dalam satu proyek nyata yang seru dan bermanfaat.

Kasus Ilustrasi

Kasus 1: Asisten Tugas Kelompok

Bayangin kamu lagi bikin proyek kelompok. Tiap minggu ada laporan. Kalau dibaca manual, bisa pusing. Nah, kalau semua laporan itu kamu unggah ke **OpenWebUI**, AI bisa langsung bikin ringkasan bulanan.

- **Dokumen:** laporan_minggu1.txt, laporan_minggu2.txt, laporan_minggu3.txt
- **Pertanyaan:** “Ringkas jadi laporan bulanan.”
- **Hasil:** AI kasih rangkuman singkat dan jelas.

Pseudocode:

```
dokumen = ["minggu1.txt", "minggu2.txt", "minggu3.txt"]
unggah(dokumen)
tanya("Buat ringkasan laporan bulanan")
```

Kasus 2: Chatbot Aturan Sekolah

Kadang kita bingung sama aturan sekolah. Contoh: bolehkah pakai sepatu putih? Kalau file peraturan sekolah (PDF) kamu unggah ke OpenWebUI, chatbot bisa jadi “guru” mini yang jawab semua aturan.

- **Dokumen:** peraturan_sekolah.pdf
- **Pertanyaan:** “Apakah boleh memakai sepatu putih?”
- **Hasil:** AI jawab sesuai isi dokumen, bukan nebak-nebak.

Pseudocode:

```
unggah("peraturan_sekolah.pdf")
tanya("Apakah siswa boleh memakai sepatu putih?")
```

Kasus 3: Panduan Game Otomatis

Kamu punya panduan game panjang banget. Ribet kalau dibaca semua. Kalau file itu diunggah, chatbot bisa jadi *asisten gaming* yang jawab cepat.

- **Dokumen:** panduan_game.txt
- **Pertanyaan:** “Gimana cara dapet item legendaris?”
- **Hasil:** AI kasih langkah-langkah jelas dari panduan.

Pseudocode:

```
unggah("panduan_game.txt")
tanya("Bagaimana cara mendapatkan item legendaris?")
```

Kasus 4: Jadwal Harian → Mingguan

Kamu punya catatan jadwal harian, tapi guru minta jadwal mingguan. Nggak perlu bikin ulang, cukup unggah dokumen dan tanya ke chatbot.

- **Dokumen:** jadwal_harian.txt
- **Pertanyaan:** “Buat jadwal mingguan dari data ini.”
- **Hasil:** AI bikin tabel mingguan rapi.

Pseudocode:

```
unggah("jadwal_harian.txt")
tanya("Tolong buat jadwal mingguan dari data ini")
```

Kasus 5: Analisis Artikel Blog

Kamu suka nulis artikel blog, ada banyak sekali. Kalau diunggah ke OpenWebUI, chatbot bisa bantu analisis.

- **Dokumen:** artikel1.txt, artikel2.txt, artikel3.txt
- **Pertanyaan:** “Artikel mana yang paling banyak bahas teknologi?”
- **Hasil:** AI kasih daftar artikel yang relevan.

Pseudocode:

```
unggah(["artikel1.txt", "artikel2.txt", "artikel3.txt"])
tanya("Artikel mana yang paling banyak membahas tentang teknologi?")
```

Catatan

- Semua kasus ini jalan pakai **open source**: *Python, Pandas, machine learning, Keras, neural network*.
- Tidak butuh internet, jadi **privasi aman**.
- Hasilnya cepat, praktis, dan bikin kamu kelihatan pro di depan teman dan guru.

Soal Latihan & Pembahasan

Soal 1

Pertanyaan: Apa fungsi utama mengunggah dokumen ke *OpenWebUI*?

Jawaban: Membuat *basis pengetahuan pribadi* agar AI bisa menjawab sesuai isi dokumen.

Pembahasan:

Kalau tidak ada dokumen, AI cuma menebak dari pengetahuan umum. Tapi dengan dokumen, AI bisa memberikan jawaban spesifik.

Bayangin kayak kamu bawa buku catatan sendiri. AI akan membacanya sebelum menjawab.

Pseudocode:

```
unggah("catatan_fisika.pdf")
tanya("Apa rumus gaya di bab 3?")
```

Soal 2

Pertanyaan: Kamu punya buku fisika dalam bentuk PDF. Setelah diunggah ke OpenWebUI, apa yang terjadi?

Jawaban: Buku akan diproses jadi *vektor* lalu disimpan di database lokal.

Pembahasan:

Proses ini disebut *indexing*. Jadi isi buku diubah ke angka-angka khusus (vektor) supaya gampang dicari oleh AI.

Mirip seperti bikin daftar isi digital.

Pseudocode:

```
unggah("fisika.pdf")
# OpenWebUI -> indexing -> simpan di ChromaDB
```

Soal 3

Pertanyaan: Kamu sudah unggah dokumen, tapi AI masih jawab ngawur. Apa penyebabnya?

Jawaban: Bisa jadi *prompt* kurang jelas atau model AI tidak cocok.

Pembahasan:

Kalau pertanyaanmu terlalu umum, AI bingung mau ambil jawaban dari mana.

Atau, model yang dipakai belum optimal untuk RAG.

Solusi: bikin pertanyaan lebih spesifik.

Pseudocode:

```
unggah("peraturan_sekolah.pdf")
tanya("Jam berapa sekolah dimulai?")
# vs
tanya("Apa aturan tentang sepatu putih?")
```

Soal 4

Pertanyaan: Mengapa lebih praktis pakai *OpenWebUI* untuk RAG daripada bikin kode Python dari nol?

Jawaban: Karena OpenWebUI punya antarmuka grafis yang gampang dipakai.

Pembahasan:

Kalau pakai Python murni, kamu harus nulis banyak kode (import library, bikin vektor, query database).

Dengan OpenWebUI, tinggal klik, unggah, lalu tanya.

Semua itu tetap berjalan dengan teknologi *open source* seperti Python, Pandas, dan *neural network*.

Pseudocode (tanpa OpenWebUI – ribet):

```
import chromadb
db = chromadb.Client()
db.add(documents=["fisika.pdf"])
query("apa rumus gaya?")
```

Pseudocode (dengan OpenWebUI – simpel):

```
unggah("fisika.pdf")
tanya("Apa rumus gaya?")
```

Soal 5

Pertanyaan: Apa keuntungan utama QnA lokal dengan RAG di OpenWebUI?

Jawaban: Data aman, cepat, dan tidak butuh internet.

Pembahasan:

Kalau pakai layanan online, data bisa keluar ke server luar. Dengan OpenWebUI, semua disimpan di laptop sendiri. Selain itu, lebih cepat karena tidak perlu akses internet.

Pseudocode:

```
# Semua proses di laptop sendiri
unggah("jadwal_harian.txt")
tanya("Buat jadwal mingguan")
```

Catatan

- Unggah dokumen = AI punya “otak tambahan” dari catatanmu.
- Proses *indexing* bikin dokumen mudah dicari.
- Prompt yang jelas bikin jawaban lebih akurat.
- OpenWebUI lebih praktis daripada coding manual.
- QnA lokal = aman, cepat, tanpa internet.

Fun Facts!

- Fitur **RAG** di **OpenWebUI** bisa menghemat biaya perusahaan karena mereka tidak perlu berlangganan API yang mahal.
- **OpenWebUI** bisa terhubung ke **Ollama** yang berjalan di komputer lain, asalkan berada di jaringan yang sama.
- Teknik **RAG** ini adalah salah satu cara paling efektif untuk membuat AI yang tidak berhalusinasi.

Tips Belajar atau Menghafal Cepat

- **Langsung Coba!** Cari satu file PDF atau TXT di komputer kalian dan coba fitur RAG-nya.
- **Pahami Prosesnya:** Tanyakan pada diri sendiri, "Apa yang terjadi saat aku mengunggah file ini?"
- **Buat Catatan Sendiri:** Tulis di buku, "OpenWebUI itu kayak ChatGPT versi laptop kita."

Aktivitas Siswa

Aktivitas 1: Unggah Dokumen Pertama

Tujuan: Melatih siswa mengenal cara upload file ke OpenWebUI.

Langkah:

1. Buka *OpenWebUI* di browser.
2. Klik ikon RAG (biasanya simbol klip atau dokumen).
3. Pilih file TXT atau PDF, misalnya *sejarah_sekolah.pdf*.
4. Tunggu proses *indexing* selesai.
5. Coba tanya pertanyaan sederhana.

Pseudocode:

```
unggah("sejarah_sekolah.pdf")
tanya("Kapan sekolah ini didirikan?")
```

Contoh Python Script (simulasi sederhana):

```
dokumen = "sejarah_sekolah.pdf"
print("Dokumen", dokumen, "berhasil diunggah!")
```

Aktivitas 2: Chatbot Menjawab dari Dokumen

Tujuan: Membiasakan siswa bertanya sesuai isi dokumen.

Langkah:

1. Pastikan dokumen sudah diunggah.
2. Ketik pertanyaan yang jawabannya ada di dokumen.
3. Catat jawaban yang diberikan AI.
4. Ulangi dengan pertanyaan lain.

Pseudocode:

```
unggah("peraturan_sekolah.pdf")
tanya("Jam masuk sekolah jam berapa?")
```

Python Script:

```
dokumen = {"jam_masuk": "07.00 WIB"}
pertanyaan = "Jam masuk sekolah jam berapa?"
jawaban = dokumen["jam_masuk"]
print("Jawaban AI:", jawaban)
```

Aktivitas 3: Uji Batasan RAG

Tujuan: Melatih siswa kritis dengan mencoba pertanyaan yang tidak ada di dokumen.

Langkah:

1. Unggah satu dokumen, misalnya [peraturan_sekolah.pdf](#).
2. Tanyakan sesuatu yang tidak ada di dokumen, contoh: "Siapa presiden Indonesia sekarang?"
3. Lihat apakah AI jawab sesuai dokumen atau asal tebak.
4. Diskusikan hasilnya.

Pseudocode:

```
unggah("peraturan_sekolah.pdf")
tanya("Siapa presiden Indonesia sekarang?")
```

Python Script:

```
dokumen = {"aturan": "Tidak ada informasi presiden"}
pertanyaan = "Siapa presiden Indonesia sekarang?"
print("Jawaban AI: Maaf, informasi tidak ada di dokumen.")
```

Aktivitas 4: Kolaborasi Kelompok – Knowledge Base

Tujuan: Siswa belajar kerja tim dengan menggabungkan beberapa dokumen.

Langkah:

1. Bagi siswa dalam kelompok 2–3 orang.
2. Setiap siswa membuat dokumen berbeda (misalnya: sejarah sekolah, jadwal ekstrakurikuler, visi-misi).
3. Semua dokumen diunggah ke OpenWebUI.
4. Saling bertanya antar kelompok menggunakan chatbot.
5. Diskusikan hasilnya.

Pseudocode:

```
unggah(["sejarah.txt", "ekskul.txt", "visi_misi.txt"])
tanya("Apa kegiatan ekstrakurikuler di sekolah ini?")
```

Python Script:

```
dokumen = {
    "sejarah": "Sekolah berdiri tahun 1990",
```

```

        "ekskul": "Basket, Pramuka, Paskibra",
        "visi": "Mencetak siswa berprestasi"
    }
    pertanyaan = "Apa kegiatan ekstrakurikuler di sekolah ini?"
    print("Jawaban AI:", dokumen["ekskul"])

```

Aktivitas 5: Proyek Mini – Buat Ringkasan Otomatis

Tujuan: Siswa melatih AI untuk meringkas isi dokumen panjang.

Langkah:

1. Siapkan 3 laporan mingguan proyek kelas.
2. Unggah semua laporan ke OpenWebUI.
3. Tanya ke chatbot: “Buat ringkasan laporan bulanan.”
4. Cocokkan ringkasan AI dengan laporan asli.

Pseudocode:

```

unggah(["laporan1.txt", "laporan2.txt", "laporan3.txt"])
tanya("Ringkas jadi laporan bulanan")

```

Python Script (simulasi ringkasan):

```

laporan = ["Minggu 1: Diskusi awal",
            "Minggu 2: Mulai eksperimen",
            "Minggu 3: Presentasi hasil"]
ringkasan = "Proyek berjalan dari diskusi, eksperimen, hingga presentasi hasil."
print("Ringkasan AI:", ringkasan)

```

Catatan

- Aktivitas ini bikin siswa **aktif, kritis, kolaboratif**.
- Semua teknologi yang dipakai **open source**: *Python, Pandas, Keras, Machine Learning, Neural Network*.
- Dengan praktik langsung, siswa jadi tahu AI tidak sekadar “ngarang”, tapi bisa benar-benar *membaca dan memahami dokumen mereka sendiri*.

Rangkuman

Di bab ini kita sudah belajar bagaimana **Ollama, OpenWebUI, dan RAG** bekerja sama. Dengan integrasi ini, kita bisa membuat chatbot lokal yang **lebih pintar, cepat, dan aman**. **RAG** memberi kemampuan pada AI untuk membaca dokumen pribadi kita, lalu menjawab pertanyaan sesuai isinya. Semua berjalan di laptop sendiri, tanpa internet, dengan teknologi **open source** seperti *Python, Pandas, machine learning, Keras, dan neural network*.

Keunggulan utama dari sistem ini adalah **jawaban lebih akurat dan tidak ngawur**. Tanpa RAG, AI hanya menebak dari pengetahuan umum. Dengan RAG, jawaban jadi **spesifik, relevan, dan sesuai kebutuhan**. Bayangin kamu punya guru privat yang hafal catatanmu sendiri. Privasi juga terjaga, karena data tidak dikirim keluar. Jadi, chatbot lokal ini bukan hanya pintar, tapi juga **hemat biaya dan lebih aman**.

Dengan skill ini, kalian sudah punya bekal untuk membuat **asisten digital** sendiri. Bisa dipakai untuk tugas sekolah, ringkasan proyek, sampai analisis artikel. Ini bukan cuma teori, tapi juga **praktik nyata** yang bikin kalian terlihat pro di depan teman atau perusahaan. Ingatlah, **RAG + OpenWebUI = ChatGPT versi lokal**. Dan semuanya bisa kalian kuasai dengan teknologi *open source*. Seru kan?

Latihan Soal

A. Benar atau Salah (5 Soal)

1. OpenWebUI hanya bisa menjawab pertanyaan dari internet, tidak bisa dari dokumen pribadi.
Jawaban: Salah
2. Dengan RAG, AI bisa mencari informasi langsung dari dokumen yang sudah diunggah.
Jawaban: Benar
3. Ollama berfungsi sebagai mesin utama yang menjalankan model AI.
Jawaban: Benar
4. Mengunggah dokumen ke OpenWebUI akan otomatis membuat basis pengetahuan pribadi.
Jawaban: Benar
5. Integrasi RAG membuat AI lebih sering halusinasi.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

6. Fungsi utama RAG di OpenWebUI adalah:
 - a. Menghapus dokumen lama
 - b. Membuat dokumen baru
 - c. Mengambil informasi dari dokumen yang diunggah
 - d. Mengatur tampilan menu**Jawaban:** c
7. Apa keuntungan utama QnA lokal?
 - a. Jawaban lebih cepat dan privasi lebih aman
 - b. Harus selalu online

- c. Membutuhkan biaya API mahal
- d. Hanya bisa dipakai di Windows

Jawaban: a

8. Ollama diibaratkan seperti apa pada analogi warung kopi?
- a. Menu kasir
 - b. Mesin kopi
 - c. Biji kopi langka
 - d. Resep khusus

Jawaban: b

9. OpenWebUI dalam peta konsep diibaratkan sebagai:
- a. Mesin kopi
 - b. Resep khusus
 - c. Menu dan kasir
 - d. Pelanggan

Jawaban: c

10. Apa yang terjadi jika file PDF diunggah ke OpenWebUI?
- a. Diubah jadi vektor dan disimpan di database lokal
 - b. Langsung dibuang
 - c. Tidak bisa dibaca AI
 - d. Hanya bisa dilihat gambarnya

Jawaban: a

11. Teknologi *open source* yang sering dipakai dalam integrasi AI adalah:
- a. Photoshop
 - b. Python dan Pandas
 - c. Microsoft Word
 - d. Corel Draw

Jawaban: b

12. Apa nama database lokal yang biasa dipakai untuk menyimpan vektor dokumen?
- a. Excel
 - b. Chroma
 - c. PowerPoint
 - d. Access

Jawaban: b

13. Apa yang harus dilakukan jika AI menjawab ngawur padahal dokumen sudah diunggah?
- a. Restart komputer
 - b. Gunakan prompt lebih spesifik
 - c. Hapus semua dokumen
 - d. Pindah ke aplikasi lain

Jawaban: b

14. Menggunakan RAG di OpenWebUI lebih mudah daripada coding manual dengan Python, karena...
- Gratis tanpa listrik
 - Sudah ada antarmuka ramah pengguna
 - Tidak perlu komputer
 - Tidak butuh dokumen

Jawaban: b

15. Hal apa yang membuat QnA lokal lebih baik daripada QnA online?
- Lambat
 - Data pribadi lebih aman
 - Membutuhkan internet cepat
 - Hanya bisa diakses di server besar

Jawaban: b

C. Isian Singkat (5 Soal)

16. Ollama dalam sistem QnA lokal berperan sebagai _____.

Jawaban: mesin AI / mesin utama

17. Dokumen yang diunggah ke OpenWebUI akan diproses menjadi _____ di database lokal.

Jawaban: vektor

18. Teknologi *open source* seperti _____ digunakan untuk mengolah data AI dengan mudah.

Jawaban: Python

19. Fitur RAG membantu AI agar tidak _____ saat menjawab pertanyaan.

Jawaban: halusinasi

20. Contoh aplikasi RAG di sekolah adalah membuat chatbot yang menjawab pertanyaan tentang _____.

Jawaban: aturan sekolah

D. Soal Eksplorasi (3 Soal)

Soal 1

Eksplorasi – Chatbot Catatan Pelajaran

Bayangkan kamu punya catatan matematika dalam file Word. Unggah ke OpenWebUI lalu tanyakan soal rumus luas segitiga.

Jawaban siswa (contoh):

- Langkah: unggah `catatan_mtk.docx` → ketik pertanyaan → chatbot jawab sesuai isi file.
- Pseudocode:*

```
unggah("catatan_mtk.docx")
tanya("Apa rumus luas segitiga?")
```

Soal 2

Eksplorasi – Chatbot Buku Resep

Kamu mengunggah buku resep ke OpenWebUI. Tanyakan cara membuat nasi goreng spesial.

Jawaban siswa (contoh):

- Langkah: unggah **resep_makanan.pdf** → ketik pertanyaan → chatbot menampilkan langkah sesuai dokumen.
- *Pseudocode:*

```
unggah("resep_makanan.pdf")
tanya("Bagaimana cara membuat nasi goreng spesial?")
```

Soal 3

Eksplorasi – Analisis Blog Teknologi

Kamu punya 5 artikel blog tentang teknologi. Unggah semuanya, lalu minta chatbot menentukan artikel mana yang paling banyak membahas AI.

Jawaban siswa (contoh):

- Langkah: unggah **artikel1.txt ... artikel5.txt** → tanyakan → chatbot analisis.
- *Pseudocode:*

```
unggah(["artikel1.txt", "artikel2.txt", "artikel3.txt"])
tanya("Artikel mana yang paling banyak membahas AI?")
```

Hasil akhir: siswa bisa belajar teori, praktik, dan eksplorasi nyata dengan **RAG + OpenWebUI + Ollama**, semua dengan teknologi *open source*.

Tugas Proyek

Proyek 1: Chatbot Kamus Istilah Teknologi

Tugas:

Buat file berisi 15 istilah teknologi (contoh: *Python, Machine Learning, Neural Network*). Unggah ke OpenWebUI. Minta chatbot menjelaskan arti kata-kata itu.

Langkah:

1. Buat file **kamus_teknologi.txt**.
2. Isi dengan daftar istilah dan penjelasannya.
3. Unggah file ke OpenWebUI.
4. Coba tanyakan arti dari beberapa istilah.

Pseudocode:

```
unggah("kamus_teknologi.txt")
tanya("Apa itu neural network?")
```

Python Script:

```
kamus = {"Neural Network": "Model AI yang terinspirasi dari otak manusia"}
print("Jawaban AI:", kamus["Neural Network"])
```

Proyek 2: Chatbot Resep Minuman Favorit**Tugas:**

Buat dokumen berisi 5 resep minuman sederhana (teh manis, kopi susu, es jeruk, dll). Unggah ke OpenWebUI. Chatbot harus bisa jawab pertanyaan tentang bahan atau cara membuatnya.

Langkah:

1. Buat file `resep_minuman.txt`.
2. Tulis 5 resep singkat.
3. Unggah ke OpenWebUI.
4. Tanyakan resep spesifik, misalnya: "Apa bahan es jeruk?"

Pseudocode:

```
unggah("resep_minuman.txt")
tanya("Apa bahan es jeruk?")
```

Python Script:

```
resep = {"Es Jeruk": "Air jeruk, gula, es batu"}
print("Jawaban AI:", resep["Es Jeruk"])
```

Proyek 3: FAQ Ekstrakurikuler Sekolah**Tugas:**

Buat file berisi daftar pertanyaan dan jawaban tentang ekstrakurikuler sekolah (misalnya basket, pramuka, robotik). Upload ke OpenWebUI, lalu coba bertanya seolah chatbot adalah ketua ekstrakurikuler.

Langkah:

1. Buat file `faq_ekskul.txt`.
2. Tulis minimal 10 tanya-jawab singkat.
3. Unggah file ke OpenWebUI.
4. Coba tanyakan, "Kapan ekstrakurikuler basket diadakan?"

Pseudocode:

```
unggah("faq_ekskul.txt")
tanya("Kapan ekskul basket diadakan?")
```

Python Script:

```
faq = {"Ekskul Basket": "Setiap hari Jumat jam 3 sore"}
print("Jawaban AI:", faq["Ekskul Basket"])
```

Proyek 4: Analisis Lagu Favorit

Tugas:

Kumpulkan lirik 3 lagu favoritmu dalam file teks. Unggah ke OpenWebUI. Chatbot harus bisa menjawab pertanyaan tentang isi lirik, misalnya tema atau kata yang sering muncul.

Langkah:

1. Buat file `lirik_lagu.txt`.
2. Tulis lirik 3 lagu.
3. Unggah ke OpenWebUI.
4. Tanyakan hal menarik, contoh: "Tema apa yang paling banyak muncul?"

Pseudocode:

```
unggah("lirik_lagu.txt")
tanya("Tema apa yang paling banyak muncul di lirik ini?")
```

Python Script (analisis kata):

```
lirik = "cinta cinta rindu rindu cinta"
kata = lirik.split()
tema = max(set(kata), key=kata.count)
print("Tema paling sering:", tema)
```

Proyek 5: Jadwal Belajar Otomatis

Tugas:

Buat file berisi jadwal belajar harian (Senin–Jumat). Unggah ke OpenWebUI. Minta chatbot membuat versi mingguan atau ringkasannya.

Langkah:

1. Buat file `jadwal_belajar.txt`.
2. Tulis kegiatan belajar tiap hari.
3. Unggah file ke OpenWebUI.
4. Tanyakan: "Ringkas jadwal belajar mingguan."

Pseudocode:

```
unggah("jadwal_belajar.txt")
tanya("Ringkas jadwal belajar mingguan")
```

Python Script:

```
jadwal = {  
    "Senin": "Matematika",  
    "Selasa": "Fisika",  
    "Rabu": "Biologi",  
    "Kamis": "Bahasa Inggris",  
    "Jumat": "Informatika"  
}  
ringkasan = ", ".join(jadwal.values())  
print("Ringkasan AI:", ringkasan)
```

Catatan

Lima proyek ini bikin kamu bisa main-main dengan OpenWebUI + RAG. Semua gampang dicoba, tapi tetap menantang. Kamu bisa bikin chatbot untuk kamus, resep, ekskul, lirik lagu, sampai jadwal belajar. Dan ingat, semua berbasis teknologi open source seperti Python, Pandas, Machine Learning, Keras, Neural Network.

BAB 6: Etika & Keamanan Data dalam RAG

Sekarang kita akan masuk ke bab yang paling penting: **Etika & Keamanan Data**. AI itu seperti pisau, bisa dipakai untuk hal baik, bisa juga untuk hal yang jahat. Nah, di bab ini, kita akan belajar bagaimana caranya menggunakan **RAG** dengan bijak dan aman. Ini adalah tanggung jawab kita sebagai calon profesional di dunia teknologi!

Tujuan Pembelajaran

- Memahami mengapa **etika** dan **keamanan data** adalah pondasi **RAG** yang kuat.
- Mengenali jenis-jenis ancaman seperti **data bias** dan **disinformasi**.
- Belajar cara melindungi **privasi** pengguna dalam sistem **RAG**.
- Mengaplikasikan pedoman etika untuk membangun **chatbot** yang bertanggung jawab.

Peta Konsep Sederhana

Bayangin kita mau membuat "otak" AI untuk sebuah tim olahraga.

- **Otak AI (Sistem RAG)**: Punya kekuatan super untuk menjawab pertanyaan.
- **Shield (Keamanan)**: Ini adalah pagar yang menjaga **data pribadi** (misal, nomor HP anggota) agar tidak bocor.
- **Compass (Etika)**: Ini adalah kompas moral yang memastikan AI-nya tidak **bias** (misal, cuma fokus ke tim cowok) dan tidak menyebarkan **hoax**.

Apersepsi: Bot yang Bocorkan Rahasia Tim!

Kamu dan teman-temanmu bikin **chatbot RAG** untuk tim futsal sekolah. Kalian unggah semua dokumen, termasuk jadwal latihan, strategi rahasia, dan nomor kontak anggota.

Suatu hari, ada anggota tim lawan yang iseng tanya, "Apa strategi rahasia tim kalian?" Dan si **chatbot** yang naif itu malah jawab, "Strategi rahasia kami adalah 'Formasi Serangan Naga Merah'." Gawat, kan?!

Kenapa ini terjadi? Karena kita tidak memikirkan **etika** dan **keamanan data**. Sistem **RAG** hanya akan menjawab dari dokumen yang kita berikan, tanpa membedakan mana yang rahasia atau bukan.

Penjelasan Teori

1) Kenapa Etika & Keamanan Itu Wajib?

- AI itu seperti **pisau**. Bisa bantu, bisa juga melukai.
- Di sistem **RAG** (*Retriever-Augmented Generation*), AI **mengutip** dari dokumen yang kamu unggah.
- Kalau dokumennya salah atau rahasia, **jawaban AI juga salah** atau **membocorkan rahasia**.
- Jadi, **etika** = kompas moral. **Keamanan** = pagar pelindung. Keduanya menyelamatkanmu dari masalah.

Inti penting:

- **Etika** memastikan AI **adil** dan **tidak merugikan** orang lain.
- **Keamanan** memastikan data **aman**, **privasi terjaga**, dan **akses terbatas**.

2) Tiga Pilar Utama

a) Keamanan & Privasi

- Tujuan: **melindungi data pribadi** (nama, nomor HP, alamat, nilai, keuangan).
- Prinsip: **minimum exposure** → hanya unggah yang perlu.
- Di RAG: **filter** dulu dokumen. Kalau ada data sensitif, **samarkan** atau **hapus**.

b) Bias & Fairness

- Bias = data miring ke satu sisi.
- Dampak: jawaban AI jadi **tidak adil**.
- Contoh: hanya ada data kelas X, maka chatbot akan **melupakan** kelas XI & XII.
- Solusi: seimbangkan data, wakili semua pihak yang relevan.

c) Disinformasi & Hoax

- RAG **percaya** dokumenmu. Kalau kamu unggah hoax, AI ikut menyebar hoax.
- Solusi: **cek sumber** sebelum unggah (buku pelajaran, situs resmi, dokumen terverifikasi).

3) Peta Bahaya (Threat Model) Sederhana

Bayangin kamu bikin **chatbot RAG** untuk sekolah.

- **Kebocoran data**: jadwal ujian rahasia tersebar.
- **Akses tanpa izin**: siswa luar kelas ikut pakai dokumen internal.
- **Bias data**: hanya menyorot satu jurusan (misal TKJ), jurusan lain tidak terlayani.
- **Dokumen bertentangan**: dua jadwal berbeda → jawaban AI membingungkan.
- **Hoax**: unggah artikel palsu → AI ikut menyebarkan kabar bohong.

Kuncinya: identifikasi **siapa** yang boleh lihat, **apa** yang boleh dibaca, dan **kapan** dokumen dipakai.

4) Prinsip “Aman Secara Bawaan” (*Safe by Default*)

Pakai alur sederhana ini saat membangun RAG:

1. **Kumpulkan** dokumen dari sumber terpercaya.
2. **Bersihkan** data: hapus/samarkan data sensitif (nomor HP, email, alamat).
3. **Validasi** isi: hindari hoax, cek ganda informasi penting.
4. **Pisahkan akses:** dokumen publik vs dokumen internal/rahasia.
5. **Pantau jawaban:** kalau prompt ambigu, jawab “**tidak ditemukan di dokumen**” daripada mengarang.

Semua langkah di atas bisa kamu otomasi dengan **Python + Pandas** (untuk cek teks), atau *regex*. Tetap *open source* dan ringan.

5) Aturan Emas (Golden Rules) untuk Siswa SMK

- **Unggah secukupnya.** Jangan masukkan data pribadi.
- **Verifikasi sumber.** Utamakan buku pelajaran, situs resmi, dokumen sekolah.
- **Seimbangkan data.** Hindari bias.
- **Tolak halusinasi.** Jika tidak ada di dokumen, **katakan tidak tahu**.
- **Jejak audit.** Simpan catatan: dokumen apa dipakai, versi berapa, siapa yang unggah.

6) Red Flag vs Green Flag (Cepat Hafal)

Red flag (hindari):

- Nomor HP, alamat, nilai mentah, laporan keuangan internal.
- Artikel tanpa sumber jelas.
- Dokumen dengan opini berat sebelah.
- Dua dokumen isinya saling bertentangan.

Green flag (boleh):

- Buku pelajaran, modul resmi, pengumuman sekolah publik.
- Data yang sudah **disamarkan** (anonim).
- Ringkasan materi yang **netral** dan **terverifikasi**.

7) Peran Teknologi *Open Source*

Semua bisa dibangun dengan tool **gratis**:

- **Python** untuk logika dan *filtering*.
- **Pandas** untuk cek dan bersih-bersih data teks.
- **Keras & neural network** untuk model AI (kalau kamu ingin lanjut eksplor).

- *Machine learning* untuk deteksi pola (misal deteksi kata sensitif).
- Dipadukan dengan **Ollama + OpenWebUI** untuk RAG lokal yang **aman** dan **cepat**.

Kesimpulan:

Etika = arah. **Keamanan** = pelindung. **RAG** = mesin baca-jawab.

Kalau tiga-tiganya jalan, chatbotmu **pintar, adil, dan aman**. Kalau tidak, siap-siap **bocor data, jawaban bias**, atau **hoax**.

Jadilah **developer bertanggung jawab**: pikirkan dampak sebelum unggah, cek dokumen sebelum pakai, dan biasakan sistem menjawab **jujur** saat tidak tahu.

Kasus Ilustrasi

Kasus 1: Privasi: Samarkan Data Pribadi sebelum Diunggah

Cerita singkat: Kamu mau bikin chatbot RAG untuk OSIS. Ada file daftar kontak siswa. Itu **sensitif**.

Tujuan: Hapus/anonimkan nomor HP & email sebelum unggah.

Langkah cepat:

- Cek isi dokumen.
- Temukan pola sensitif (nomor, email).
- Samarkan jadi *** atau hapus.
- Baru unggah ke *OpenWebUI*.

Pseudocode:

```
dok = baca("kontak_siswa.txt")
dok_bersih = samarkan(dok, pola=["email", "nomor_hp"])
simpan(dok_bersih, "kontak_siswa_sanitized.txt")
unggah("kontak_siswa_sanitized.txt")
```

Contoh Python mini (regex):

```
import re, pathlib
teks = pathlib.Path("kontak_siswa.txt").read_text()
teks = re.sub(r'\b[\w.+~]+@[~\w.-]+\.\w+\b', '[email disamarkan]', teks)
teks = re.sub(r'\b(\+?62|0)\d{8,15}\b', '[nomor disamarkan]', teks)
pathlib.Path("kontak_siswa_sanitized.txt").write_text(teks)
```

Kasus 2: Akses: Pisahkan Dokumen Publik vs Rahasia

Cerita singkat: Klub olahraga punya “Strategi Rahasia” dan “Pengumuman Publik”. Jangan campur!

Tujuan: Hanya pengguna berizin yang bisa mengakses dokumen rahasia.

Langkah cepat:

- Folder **public/** untuk materi umum.
- Folder **internal/** untuk dokumen rahasia.
- Buat daftar izin: siapa boleh baca apa.
- Saat *query*, batasi *retrieval* hanya ke koleksi yang sesuai.

Pseudocode:

```
izin = {"andi": ["public"], "budi": ["public","internal"]}
pengguna = "andi"
koleksi = izin[pengguna]          # → ["public"]
ctx = ambil_konteks(dari=koleksi)  # hanya public
jawab(pertanyaan, konteks=ctx)
```

Kasus 3: Bias: Seimbangkan Data agar Jawaban Adil

Cerita singkat: Chatbot beasiswa hanya punya data penerima laki-laki. Jawabannya jadi bias.

Tujuan: Tambahkan data dari semua kelompok relevan (gender/kelas/jurusan).

Langkah cepat:

- Audit isi: hitung proporsi tiap kategori.
- Jika timpang, tambahkan dokumen pelengkap.
- Cek ulang sampai seimbang.

Pseudocode (pakai *Pandas*):

```
import pandas as pd
df = pd.read_csv("data_beasiswa.csv")    # kolom: nama, gender,
jurusan
proporsi = df["gender"].value_counts(normalize=True)
if proporsi.min() < 0.3:                  # ambang sederhana
    tambah_dokumen("profil_penerima_perempuan.txt")
unggah_semua()
```

Kasus 4: Disinformasi: Verifikasi Sumber sebelum Indexing

Cerita singkat: Ada artikel “viral” tanpa sumber jelas. Jangan dipercaya mentah-mentah.

Tujuan: Hanya *index* dokumen dari sumber tepercaya (buku pelajaran, situs resmi).

Langkah cepat:

- Buat daftar *whitelist* domain/sumber.
- Tolak dokumen dari sumber *meragukan*.
- Tandai dokumen “butuh verifikasi” jika ragu.

Pseudocode:

```
whitelist = ["kemdikbud.go.id", "buku_resmi.pdf",
            "panduan_sekolah.md"]
def aman(dok):
    return dok.sumber in whitelist

untuk dok in daftar_dokumen:
    if aman(dok):
        index(dok)
    else:
        tandai(dok, "perlu_verifikasi")
```

Kasus 5: Konsistensi: Hindari Dokumen Bertentangan (Versi & Tanggal)

Cerita singkat: Dua file jadwal latihan berbeda tanggal. Chatbot bingung.

Tujuan: Pakai **versi terbaru**, singkirkan versi lama, atau beri *stamp* tanggal.

Langkah cepat:

- Deteksi dokumen ganda berdasarkan judul/topik.
- Pilih yang terbaru (lihat tanggal/versi).
- Saat *retrieval*, filter hanya **status=final** atau **versi_terbaru=True**.

Pseudocode:

```
docs = muat_meta(["jadwal_v1.pdf", "jadwal_v2.pdf"])
terbaru = pilih_terbaru(docs) # berdasarkan tanggal/versi
index(terbaru)
ignore(docs - {terbaru})
```

Python mini (pemilihan terbaru):

```
files = [("jadwal_v1.pdf", "2025-08-01"),
         ("jadwal_v2.pdf", "2025-09-10")]
terbaru = max(files, key=lambda x: x[1])[0]
print("Index yang dipakai:", terbaru)
```

Tips Aman Cepat

- **Samarkan** (privasi).
- **Batasi** (akses).
- **Seimbangkan** (bias).
- **Verifikasi** (hoax).
- **Versikan** (konsistensi).

Semua alur di atas bisa kamu otomasi dengan **Python + Pandas + regex**. Kalau mau lanjut, kamu bisa menambahkan model *machine learning* (bahkan *Keras + neural network*) untuk **deteksi pola sensitif** atau **klasifikasi hoax**—tetap *open source*, tetap lokal, tetap aman.

Soal Latihan & Pembahasan

Soal 1 — Privasi Data

Pertanyaan:

Kamu membuat chatbot RAG untuk sekolah. Temanmu mau mengunggah file daftar siswa lengkap dengan nomor HP. Apa yang harus kamu lakukan?

Jawaban:

Jangan langsung unggah. Hapus atau samarkan nomor HP sebelum dimasukkan.

Pembahasan:

Privasi itu penting. Jika ada data pribadi, chatbot bisa membocorkan ke orang luar. Gunakan *Python* untuk menyamarkan data sebelum diunggah.

Pseudocode:

```
for baris in dokumen:
    if "nomor_hp" in baris:
        baris = "nomor_hp: [disamarkan]"
```

Soal 2 — Bias dalam Data

Pertanyaan:

Chatbot lomba sains hanya dilatih dengan data peserta laki-laki. Apa akibatnya?

Jawaban:

Jawaban chatbot akan bias, tidak bisa menjawab tentang peserta perempuan.

Pembahasan:

Data harus seimbang (*fair*). Kalau tidak, AI jadi tidak adil. Solusinya adalah menambah data dari peserta perempuan.

Pseudocode:

```
if "gender" hanya == "laki-laki":
    tambahkan_data("peserta_perempuan.csv")
```

Soal 3 — Deteksi Hoax

Pertanyaan:

Kamu menemukan artikel berjudul “Fakta Mengejutkan! Semua orang bisa jadi kaya dalam 1 hari!”. Apakah artikel ini boleh diunggah ke RAG?

Jawaban:

Tidak boleh, karena kemungkinan besar itu hoax.

Pembahasan:

RAG tidak bisa membedakan mana benar dan salah. Kalau kamu masukkan artikel hoax, chatbot akan ikut menyebarkan. Gunakan logika sederhana untuk memberi peringatan jika dokumen berisi kata-kata seperti *viral* atau *mengejutkan*.

Pseudocode:

```
if "viral" in teks or "mengejutkan" in teks:
    print("Warning: kemungkinan hoax")
```

Soal 4 — Konsistensi Dokumen

Pertanyaan:

Ada dua dokumen jadwal latihan: satu bilang jam 7, satunya jam 9. Bagaimana seharusnya sistem RAG memilih?

Jawaban:

Pilih dokumen dengan versi terbaru atau yang sudah disahkan.

Pembahasan:

Jika ada data ganda yang bertentangan, chatbot bisa bingung. Maka perlu dicek versi atau tanggal dokumennya.

Pseudocode:

```
if tanggal(doc1) > tanggal(doc2):
    gunakan(doc1)
else:
    gunakan(doc2)
```

Soal 5 — Etika AI Lokal

Pertanyaan:

Mengapa Ollama + OpenWebUI lebih aman untuk privasi daripada chatbot online?

Jawaban:

Karena semua data diproses di komputer lokal, tidak dikirim ke internet.

Pembahasan:

Ini adalah alasan utama mengapa banyak sekolah atau perusahaan memilih sistem lokal. Data tetap di dalam jaringan sendiri, lebih aman.

Pseudocode:

```
lokasi_proses = "lokal"
if lokasi_proses == "lokal":
    print("Data aman, tidak keluar internet")
```

Dengan 5 contoh soal ini, siswa bisa paham bahwa **etika & keamanan** bukan teori kosong, tapi bisa diuji dengan kasus nyata dan *open source code*.

Fun Facts!

- Perusahaan besar punya tim khusus yang tugasnya mencari dan membersihkan **bias** dari data AI mereka.
- **Halusinasi** AI bisa terjadi bahkan di **RAG** jika **prompt**-nya ambigu atau dokumennya tidak relevan.
- Ada pekerjaan baru yang muncul, namanya **AI Ethicist**, yang tugasnya memastikan AI tidak merugikan manusia.

Tips Belajar/Menghafal Cepat

- **Pikirkan Dampaknya:** Selalu tanya, "Apa dampaknya kalau data ini bocor?" atau "Apa dampaknya kalau informasinya salah?"
- **Gunakan Analogi:** Ingat analogi **brankas** dan **kompas**.
- **Cek Sumber:** Selalu cek sumber dari dokumen yang akan kalian unggah.

Aktivitas Siswa

Aktivitas 1: Menyaring Data Pribadi

Tujuan: Belajar menjaga privasi dengan menyamarkan data sensitif.

Langkah-langkah:

1. Buat file teks berisi daftar siswa lengkap dengan nama dan nomor HP.
2. Buka file itu di Python.
3. Ganti nomor HP menjadi kata **[disamarkan]**.

Pseudocode:

for setiap_baris in dokumen:

 jika ada "nomor_hp":

 ubah menjadi "[disamarkan]"

Python Code:

```
data = ["Nama: Budi, nomor_hp: 0812345678",
        "Nama: Ani, nomor_hp: 0898765432"]

for baris in data:
```

```

        if "nomor_hp" in baris:
            print(baris.replace(baris.split(":")[2],
"[disamarkan]"))

```

Aktivitas 2: Mendeteksi Bias Gender

Tujuan: Mengenali bias pada dokumen.

Langkah-langkah:

1. Buat file teks berisi daftar peserta lomba, tapi hanya laki-laki.
2. Jalankan Python untuk mencari apakah kata *perempuan* ada di dokumen.
3. Jika tidak ada, beri peringatan "Data Bias!".

Pseudocode:

```

jika "perempuan" tidak ditemukan di dokumen:
    tampilkan "Data Bias!"

```

Python Code:

```

dokumen = "Peserta lomba: Budi, Andi, Joko"
if "perempuan" not in dokumen:
    print("Data Bias!")

```

Aktivitas 3: Deteksi Hoax

Tujuan: Melatih AI sederhana untuk memberi peringatan hoax.

Langkah-langkah:

1. Buat kalimat teks yang mengandung kata seperti *viral* atau *mengejutkan*.
2. Jalankan Python untuk mendeteksi kata-kata itu.
3. Jika ditemukan, tampilkan "Warning: Hoax mungkin ada!".

Pseudocode:

```

jika kata "viral" atau "mengejutkan" ada di teks:
    tampilkan "Warning Hoax"

```

Python Code:

```

teks = "Fakta Mengejutkan! Semua orang bisa kaya mendadak."
if "viral" in teks or "mengejutkan" in teks.lower():
    print("Warning: Hoax mungkin ada!")

```

Aktivitas 4: Validasi Dokumen Ganda

Tujuan: Mencegah data bertentangan dalam RAG.

Langkah-langkah:

1. Buat dua dokumen berbeda, satu berisi jadwal jam 7, satu lagi jam 9.
2. Gunakan Python untuk membandingkan mana dokumen yang lebih baru.
3. Pilih jadwal dengan tanggal terbaru.

Pseudocode:

```
jika tanggal dokumen1 > dokumen2:
    gunakan dokumen1
lainnya:
    gunakan dokumen2
```

Python Code:

```
doc1 = {"jadwal": "07:00", "tanggal": 20240101}
doc2 = {"jadwal": "09:00", "tanggal": 20240105}

if doc1["tanggal"] > doc2["tanggal"]:
    print("Gunakan jadwal:", doc1["jadwal"])
else:
    print("Gunakan jadwal:", doc2["jadwal"])
```

Aktivitas 5: Checklist Etika Chatbot

Tujuan: Membuat aturan main sebelum meluncurkan chatbot.

Langkah-langkah:

1. Buka teks editor dan buat file **checklist.md**.
2. Tulis 5 aturan sederhana untuk keamanan data dan etika.
3. Simpan file itu, lalu unggah ke sistem RAG.

Contoh Isi **checklist.md:**

```
# Checklist Etika
1. Jangan unggah data pribadi.
2. Pastikan data seimbang, tidak bias.
3. Hanya gunakan sumber terpercaya.
4. Validasi dokumen sebelum dipakai.
5. Pisahkan dokumen rahasia dari publik.
```

Python Code untuk cek isi file:

```
with open("checklist.md", "r") as f:
    print(f.read())
```

Dengan 5 aktivitas ini, siswa SMK tidak hanya paham teori etika & keamanan data, tapi juga **langsung praktik** dengan *Python* sederhana dan logika dasar AI. Mereka bisa merasakan bahwa teknologi open source itu powerful sekaligus harus dipakai **bertanggung jawab**.

Rangkuman

Etika dan keamanan data adalah **pondasi utama** dalam membuat sistem *RAG* (Retriever-Augmented Generation). Tanpa etika, AI bisa menyebarkan **hoax**, menimbulkan **bias**, bahkan membocorkan **data pribadi**. Ingat, AI itu seperti **pisau**. Bisa dipakai untuk membantu (memasak) atau melukai (membocorkan rahasia). Karena itu, kita harus selalu **berpikir bijak** sebelum mengunggah dokumen.

Ada tiga hal penting yang harus diperhatikan:

- **Privasi** → jangan pernah unggah nomor HP, alamat, atau data pribadi ke chatbot.
- **Bias** → pastikan dokumen yang dipakai **seimbang**, tidak hanya mendukung satu sisi saja.
- **Disinformasi** → jangan masukkan dokumen yang berisi hoax atau berita palsu.

Dengan menjaga tiga hal ini, chatbot kita akan lebih **adil, aman, dan terpercaya**.

Semua teknologi yang kita gunakan di sini berbasis **open source**: *Python, Pandas, Machine Learning, Keras, dan Neural Network*. Itu artinya kita punya **kebebasan** untuk mempelajari, memodifikasi, bahkan memperbaiki sistem sendiri. Tapi kebebasan ini datang dengan **tanggung jawab besar**. Jadi, jadilah generasi yang pintar **menggunakan teknologi untuk kebaikan**, bukan untuk menyebarkan bahaya.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Sistem *RAG* bisa membocorkan data pribadi jika dokumen yang diunggah berisi nomor telepon siswa.
Jawaban: Benar
2. Bias data tidak berpengaruh pada hasil jawaban chatbot.
Jawaban: Salah
3. Menggunakan Ollama + OpenWebUI lebih aman karena data diproses di komputer lokal.
Jawaban: Benar
4. Chatbot tidak bisa menyebarkan hoax, meskipun diberi dokumen dari sumber palsu.
Jawaban: Salah
5. Etika dan keamanan data adalah tanggung jawab semua calon pengembang AI.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

6. Apa yang **tidak boleh** diunggah ke chatbot *RAG*?
 - a. Jadwal pelajaran umum

- b. Strategi rahasia tim futsal
- c. Artikel berita resmi
- d. Materi pembelajaran dari guru

Jawaban: b

7. Jika chatbot hanya dilatih dengan data dari kelas X, maka hasilnya?
- a. Lengkap untuk semua kelas
 - b. Bias ke kelas X
 - c. Tidak bisa digunakan
 - d. Sama saja

Jawaban: b

8. Analogi untuk menjaga data pribadi adalah:
- a. Pisau dapur
 - b. Kompas moral
 - c. Kunci brankas
 - d. Resep masakan

Jawaban: c

9. Jika chatbot membaca artikel palsu, ia akan...
- a. Menolak menjawab
 - b. Menyebarkan informasi palsu
 - c. Memperbaiki datanya sendiri
 - d. Tetap netral

Jawaban: b

10. Apa solusi jika data nilai siswa ingin dipakai dalam RAG?
- a. Unggah semua nama asli
 - b. Samarkan identitas siswa
 - c. Jangan pakai sama sekali
 - d. Tidak perlu peduli privasi

Jawaban: b

11. Apa fungsi *shield* dalam analogi konsep RAG?
- a. Membuat strategi tim
 - b. Melindungi data pribadi
 - c. Membaca dokumen hoax
 - d. Menjawab soal ujian

Jawaban: b

12. Apa akibat dari dokumen yang kontradiktif (isinya berbeda-beda)?
- a. Chatbot jadi pintar
 - b. Chatbot jadi bingung
 - c. Chatbot tetap netral
 - d. Chatbot menghapus dokumen

Jawaban: b

13. Disinformasi artinya...
- a. Informasi yang sudah dicek kebenarannya
 - b. Informasi palsu yang terlihat meyakinkan
 - c. Data yang dirahasiakan
 - d. Data yang disimpan di lokal

Jawaban: b

14. Salah satu tips cepat belajar etika AI adalah...
- a. Abaikan sumber dokumen
 - b. Gunakan analogi sederhana
 - c. Upload semua data tanpa filter
 - d. Gunakan data dari kelas X saja

Jawaban: b

15. Apa pekerjaan baru yang muncul terkait etika AI?
- a. AI Designer
 - b. AI Ethicist
 - c. AI Marketer
 - d. AI Player

Jawaban: b

C. Isian Singkat (5 Soal)

16. RAG singkatan dari _____.

Jawaban: Retriever-Augmented Generation

17. Data pribadi yang **tidak boleh** diunggah contohnya _____.

Jawaban: Nomor HP, alamat rumah

18. Chatbot bisa bias jika data hanya diambil dari _____.

Jawaban: Satu kelompok / satu kelas

19. Untuk mendeteksi pola nomor telepon di teks kita bisa menggunakan _____ di Python.

Jawaban: Regular Expression (regex)

20. Ollama dan OpenWebUI lebih aman karena semua data diproses di _____.

Jawaban: Komputer lokal

D. Soal Explorasi (3 Soal)

Soal 1:

Bayangkan kamu sedang membuat chatbot untuk OSIS sekolah. Namun, seorang teman tidak sengaja mengunggah dokumen yang berisi nomor HP guru.

- Jelaskan risiko apa yang terjadi jika dokumen itu dipakai chatbot.
- Buat *pseudocode* sederhana untuk menyaring kata “nomor_hp” agar tidak muncul di chatbot.

Contoh Jawaban:

Risiko: Nomor HP guru bisa bocor ke publik, melanggar privasi.

Pseudocode:

```
for setiap baris dokumen:
    jika "nomor_hp" ditemukan:
        ganti dengan "[privasi]"
```

Soal 2:

Kamu membuat chatbot lomba sains, tapi data pesertanya hanya laki-laki. Jelaskan kenapa chatbot ini bias. Kemudian, buat Python script sederhana untuk mendeteksi apakah teks hanya berisi kata "laki-laki".

Contoh Jawaban:

Bias karena chatbot hanya mengenal peserta laki-laki, tidak adil untuk peserta perempuan.

Python Script:

```
teks = "Peserta lomba: laki-laki"
if "laki-laki" in teks and "perempuan" not in teks:
    print("Data Bias!")
```

Soal 3:

Kamu diminta membuat sistem filter berita untuk chatbot sekolah. Sistem ini harus memberi peringatan jika ada kata "viral" atau "heboh".

- Jelaskan kenapa filter ini penting.
- Buat *Python script* sederhana untuk mendeteksi kata tersebut.

Contoh Jawaban:

Filter penting agar chatbot tidak menyebarkan berita hoax atau bombastis.

Python Script:

```
teks = "Berita ini sangat viral!"
if "viral" in teks or "heboh" in teks:
    print("Warning: Berita berpotensi hoax")
```

Dengan soal ini, siswa SMK bisa belajar **konsep etika, keamanan data, bias, privasi, dan hoax** lewat cara praktis pakai **Python open source**.

Tugas Proyek

Proyek 1: Samarkan Data Pribadi

Tantangan:

Buat program Python yang bisa mengganti nomor telepon dalam dokumen dengan tulisan `[disamarkan]`.

Langkah:

1. Siapkan teks yang berisi data siswa.
2. Cari pola nomor HP.
3. Ganti dengan `[disamarkan]`.
4. Simpan hasilnya.

Pseudocode:

```
for setiap baris dalam dokumen:
    jika ada "nomor_hp":
        ganti dengan "[disamarkan]"
```

Python Script:

```
import re

teks = "Nama: Budi, nomor_hp: 08123456789"
hasil = re.sub(r'\d{10,13}', '[disamarkan]', teks)
print(hasil)
```

Proyek 2: Deteksi Kata Bias

Tantangan:

Buat program Python yang bisa mendeteksi kata bias, misalnya hanya menyebut "laki-laki" tanpa "perempuan".

Langkah:

1. Buat list kata bias.
2. Periksa dokumen.
3. Jika ada kata bias, tampilkan peringatan.

Pseudocode:

```
list_bias = ["laki-laki saja", "khusus pria"]
untuk setiap kalimat:
    jika ada kata bias:
        tampilkan "Dokumen Bias!"
```

Python Script:

```
bias_words = ["laki-laki saja", "khusus pria"]
```

```
dokumen = "Peserta lomba hanya untuk laki-laki saja."

for kata in bias_words:
    if kata in dokumen:
        print("Dokumen Bias!")
```

Proyek 3: Filter Hoax

Tantangan:

Buat program sederhana yang memberi peringatan jika teks mengandung kata-kata seperti *viral*, *heboh*, atau *mengejutkan*.

Langkah:

1. Definisikan kata hoax.
2. Periksa dokumen.
3. Jika ada, beri warning.

Pseudocode:

```
list_hoax = ["viral", "heboh", "mengejutkan"]
jika kata ada di teks:
    print("Warning Hoax!")
```

Python Script:

```
hoax_words = ["viral", "heboh", "mengejutkan"]
teks = "Berita ini sangat viral dan mengejutkan!"

if any(word in teks.lower() for word in hoax_words):
    print("Warning: kemungkinan hoax")
```

Proyek 4: Checklist Etika Chatbot

Tantangan: Buat file Markdown (.md) yang berisi pedoman etika sebelum meluncurkan chatbot RAG di sekolah.

Langkah:

1. Buat list aturan etika.
2. Simpan dalam file Markdown.

Pseudocode:

```
buat list_pedoman
simpan list_pedoman ke file checklist.md
```

Python Script:

```
pedoman = [
    "- Jangan unggah data pribadi tanpa izin",
```

```

        "- Hindari dokumen bias",
        "- Pastikan sumber terpercaya",
        "- Validasi dokumen sebelum digunakan"
    ]

    with open("checklist.md", "w") as f:
        for p in pedoman:
            f.write(p + "\n")

```

Proyek 5: Chatbot Aman dengan Data Terverifikasi

Tantangan:

Buat sistem sederhana yang hanya menerima dokumen dari folder **verified_docs/**.

Langkah:

1. Siapkan folder **verified_docs/**.
2. Program hanya membaca file dari folder ini.
3. Jika dokumen dari folder lain → tolak.

Pseudocode:

```

folder_aman = "verified_docs/"
jika file ada di folder_aman:
    proses dokumen
lainnya:
    tolak dokumen

```

Python Script:

```

import os

folder_aman = "verified_docs/"
file = "verified_docs/peraturan_sekolah.txt"

if file.startswith(folder_aman):
    print("Dokumen diproses")
else:
    print("Dokumen ditolak")

```

Dengan 5 proyek ini, siswa SMK belajar langsung:

- Menyaring data pribadi.
- Mendeteksi bias.
- Menolak hoax.
- Membuat pedoman etika.
- Mengamankan sumber data chatbot.

Semua menggunakan **Python dan open source tools**.

BAB 7: Optimasi Penggunaan RAG di Komputer Sekolah (CPU/GPU Lokal)

Selamat datang kembali. Setelah kemarin kita berhasil bikin **chatbot** super cerdas pakai **RAG**, sekarang kita bakal jadi "tukang servis" yang handal. Di bab ini, kita akan belajar cara **mengoptimalkan** sistem **RAG** kita biar larinya kencang, terutama kalau laptop sekolah kita speknya biasa aja. Ini adalah skill yang paling dicari di industri game dan teknologi, lho, karena bisa bikin program **ringan** dan **cepat**!

Tujuan Pembelajaran

- Memahami hubungan antara **komponen komputer** (CPU, GPU, RAM) dengan performa **RAG**.
- Menganalisis dan memilih **model LLM** yang paling efisien untuk hardware lokal.
- Menggunakan trik-trik optimasi untuk mempercepat respons **chatbot RAG**.
- Menjelaskan mengapa optimasi itu penting dalam pengembangan teknologi.

Peta Konsep Sederhana

Bayangin ini seperti balapan mobil.

- **Mesin Mobil (CPU/GPU)**: Ini adalah "otak" komputer yang melakukan semua perhitungan. **CPU** itu kayak supir yang serba bisa. **GPU** itu kayak tim mekanik yang jago banget ngerjain satu tugas (misal: perhitungan AI) secara massal dan super cepat.
- **Bensin di Tangki (RAM)**: Ini adalah **memori kerja** komputer. Makin banyak RAM, makin banyak "bensin" yang bisa dipakai untuk menjalankan program.
- **Setelan Mesin (Optimasi)**: Ini adalah trik-trik yang kita lakukan, seperti memilih model AI yang lebih ringan, agar mobil kita bisa lari kencang tanpa bensin boros.

Apersepsi: Game Keren Tapi **Lag**!

Kamu mau main game terbaru, grafisnya keren banget. Tapi, laptop sekolahmu speknya terbatas. Saat main, game-nya **lag**, **nge-freeze**, bahkan **crash**.

Nah, hal yang sama bisa terjadi pada **chatbot RAG**. Kalau kita pakai model AI yang terlalu besar atau dokumen yang terlalu banyak, **chatbot** kita bisa jadi sangat lambat. Jadi, kita harus tahu cara menyyetel "mesin" AI kita agar bisa jalan lancar.

Penjelasan Teori

1) Apa itu “Optimasi” di RAG?

Bayangin kamu punya motor matic. Kalau disetel pas, motornya irit tapi tetap ngacir. Optimasi RAG artinya **setel pintar** agar chatbot kita **cepat, ringan, dan stabil** di laptop sekolah.

- **RAG = Retriever + Generator.**
 - *Retriever* (pencari): cari potongan dokumen terpenting di *vector database* (contoh: **FAISS, Chroma**).
 - *Generator* (penjawab/LLM): susun jawaban dari potongan itu (contoh: model **LLaMA, Mistral, Gemma** di **Ollama**).
- **Target optimasi:** turunkan *latency* (waktu tunggu), menaikkan *throughput* (banyaknya pertanyaan per menit), dan jaga *akurasi* tetap oke.

2) Peran Perangkat Keras (Kayak Tim Balap)

- **CPU** = sopir serba bisa. Bisa jalanin LLM kecil. Tapi **lebih lambat**.
- **GPU** = tim mekanik kilat. Jago hitung paralel. LLM jadi **lebih cepat**.
- **RAM** = bensin di tangki. Model besar = butuh banyak RAM. Jika RAM penuh → **lag** atau **crash**.
- **Storage/SSD** = garasi. Semakin cepat SSD, proses *load* model dan *indexing* dokumen makin cepat.

3) Memilih Model (Kecil vs Besar)

- Nama model sering ada angkanya: **2B, 7B, 8B, 13B, 70B** (B = miliar parameter).
- **Model kecil (2B–8B):** **cepat**, ringan, pas untuk CPU atau GPU sekolah. Cocok untuk QnA dasar.
- **Model besar (13B+):** **lebih pintar**, tapi berat. Butuh GPU kuat dan RAM besar.
- Tips: **Mulai kecil dulu**, cek hasil. Kalau kurang pintar, naikan pelan-pelan.

4) Trik “Ngirit” Wajib Tahu

- **Quantization (INT8/INT4, format GGUF)**
Turunkan presisi angka dari *float* ke *int*. Hasil: ukuran model lebih kecil, *inference* lebih cepat.
Dampak: kadang akurasi turun sedikit, tapi **worth it** untuk laptop sekolah.
- **Chunking dokumen**
Pecah dokumen jadi potongan kecil (misal 300–500 token) dengan *overlap* 10–20%.
Manfaat: pencarian relevan jadi **lebih presisi** dan **lebih cepat**.
- **Top-k & Top-p sederhana**
Jangan tarik terlalu banyak potongan (*k* terlalu besar). Cukup **k=3–5**.
- **Context window**
Jangan kirim konteks terlalu panjang ke LLM. Hemat *token*. Ambil yang benar-benar relevan saja.

- **Caching**
Simpan sementara hasil *embedding* dan *retrieval*. Pertanyaan mirip jadi **lebih cepat** dijawab.
- **Streaming**
Tampilkan jawaban sambil diketik. Terasa **lebih responsif** bagi pengguna.

5) Optimasi Retriever (FAISS/Chroma)

- Pilih **Chroma** untuk mudah (plug-and-play).
- Pilih **FAISS** untuk kecepatan tinggi dan skala lebih besar.
- Atur parameter penting:
 - **Model embedding**: pakai yang ringan *open source*.
 - **Dimensi vektor**: ikut *embedding model* yang dipakai.
 - **Metric**: *cosine* atau *L2*—uji mana yang paling pas.
 - **Indexing**: lakukan 1x dengan benar, simpan, dan **jangan ulang** setiap kali aplikasi start.

6) Optimasi Generator (LLM di Ollama + OpenWebUI)

- Pilih model **ter-quantized** (contoh: **:Q4_K_M** di **Ollama**).
- **Batasi panjang jawaban** (*max tokens*) agar tidak boros.
- **Pakai system prompt ringkas** dan fokus.
- **Matikan fitur berat** yang tidak perlu (misal *tool use* kalau tidak dipakai).

7) Batas Aman di Komputer Sekolah

- **RAM 4–8 GB + tanpa GPU** → model mini (2B–3B) + *quantization INT4*.
- **RAM 8–16 GB + iGPU/GTX lawas** → 3B–7B *quantized*.
- **Ada GPU RTX kelas menengah** → 7B–13B *quantized* bisa masuk.

8) Cara Ukur “Sudah Optimal atau Belum”

- **Metrik praktis** yang gampang dicek:
 - *Latency* (detik per jawaban) → makin kecil makin baik.
 - *Peak RAM* (GB) → jangan lewat batas.
 - *Hit rate cache* (%) → makin tinggi makin cepat untuk pertanyaan berulang.
 - *Akurasi subjektif* → apakah jawaban pas dengan dokumen?

Checklist Cepat (Biar “Ngebut” Tanpa Drama)

- **Mulai dari model kecil** (*Gemma 2B, Mistral 7B* versi *quantized*) di **Ollama**.
- **Pakai quantization INT4/INT8** (format **GGUF**).
- **Chunk 300–500, overlap 10–20%, top-k=3–5**.
- **Gunakan Chroma/FAISS** dan **simpan index**.
- **Aktifkan caching embeddings** dan hasil *retrieval*.
- **Batasi context** dan **max tokens** jawaban.
- **Monitoring**: buka *Task Manager* (Windows) atau *htop* (Linux) saat uji.

- **Uji A/B:** bandingkan 2 model, pilih yang **lebih cepat** tapi **jawabannya tetap masuk akal**.

Ilustrasi Mini (Ringan, tapi Ngena)

- **Motor kecil tapi lincah** → model 2B *quantized*, pas buat sekolah.
- **Bensin pas, bagasi lega** → RAM cukup + *chunking* rapi.
- **Peta jelas** → *retriever* setelan tepat (FAISS/Chroma + *top-k* kecil).
- **Supir disiplin** → *prompt* jelas, *max tokens* dibatasi.
- **Pit stop cepat** → *cache* diaktifkan, *streaming* respons.

Bonus: Skema Alur Otak RAG (Singkat)

1. Dokumen di-*chunk* → di-*embedding* → masuk **Chroma/FAISS**.
2. Pertanyaan jadi vektor → **cari top-k** potongan terdekat.
3. Potongan + pertanyaan → masuk LLM (**Ollama**) → jawaban keluar di **OpenWebUI**.
4. Aktifkan *cache* di langkah 1–3 agar **ulangannya makin ngebut**.

Kasus Ilustrasi

Kasus 1: RAG Kilat di Laptop CPU 4 GB

Situasi: Laptop cuma ada *CPU*. RAM 4 GB. Jawaban chatbot lama.

Tujuan: Cepat dan stabil.

Strategi singkat:

- Pilih model kecil (mis. **gemma:2b** di **Ollama**) dengan *quantization* INT4.
- *Chunking* dokumen **300–500** token, *overlap* **10–20%**.
- Batasi *top-k* = **3–5** dan *context window* pendek.
- Simpan *index* biar tidak bikin ulang.

Pseudocode:

```
SET model = "gemma:2b:Q4"
SET chunk_size = 400
SET overlap = 80
SET top_k = 3
LOAD or BUILD vector_index
FOR each question:
    chunks = RETRIEVE(question, top_k)
    answer = GENERATE(model, question, chunks, max_tokens=256)
RETURN answer
```

Python (mini demo Chroma + sentence-transformers):

```
# pip install chromadb sentence-transformers
```

```

import chromadb, time
from sentence_transformers import SentenceTransformer

embed = SentenceTransformer("all-MiniLM-L6-v2") # ringan & open
source
chroma = chromadb.Client().create_collection("school_docs")

def add_doc(doc_id, text):
    vec = embed.encode([text])[0].tolist()
    chroma.add(documents=[text], embeddings=[vec], ids=[doc_id])

def ask(q, top_k=3):
    qv = embed.encode([q])[0].tolist()
    hits = chroma.query(query_embeddings=[qv], n_results=top_k)
    context = "\n".join(hits["documents"][0])
    # kirim ke Ollama via OpenWebUI (disederhanakan di sini)
    return f"[CTX]\n{context}\n[ANS] Ringkas & tepat."

t0 = time.time()
print(ask("Apa isi bab pendahuluan?"))
print("Latency:", round(time.time()-t0, 2), "detik")

```

Kasus 2: GPU Pas-Pasan, Tetap Ngebut

Situasi: Ada *GPU*, tapi kecil. Model **llama3:8b** sering *crash*.

Tujuan: Tetap jalan, tanpa *crash*.

Strategi singkat:

- Deteksi GPU dulu.
- Jika VRAM kecil → pakai model **lebih kecil** atau *quantized* (INT4).
- Kurangi *max tokens*.
- Gunakan *batching* kecil di *retriever*.

Pseudocode (auto pilih model):

```

IF gpu_vram < 6GB:
    model = "mistral:7b:Q4"
ELSE:
    model = "llama3:8b:Q4"
SET max_tokens = 256
RUN RAG(model, max_tokens)

```

Python (logika pemilihan sederhana):

```

# pilih model berdasar "perkiraan" VRAM
def pick_model(vram_gb):
    if vram_gb < 6:
        return "mistral:7b:Q4"
    return "llama3:8b:Q4"

print("Model pakai:", pick_model(vram_gb=4))

```

Kasus 3: Retrieval Lemot karena Indexing Salah

Situasi: Dokumen cuma 1–2, tapi *retrieval* super lambat.

Akar masalah: *Indexing* diulang terus, atau index tidak disimpan.

Tujuan: *Retrieval* < 1 detik.

Strategi singkat:

- Bangun **sekali** *vector index* (FAISS/Chroma), lalu **persist**.
- Pastikan dimensi *embedding* konsisten.
- Gunakan metrik *cosine* dan *top-k* kecil.

Pseudocode:

```
IF index_not_exists:
    BUILD index with embeddings
    SAVE index to disk
ELSE:
    LOAD index from disk
RETURN SEARCH(query_vector, top_k=3)
```

Python (FAISS persist sederhana):

```
# pip install faiss-cpu sentence-transformers
import faiss, numpy as np, pickle
from sentence_transformers import SentenceTransformer

embed = SentenceTransformer("all-MiniLM-L6-v2")
docs = ["Visi sekolah ...", "Aturan laboratorium ..."]

X = embed.encode(docs).astype("float32")
index = faiss.IndexFlatIP(X.shape[1]) # cosine ~ inner product
jika vektor dinormalisasi
faiss.normalize_L2(X)
index.add(X)

# simpan
faiss.write_index(index, "school.index")
pickle.dump(docs, open("docs.pkl", "wb"))

# load & query
idx = faiss.read_index("school.index")
docs_loaded = pickle.load(open("docs.pkl", "rb"))
q = "Apa aturan lab?"
qv = embed.encode([q]).astype("float32")
faiss.normalize_L2(qv)
D, I = idx.search(qv, k=3)
print([docs_loaded[i] for i in I[0]])
```

Kasus 4: Chatbot Boros RAM (Sering Crash)

Situasi: Saat jalan, RAM tiba-tiba 99% → *crash*.

Tujuan: Hemat RAM, tetap akurat.

Strategi singkat:

- Ganti model ke **lebih kecil** atau *quantized*.
- Kecilkan *context window* dan *max tokens*.
- Tutup aplikasi lain.
- Batasi jumlah dokumen yang ikut ke *prompt* (mis. maksimal 3 *chunks*).

Pseudocode:

```
IF RAM_usage > 85%:  
    REDUCE max_tokens  
    REDUCE top_k  
    SWITCH model -> smaller_quantized  
    LIMIT context_chunks = 3  
RETRY ask()
```

Python (cek RAM cepat pakai psutil):

```
# pip install psutil  
import psutil  
ram = psutil.virtual_memory()  
if ram.percent > 85:  
    max_tokens = 192  
    top_k = 3  
else:  
    max_tokens = 256  
    top_k = 5  
print("Setting:", max_tokens, top_k)
```

Kasus 5: A/B Test: Mistral vs Gemma (Pilih yang Paling Ngebut)

Situasi: Bingung pilih **Mistral** atau **Gemma** untuk laptop sekolah.

Tujuan: Pilih model **paling cepat** dengan jawaban tetap **masuk akal**.

Strategi singkat:

- Siapkan **daftar 5 pertanyaan** tetap.
- Ukur **latency** tiap model.
- Cek **kualitas** (mis. ada kata kunci yang harus muncul).
- Pilih yang **latency** kecil dan **kualitas** cukup.

Pseudocode:

```
questions = [q1..q5]  
models = ["mistral:7b:Q4", "gemma:2b:Q4"]  
FOR m in models:  
    total_time = 0  
    score = 0  
    FOR q in questions:
```

```

t = TIME(ANSWER(m, q))
total_time += t
IF keyword in answer: score += 1
REPORT m, avg_time = total_time/len(questions), score
PICK model with min(avg_time) AND score >= threshold

```

Python (kerangka uji sederhana):

```

import time, random

def fake_ask(model, q):
    # simulasi waktu berbeda-beda
    base = 0.9 if "gemma" in model else 1.2
    time.sleep(base + random.uniform(0.0, 0.2))
    return f"Jawaban {model} untuk: {q}"

qs = ["Apa visi sekolah?", "Aturan lab?", "Jadwal OSIS?",
      "Cara pinjam buku?", "SOP lab komputer?"]
models = ["mistral:7b:Q4", "gemma:2b:Q4"]

for m in models:
    ttot, hits = 0.0, 0
    for q in qs:
        t0 = time.time()
        ans = fake_ask(m, q)
        ttot += (time.time() - t0)
        if "Jawaban" in ans: hits += 1 # dummy kualitas
    print(m, "avg latency:", round(ttot/len(qs), 2),
          "score:", hits, "/5")

```

Catatan

- Gunakan model **kecil + quantization** (mis. **gemma:2b:Q4**) untuk laptop sekolah.
- Rapikan *retrieval* dengan **Chroma/FAISS**, *chunking* **400/overlap 80**, *top-k* **3–5**.
- **Persist index**, aktifkan *cache*, batasi *context window* & *max tokens*.
- Selalu **ukur**: *latency*, RAM, kualitas jawaban.
- Lakukan **A/B test**. Pilih yang **cepat** dan **cukup akurat**.

Soal Latihan & Pembahasan

Soal 1 — Pilih Model Biar Ngebut di CPU 4 GB

Deskripsi singkat: Laptop hanya *CPU*, *RAM* 4 GB. *Chatbot RAG* lambat.

Pertanyaan:

Model mana yang lebih tepat dipakai, dan setelan apa yang perlu diubah agar jawaban cepat?

Pseudocode:

```
SET hardware = "CPU_only"
IF RAM <= 4GB:
    model = "gemma:2b:Q4"          # kecil + quantization
    chunk_size = 400
    overlap = 80
    top_k = 3
    max_tokens = 200
RETURN model, chunk_size, overlap, top_k, max_tokens
```

Jawaban singkat:

Pakai model kecil **gemma:2b:Q4**, *chunking* 400/overlap 80, *top-k* 3, *max_tokens* 200.

Pembahasan:

Model kecil + *quantization* menghemat *RAM* dan menurunkan *latency*. *Chunking* sedang dan *top-k* kecil menjaga *context window* ringkas, jadi generasi teks lebih cepat di *CPU*.

Python mini (opsional):

```
model = "gemma:2b:Q4"
chunk_size, overlap = 400, 80
top_k, max_tokens = 3, 200
print(model, chunk_size, overlap, top_k, max_tokens)
```

Soal 2 — *Retrieval* Lemot Padahal Dokumen Cuma Satu

Deskripsi singkat: *Retriever* lama banget. Padahal hanya 1 file PDF.

Pertanyaan:

Apa langkah paling aman agar *retrieval* cepat dan tidak membangun ulang *index*?

Pseudocode:

```
IF index_exists("school.index"):
    index = LOAD("school.index")
ELSE:
    embeddings = ENCODE(documents)          #
    sentence-transformers
```

```

        index = BUILD_FAISS(embeddings)           # vector database
        SAVE(index, "school.index")
    q_vec = ENCODE(query)
    hits = SEARCH(index, q_vec, top_k=3)
    RETURN hits

```

Jawaban singkat:

Bangun *vector index* sekali (FAISS/Chroma), **persist** ke disk, lalu **load** saat tanya.

Pembahasan:

Masalahnya biasanya *indexing* diulang setiap tanya. Dengan menyimpan *index* (persist), *retrieval* hanya *load + search*, jadi cepat.

Python mini (opsional):

```

# konsep persist FAISS
import faiss, pickle, numpy as np
# ... bangun lalu:
faiss.write_index(index, "school.index")
pickle.dump(docs, open("docs.pkl", "wb"))

```

Soal 3 — GPU Kecil, Model Sering Crash

Deskripsi singkat: Ada *GPU*, tapi VRAM kecil. **llama3:8b** sering *crash*.

Pertanyaan:

Bagaimana memilih model dan setelan agar tetap jalan stabil?

Pseudocode:

```

IF vram_gb < 6:
    model = "mistral:7b:Q4"
    max_tokens = 192
    batch = 1
ELSE:
    model = "llama3:8b:Q4"
    max_tokens = 256
    batch = 2
top_k = 3
RETURN model, max_tokens, batch, top_k

```

Jawaban singkat:

Turunkan ke **mistral:7b:Q4**, kecilkan *max_tokens*, *batch* 1, *top-k* 3.

Pembahasan:

VRAM kecil harus pakai model lebih ringan dan *quantized*. *Max tokens* kecil mengurangi memori generasi. *Batch* 1 menekan lonjakan VRAM. Hasilnya stabil.

Python mini (opsional):

```

def pick_model(vram_gb):
    return "mistral:7b:Q4" if vram_gb < 6 else "llama3:8b:Q4"

```

```
print(pick_model(4))
```

Soal 4 — RAM 99% Lalu *Crash*

Deskripsi singkat: Saat jalan, *RAM* penuh, aplikasi berhenti.

Pertanyaan:

Parameter mana yang harus diturunkan dulu, dan kenapa?

Pseudocode:

```
IF RAM_usage > 85%:
    max_tokens = max(128, max_tokens - 128)
    top_k = min(top_k, 3)
    context_chunks = 3
    SWITCH model -> smaller_quantized
RETURN new_settings
```

Jawaban singkat:

Turunkan **max_tokens** dulu, batasi **top-k** dan **context_chunks**, lalu ganti model ke yang **lebih kecil + quantized**.

Pembahasan:

Max tokens paling langsung berpengaruh ke memori generasi. *Top-k* dan jumlah *chunks* mengatur panjang konteks. Model kecil + *quantization* menurunkan jejak *RAM*.

Python mini (opsional):

```
import psutil
ram = psutil.virtual_memory().percent
max_tokens, top_k, context_chunks = 256, 5, 5
if ram > 85:
    max_tokens = 128
    top_k = 3
    context_chunks = 3
print(max_tokens, top_k, context_chunks)
```

Soal 5 — A/B Test: Mistral vs Gemma untuk *Latency* Tercepat

Deskripsi singkat: Mau pilih model tercepat tapi jawaban tetap masuk akal.

Pertanyaan:

Rancang uji sederhana untuk membandingkan *latency* dan kualitas jawaban 2 model.

Pseudocode:

```
questions = [q1..q5]
models = ["mistral:7b:Q4", "gemma:2b:Q4"]
FOR each model m:
```

```

total_time = 0
score = 0
FOR each q in questions:
    t_start = NOW()
    ans = GENERATE(m, q, top_k=3, max_tokens=200)
    total_time += (NOW()-t_start)
    IF contains_required_keywords(ans): score += 1
report[m] = (avg_time = total_time/len(questions), quality =
score)
PICK model with min(avg_time) AND quality >= threshold

```

Jawaban singkat:

Ukur **rata-rata *latency*** dan **skor kualitas** (berdasar kata kunci/cek isi). Pilih model dengan *latency* terendah dan skor $\geq$ ambang.

Pembahasan:

Benchmark kecil seperti ini membantu keputusan cepat. *Gemma 2B* sering lebih cepat di *CPU*. *Mistral 7B* bisa lebih pintar, tapi cek *hardware*. Pilih yang **cukup akurat** dan **paling cepat** di perangkat kalian.

Python mini (opsional, pakai **Pandas** untuk tabel):

```

import time, random, pandas as pd

qs = ["Visi sekolah?", "Aturan lab?", "Jadwal OSIS?",
      "Peminjaman buku?", "SOP lab?"]
models = ["mistral:7b:Q4", "gemma:2b:Q4"]
rows = []
for m in models:
    ttot, score = 0.0, 0
    for q in qs:
        t0 = time.time()
        time.sleep((1.2 if "mistral" in m else 0.9) +
random.uniform(0,0.2)) # simulasi
        ans = "jawaban yang memuat kata kunci"
        ttot += time.time()-t0
        score += 1 if "kata kunci" in ans else 0
    rows.append({"model": m, "avg_latency_s":
round(ttot/len(qs),2), "quality_score": score})
print(pd.DataFrame(rows))

```

Catatan

- Selalu *persist index* di *vector database* (FAISS/Chroma).
- Pakai *embedding* ringan (*all-MiniLM-L6-v2*) untuk hemat komputasi.
- Model kecil + *quantization* (INT4/INT8) = **ringan**, cocok *CPU/GPU* pas-pasan.
- Tuning utama: **max_tokens**, **top-k**, **context window**, jumlah *chunks*.
- Coba-coba, ukur, catat pakai **Pandas**. Data bicara, bukan feeling.

Fun Facts!

- Di dunia nyata, perusahaan teknologi besar menghabiskan jutaan dollar hanya untuk mengoptimalkan model AI agar bisa berjalan efisien.
- Laptop gaming modern sekarang punya **chip** khusus yang didesain untuk mempercepat AI.
- **Ollama** punya fitur yang bisa mendeteksi **hardware** di laptopmu dan memberikan saran model yang paling cocok.

Tips Belajar atau Menghafal Cepat

- **Kenali Hardware-mu:** Ketik **dxdiag** di Windows untuk tahu spesifikasi komputermu. Ini adalah langkah pertama yang paling penting.
- **Coba-Coba:** Jangan takut bereksperimen. Coba unduh model yang berbeda dan lihat perbedaannya.
- **Gunakan Task Manager:** Saat menjalankan **chatbot**, buka **Task Manager** dan lihat bagaimana CPU, GPU, dan RAM-mu bekerja.

Aktivitas Siswa

Aktivitas 1 — Cek Spek & Rekomendasi Model Otomatis

Tujuan: mengenali *CPU/GPU/RAM* dan memilih model yang pas.

Langkah:

- Buka editor *Python*.
- Salin kode di bawah. Jalankan.
- Catat saran model dan setelan.

Pseudocode:

```
READ cpu_info, gpu_info, ram_total
IF only_CPU OR RAM <= 4GB:
    model = "gemma:2b:Q4"
    max_tokens = 200
ELSE IF GPU_VRAM < 6GB:
    model = "mistral:7b:Q4"
    max_tokens = 192
ELSE:
    model = "llama3:8b:Q4"
SET chunk_size=400, overlap=80, top_k=3
PRINT rekomendasi
```

Contoh Python:

```

import platform, psutil

def rekomendasi():
    ram_gb = psutil.virtual_memory().total / (1024**3)
    has_gpu = False # sederhana: isi True jika ada GPU (opsional
deteksi lanjut)
    vram_gb = 0      # isi manual kalau tahu VRAM

    if ram_gb <= 4 or not has_gpu:
        model, max_tokens = "gemma:2b:Q4", 200
    elif has_gpu and vram_gb < 6:
        model, max_tokens = "mistral:7b:Q4", 192
    else:
        model, max_tokens = "llama3:8b:Q4", 256

    cfg = dict(model=model, chunk_size=400, overlap=80, top_k=3,
max_tokens=max_tokens)
    return ram_gb, cfg

ram_gb, cfg = rekomendasi()
print(f"RAM terdeteksi ~{ram_gb:.1f} GB")
print("Saran:", cfg)

```

Jawaban contoh (yang diharapkan tercetak):

- “Saran: gemma:2b:Q4, chunk_size=400, top_k=3, max_tokens=200.”

Aktivitas 2 — Benchmark Latency Sederhana 2 Model

Tujuan: membandingkan kecepatan jawaban (*latency*) dua model kecil.

Langkah:

- Siapkan 5 pertanyaan pendek.
- Simulasikan *latency* (tanpa koneksi Ollama).
- Hitung rata-rata waktu.

Pseudocode:

```

questions = [5 pertanyaan]
models = ["mistral:7b:Q4", "gemma:2b:Q4"]
FOR each model:
    total_time = 0
    FOR q in questions:
        start = now()
        simulate_generate() # tidur 0.9s / 1.2s
        total_time += now()-start
    avg = total_time/len(questions)
PRINT tabel avg_latency
PILIH model dengan avg_latency terkecil

```

Contoh Python (pakai *Pandas* untuk tabel):

```

import time, random, pandas as pd

qs = ["Apa visi sekolah?", "Aturan lab?", "Jam perpustakaan?",
      "Prosedur OSIS?", "SOP lab?"]
models = ["mistral:7b:Q4", "gemma:2b:Q4"]

rows = []
for m in models:
    total = 0.0
    for q in qs:
        t0 = time.time()
        time.sleep((1.2 if "mistral" in m else 0.9) +
                    random.uniform(0,0.2)) # simulasi
        total += time.time() - t0
    rows.append({"model": m, "avg_latency_s": round(total/len(qs),
2)})

df = pd.DataFrame(rows).sort_values("avg_latency_s")
print(df)

```

Jawaban contoh (hasil yang diharapkan):

- Tabel menampilkan **gemma:2b:Q4** lebih cepat ($\approx 0.9\text{--}1.1$ s) daripada **mistral:7b:Q4** ($\approx 1.2\text{--}1.4$ s).

Aktivitas 3 — Chunking & top-k untuk Retrieval Cepat

Tujuan: menguji pengaruh *chunk size* dan *top-k* pada kecepatan cari.

Langkah:

- Siapkan daftar paragraf teks (simulasi dokumen).
- Bangun *embedding* (simulasi vektor acak agar ringan).
- Ubah *chunk_size* dan *top-k*, ukur waktu cari.

Pseudocode:

```

docs = LOAD_texts()
FOR chunk_size in [200, 400, 800]:
    chunks = SPLIT(docs, chunk_size, overlap=chunk_size*0.2)
    index = BUILD_FAISS(chunks_embeddings)
    FOR top_k in [3,5,10]:
        t = measure(SEARCH(index, query, top_k))
        LOG(chunk_size, top_k, t)
PICK kombinasi tercepat dengan hasil relevan

```

Contoh Python (simulasi ringan):

```

import time, random, pandas as pd

def simulate_search(n_chunks, top_k):

```

```

    # makin banyak chunk -> cari lebih lama, makin besar top_k ->
    ambil lebih lama
    base = 0.05 + n_chunks*0.0005 + top_k*0.002
    time.sleep(base + random.uniform(0, 0.01))
    return base

results = []
for chunk_size in [200, 400, 800]:
    n_chunks = 2000 // chunk_size * 50 # simulasi jumlah chunk
    for top_k in [3, 5, 10]:
        t = simulate_search(n_chunks, top_k)
        results.append({"chunk_size": chunk_size, "top_k": top_k,
            "time_s": round(t, 3)})

df = pd.DataFrame(results).sort_values("time_s")
print(df)

```

Jawaban contoh (yang diharapkan):

- Kombinasi **chunk_size=400 & top_k=3** sering jadi **paling cepat** dengan hasil masih relevan.

Aktivitas 4 — Penghemat RAM: Turunkan max_tokens + quantization

Tujuan: mencegah *crash* saat *RAM* mepet.

Langkah:

- Cek penggunaan *RAM*.
- Jika >85%, kecilkan **max_tokens**, **top_k**, dan jumlah *context chunks*.
- Pakai model *quantized* (misal Q4).

Pseudocode:

```

ram = READ_RAM_percent()
IF ram > 85%:
    max_tokens = 128
    top_k = 3
    context_chunks = 3
    model = smaller_quantized
PRINT konfigurasi hemat RAM

```

Contoh Python:

```

import psutil
max_tokens, top_k, context_chunks = 256, 5, 5
ram = psutil.virtual_memory().percent

if ram > 85:
    max_tokens = 128
    top_k = 3
    context_chunks = 3

```

```

        model = "gemma:2b:Q4"
    else:
        model = "mistral:7b:Q4"

    print({"ram_percent": ram, "model": model, "max_tokens": max_tokens,
          "top_k": top_k, "context_chunks": context_chunks})

```

Jawaban contoh (yang diharapkan):

- Saat *RAM* tinggi, keluaran merekomendasikan **gemma:2b:Q4**, **max_tokens=128**, **top_k=3**, **context_chunks=3**.

Aktivitas 5 — Routing Otomatis: Pilih Jalur CPU atau GPU

Tujuan: jalankan model via *GPU* kalau ada, kalau tidak pakai *CPU*.

Langkah:

- Cek ketersediaan *GPU* (*torch*).
- Jika ada *GPU*, gunakan *batch* lebih besar dan *max_tokens* sedikit lebih longgar.
- Jika tidak, pakai *CPU* dengan setelan hemat.

Pseudocode:

```

IF torch.cuda.is_available():
    device = "cuda"
    model = "mistral:7b:Q4"
    batch = 2
    max_tokens = 256
ELSE:
    device = "cpu"
    model = "gemma:2b:Q4"
    batch = 1
    max_tokens = 200
PRINT rute, model, setelan

```

Contoh Python:

```

try:
    import torch
    has_cuda = torch.cuda.is_available()
except Exception:
    has_cuda = False

if has_cuda:
    route = "GPU"
    model, batch, max_tokens = "mistral:7b:Q4", 2, 256
else:
    route = "CPU"
    model, batch, max_tokens = "gemma:2b:Q4", 1, 200

```

```
print({"route": route, "model": model, "batch": batch, "max_tokens":  
max_tokens})
```

Jawaban contoh (yang diharapkan):

- Jika ada *GPU*: “route: **GPU**, model: **mistral:7b:Q4**, batch: 2, max_tokens: 256.”
- Jika tidak ada *GPU*: “route: **CPU**, model: **gemma:2b:Q4**, batch: 1, max_tokens: 200.”

Tips kecil biar makin ngebut

- Persist *index* di *vector database* (FAISS/Chroma) supaya *retrieval* cepat.
- Pakai *embedding* ringan (misal *all-MiniLM-L6-v2*).
- Atur **chunk_size** ~ 300–500, **overlap** 15–20%, **top-k** 3–5.
- Gunakan *quantization* (Q4/Q5) untuk hemat *RAM/VRAM*.
- Catat hasil uji di *Pandas* biar rapi. Data yang bicara, bukan perasaan.

Rangkuman

Berani ngebut dengan RAG di lab sekolah? Intinya begini: **pilih model yang pas dengan spek**. Pahami peran *CPU*, *GPU*, dan *RAM*. *CPU* itu serba bisa. *GPU* itu jurus cepat untuk matematika paralel. *RAM* itu “bensin” saat model jalan. **Model kecil lebih ringan dan cepat**, model besar lebih pintar tapi haus sumber daya. Semua bisa dikendalikan dengan trik *optimasi* sederhana.

Agar lancar, fokus pada tiga hal: **model, potongan dokumen, dan batas keluaran**. Pakai model *open source* yang ringan, misal *gemma:2b* ter-*quantized*. Atur *chunking* rapi (sekitar 300–500 kata, overlap 15–20%). Simpan vektor di *vector database* seperti *FAISS* atau *Chroma* agar *retrieval* cepat. Batasi *max_tokens* dan *top-k* supaya **latensi turun** dan **RAM aman**. Untuk olah data kecil, *Python* + *Pandas* sudah cukup. Kalau butuh *machine learning* lanjut, coba *Keras* dan bangun *neural network* mini di lokal.

Selalu **ukur, catat, bandingkan**. Lihat *Task Manager* untuk cek *CPU/GPU/RAM*. Uji dua model berbeda, hitung waktu respons, pilih yang **stabil dan cepat** di komputer kalian. Ingat, **optimasi itu kunci**: atur *context window*, kurangi aplikasi lain, dan pakai *quantization* (misal Q4/Q5). Dengan kebiasaan eksperimen kecil dan tool *open source*—*Python*, *Pandas*, *machine learning*, *Keras*, *neural network*—kalian bisa bikin chatbot RAG yang **ringan, kencang, dan siap dipakai** di komputer sekolah.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. CPU bisa menjalankan model LLM, tetapi biasanya lebih lambat dari GPU.
Jawaban: Benar
2. Menambah RAM tidak berpengaruh pada stabilitas saat model besar berjalan.
Jawaban: Salah
3. Model lebih kecil biasanya memberi respons lebih cepat di komputer sekolah.
Jawaban: Benar
4. Mengunggah banyak dokumen tanpa *indexing* tetap membuat *retrieval* cepat.
Jawaban: Salah
5. *Quantization* (misal Q4/Q5) dapat menurunkan kebutuhan memori model.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Komponen yang paling cocok untuk perhitungan paralel AI adalah...
A. SSD B. CPU C. GPU D. PSU
Jawaban: C
2. Jika RAM 4 GB, model yang paling aman dicoba lebih dulu...
A. 70B B. 8B C. 2B D. 13B
Jawaban: C
3. Tugas RAM saat model berjalan adalah...
A. Menyimpan listrik B. Menyimpan model dan tensor kerja
C. Menampilkan gambar D. Mengunduh model
Jawaban: B
4. *Chunk* terlalu besar dapat menyebabkan...
A. *Retrieval* cepat B. Latensi naik
C. RAM lebih kecil D. Jawaban selalu akurat
Jawaban: B
5. Untuk mempercepat pencarian vektor lokal, pilihan *open source* adalah...
A. FAISS atau Chroma B. Excel C. Photoshop D. Premiere
Jawaban: A
6. Di laptop tanpa GPU, strategi terbaik adalah...
A. Memaksa 70B B. Pakai model kecil dan batasi *top_k*
C. Tambah efek visual D. Naikkan resolusi layar
Jawaban: B
7. Manakah setelan yang paling membantu menurunkan latensi?
A. *max_tokens* sangat besar B. *top_k* kecil
C. *context window* tidak terbatas D. Tidak pakai *cache*

Jawaban: B

8. tool antarmuka *open source* yang nyaman untuk RAG lokal adalah...
A. OpenWebUI B. Spreadsheet C. VLC D. Notepad

Jawaban: A

9. Jika model *crash* saat *load*, kemungkinan besar karena...
A. Keyboard rusak B. RAM habis
C. Internet lambat D. Layar mati

Jawaban: B

10. Pilih baris kode Python yang benar untuk ukur waktu sederhana:
A. `t = now.time()` B. `time = stamp()`
C. `import time; t0=time.time()` D. `clock=tick()`

Jawaban: C

C. Isian Singkat (5 Soal)

1. GPU cocok untuk komputasi _____ skala besar.
Jawaban: paralel
2. Model 8B sering lebih berat daripada model _____.
Jawaban: 2B
3. Basis data vektor yang populer dan *open source* adalah _____ atau _____.
Jawaban: FAISS, Chroma
4. Mengurangi ukuran model dengan Q4/Q5 disebut _____.
Jawaban: quantization
5. Membagi dokumen menjadi potongan kecil untuk dicari disebut _____.
Jawaban: chunking

D. Soal Explorasi (3 Soal)

Soal 1:

Rancang eksperimen membandingkan dua model (2B vs 8B) di laptop CPU-only. Catat waktu respons untuk 5 pertanyaan dan jumlah RAM terpakai. Tulis ringkasan 3 kalimat.

Kunci Jawaban (contoh):

Model 2B rata-rata 0,4 s; 8B rata-rata 0,8 s.

RAM 2B $\pm 2,5$ GB; 8B $\pm 5-6$ GB.

Saya memilih 2B untuk sekolah karena lebih cepat dan stabil di CPU.

Soal 2:

Buat skrip Python mini yang mengaktifkan cache jawaban. Tanyakan tiga kali pertanyaan yang sama dan tampilkan status HIT/MISS serta total waktu. Simpulkan

dalam 2 kalimat.

Kunci Jawaban (contoh):

HIT muncul pada panggilan ke-2 dan ke-3; total waktu turun signifikan. Cache efektif untuk pertanyaan berulang di RAG lokal.

Contoh kode:

```
import time
cache={}
def ask(q):
    if q in cache: return cache[q], "HIT"
    time.sleep(0.4); cache[q]="ok"; return "ok","MISS"
for i in range(3): print(ask("apa itu rag?")[1])
```

Soal 3:

Uji 3 ukuran chunk (200/400/600) pada satu dokumen panjang. Ambil top_k=3 skor paling relevan (boleh disimulasikan). Pilih ukuran terbaik dan beri alasan 2 kalimat.

Kunci Jawaban (contoh):

Chunk 400 memberi skor rata-rata tertinggi dan jumlah chunk sedang. Cukup konteks, tetap cepat saat retrieval.

Tugas Proyek

Proyek 1 — Speed-O-Meter RAG: ukur latensi model kecil vs besar

Tujuan: membandingkan kecepatan dua model (misal *gemma:2b* vs *llama3:8b*) secara sederhana.

Fokus: waktu respons, RAM, log ke CSV pakai *Pandas*.

Langkah:

- Tentukan 2 “model” (simulasi nama saja).
- Siapkan 5 pertanyaan singkat.
- Ukur waktu jawab (simulasi dengan `time.sleep()` beda durasi).
- Simpan hasil ke `latensi.csv`.
- Bandingkan rata-rata waktu.

Pseudocode:

```
SET models = ["gemma:2b", "llama3:8b"]
SET questions = ["apa itu rag?", "fungsi gpu?", "apa itu ram?",
"chunk itu apa?", "cara cepat?"]
FOR each model IN models:
    FOR each q IN questions:
        start = waktu sekarang
        jawab = panggil_model(model, q) // disimulasikan
        end = waktu sekarang
        simpan ke tabel: model, q, (end - start)
HITUNG rata-rata per model
TAMPILKAN model tercepat
```

Contoh Python:

```
import time, random
import pandas as pd

def fake_answer(model, q):
    base = 0.25 if "gemma" in model else 0.55 # simulasi: model
    kecil lebih cepat
    time.sleep(base + random.uniform(0.0, 0.15))
    return "jawaban simulasi"

models = ["gemma:2b", "llama3:8b"]
questions = ["apa itu rag?", "fungsi gpu?", "apa itu ram?", "apa
itu chunk?", "cara cepat?"]

rows = []
for m in models:
    for q in questions:
        t0 = time.time()
        _ = fake_answer(m, q)
        t1 = time.time()
        rows.append({"model": m, "question": q, "latency_s":
round(t1 - t0, 3)})

df = pd.DataFrame(rows)
df.to_csv("latensi.csv", index=False)
print(df.groupby("model")["latency_s"].mean())
```

Kunci Jawaban (contoh):

- **Gemma:2b** rata-rata **≈0.35–0.40 s**.
- **Llama3:8b** rata-rata **≈0.65–0.75 s**.
- **Kesimpulan:** model kecil lebih cepat, cocok untuk *CPU* sekolah.

Proyek 2 — RAM Guardian: rekomendasi model berdasarkan RAM

Tujuan: memberi saran ukuran model aman dari kapasitas *RAM* (rule of thumb).

Fokus: perhitungan sederhana + margin aman.

Langkah:

- Cek *RAM* (boleh lihat Task Manager, atau input manual).
- Gunakan aturan: **model_ukuran_GB ≤ 0.4 × RAM_GB** untuk aman di *CPU* (simulasi).
- Keluarkan saran model (2B, 7/8B, dst).

Pseudocode:

```
INPUT ram_gb
limit = 0.4 * ram_gb
IF limit < 3: saran = "gemma:2b / tinydolphin"
ELIF limit < 8: saran = "llama3:8b (hati-hati)"
```

```
ELSE: saran = "model > 8b"
PRINT saran
```

Contoh Python:

```
def saran_model(ram_gb):
    limit = 0.4 * ram_gb
    if limit < 3:
        return "Saran: gemma:2b atau model 2-3GB (quantized Q4/Q5)."
```

Q4/Q5)."

```
    elif limit < 8:
        return "Saran: llama3:8b quantized. Tutup aplikasi lain."
    else:
        return "Saran: bisa coba 8-13B quantized. Pantau RAM."
print(saran_model(4))
print(saran_model(8))
```

Kunci Jawaban (contoh):

- RAM **4GB** → **gemma:2b**.
- RAM **8GB** → **llama3:8b (quantized)**, tapi tetap pantau *RAM*.
- **Inti: jangan memaksa model besar** di *RAM* kecil.

Proyek 3 — Chunk Chef: cari ukuran chunk terbaik untuk retrieval

Tujuan: memilih ukuran *chunk* yang pas agar *retrieval* cepat dan relevan.

Fokus: *chunking* 200/400/600 kata, uji skor *cosine similarity*.

Langkah:

- Ambil satu teks panjang (boleh dokumen sekolah).
- Bagi jadi *chunk* 200, 400, 600 kata (overlap 50 kata).
- Untuk sebuah pertanyaan, hitung kemiripan *cosine* (simulasi *embedding* dengan bag-of-words).
- Pilih ukuran *chunk* yang memberi skor relevansi tertinggi **dan** jumlah *chunks* terambil (**top-k**) paling sedikit.

Pseudocode:

```
FUNCTION chunk(text, size, overlap):
    hasil = []
    i = 0
    WHILE i < len(kata):
        potong = kata[i : i+size]
        hasil.append(potong)
        i += (size - overlap)
    RETURN hasil

FOR size IN [200, 400, 600]:
    buat_chunks(size)
    hitung_cosine_score(pertanyaan, tiap_chunk)
```

```

        ambil top_k=3
        catat (rata-rata skor, waktu uji)
    PILIH size terbaik (skor tinggi, waktu cepat)

```

Contoh Python (simulasi sederhana):

```

import time, math
from collections import Counter

def bow_vector(tokens):
    c = Counter(tokens)
    return c

def cosine(a, b):
    keys = set(a) | set(b)
    dot = sum(a.get(k,0)*b.get(k,0) for k in keys)
    na = math.sqrt(sum(v*v for v in a.values()))
    nb = math.sqrt(sum(v*v for v in b.values()))
    return dot/(na*nb) if na*nb else 0.0

def chunk_words(words, size, overlap):
    out, i = [], 0
    while i < len(words):
        out.append(words[i:i+size])
        i += max(1, size - overlap)
    return out

text = ("rag di sekolah membantu qna lokal cepat aman "
        "optimasi cpu gpu ram chunk retrieval chroma faiss
").split()*80
question = "bagaimana mempercepat retrieval rag lokal".split()

qv = bow_vector(question)
for size in [200, 400, 600]:
    t0 = time.time()
    chunks = chunk_words(text, size, overlap=50)
    scores = []
    for ch in chunks:
        cv = bow_vector(ch)
        scores.append(cosine(qv, cv))
    top3 = sorted(scores, reverse=True)[:3]
    t1 = time.time()
    print(size, "avg_top3=", round(sum(top3)/3,3), "chunks=",
          len(chunks), "time=", round(t1-t0,3))

```

Kunci Jawaban (contoh):

- **Ukuran 400 kata** sering jadi **titik tengah**: cukup konteks, tidak kebanyakan *chunk*.
- **Prinsip: chunk terlalu kecil** → *retrieval* cepat tapi konteks kurang. **Terlalu besar** → *retrieval* lambat.

Proyek 4 — Cache Ninja: hemat waktu dengan answer cache

Tujuan: membuat *cache* jawaban agar pertanyaan berulang lebih cepat.

Fokus: *dictionary* sebagai *cache*, hit/miss counter.

Langkah:

- Buat fungsi `ask(q)` yang cek *cache* dulu.
- Jika ada, kembalikan jawaban *cache* (hit).
- Jika tidak, “panggil model” (simulasi lambat), simpan ke *cache* (miss).
- Tampilkan rasio hit/miss.

Pseudocode:

```
SET cache = {}
FUNCTION ask(q):
    IF q IN cache: return cache[q], "HIT"
    ELSE:
        ans = panggil_model(q)    // simulasi lambat
        cache[q] = ans
        return ans, "MISS"
FOR daftar_pertanyaan:
    cetak status (HIT/MISS)
HITRATE = total_HIT / total_semua
```

Contoh Python:

```
import time

cache = {}
def slow_model(q):
    time.sleep(0.4) # simulasi lambat
    return f"jawaban-{q}"

def ask(q):
    if q in cache:
        return cache[q], "HIT"
    ans = slow_model(q)
    cache[q] = ans
    return ans, "MISS"

qs = ["apa itu rag?", "fungsi gpu?", "apa itu rag?", "apa itu chunk?",
      "apa itu rag?"]
hit = miss = 0
for q in qs:
    _, s = ask(q)
    if s == "HIT": hit += 1
    else: miss += 1
print("hit:", hit, "miss:", miss, "hit_rate:", round(hit/(hit+miss),
2))
```

Kunci Jawaban (contoh):

- Dengan *cache*, pertanyaan berulang jadi **instan**.
- **Hit rate** tinggi = **latensi turun**. Cocok untuk *QnA* sekolah yang sering menanyakan hal sama.

Proyek 5 — Tiny Latency Predictor: neural network mini pakai Keras

Tujuan: memprediksi waktu respons dari fitur sederhana: *model_size*, *chunk_size*, *top_k*.

Fokus: *machine learning* dasar, Keras, *neural network* kecil.

Langkah:

- Buat data sintetis (20–50 baris) berisi fitur & label *latency_ms*.
- Bangun *neural network* kecil (2–3 *Dense layer*).
- Latih cepat (5–20 *epochs*).
- Uji prediksi untuk kombinasi baru.

Pseudocode:

```
BUAT dataset: X=[model_size, chunk_size, top_k], y=latency_ms
NORMALISASI X (opsional)
BANGUN NN: Input -> Dense(16) -> Dense(8) -> Dense(1)
TRAIN 10 epochs
PREDIKSI untuk (model_size=2, chunk_size=400, top_k=3)
```

Contoh Python (sketsa sederhana):

```
# Catatan: butuh tensorflow/keras terpasang (open source).
import numpy as np

# data sintetis kecil
X = np.array([
    [2, 200, 3],
    [2, 400, 3],
    [8, 400, 3],
    [8, 600, 5],
    [2, 600, 5],
    [4, 400, 3],
    [4, 600, 5],
    [8, 200, 3],
], dtype=float)
y = np.array([280, 320, 700, 900, 420, 500, 650, 620], dtype=float) # ms

# normalisasi sederhana
Xn = (X - X.mean(0)) / (X.std(0) + 1e-6)

from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(3,)),
```

```

        layers.Dense(16, activation="relu"),
        layers.Dense(8, activation="relu"),
        layers.Dense(1)
    ])
    model.compile(optimizer="adam", loss="mse")
    model.fit(Xn, y, epochs=20, verbose=0)

    # prediksi skenario baru
    test = np.array([[2, 400, 3]], dtype=float)
    testn = (test - X.mean(0)) / (X.std(0) + 1e-6)
    pred = model.predict(testn, verbose=0)
    print("Prediksi latency (ms):", float(pred[0][0]))

```

Kunci Jawaban (contoh):

- Prediksi untuk **model_size=2, chunk=400, top_k=3** $\approx$ **300–350 ms** (tergantung data).
- **Insight:** model kecil + *chunk* sedang + *top-k* kecil $\rightarrow$ **latensi lebih rendah**.

Tips

- **Batasi *max_tokens*** dan ***top-k***.
- Gunakan *quantization* (Q4/Q5) untuk model *LLM* lokal.
- Simpan vektor di *FAISS* / *Chroma* (keduanya *open source*).
- Monitor *CPU/GPU/RAM* saat uji.
- Catat semua hasil uji di *Pandas* $\rightarrow$ gampang dianalisis.

Dengan 5 proyek ini, kalian sudah praktik *optimasi* ala profesional. Campuran *Python*, *Pandas*, *machine learning*, *Keras*, dan *neural network* mini bikin kalian paham “*setting mesin*” AI. **Tujuan kita:** RAG lokal yang **ringan, cepat, dan stabil** di komputer sekolah.

BAB 8: Proyek – Chatbot Sekolah Berbasis Dokumen Kurikulum

Selamat! Sekarang kita sudah sampai di bab paling seru: **Proyek Akhir!** Setelah kita belajar banyak hal, dari AI, RAG, sampai etika, sekarang kita akan gabungkan semuanya dalam sebuah **proyek nyata**. Kalian bakal jadi ilmuwan data beneran!

Tujuan Pembelajaran

- Mengaplikasikan semua ilmu yang sudah didapat (Ollama, OpenWebUI, RAG).
- Mengikuti alur kerja analisis data dari awal sampai akhir.
- Membuat **chatbot** yang bisa menjawab pertanyaan dari dokumen kurikulum.
- Menganalisis dan mengevaluasi hasil proyek.

Peta Konsep Sederhana

Bayangkan ini adalah proses membuat game.

- **Tulis Skrip (Mendapatkan Dataset):** Kita cari data yang ingin kita olah.
- **Bersihkan Kode (Menyiapkan Dokumen):** Kita rapikan data yang kotor pakai Python.
- **Rancang Level (Menginstal RAG):** Kita instal **Ollama** dan **OpenWebUI**.
- **Uji Coba Game (Membangun RAG):** Kita bangun **chatbot** yang cerdas.
- **Rilis Game (Menjawab Pertanyaan):** Kita pamerkan hasilnya dalam bentuk laporan yang keren.

Apersepsi: Kamu Mau Tahu Respon Penonton Film!

Kamu mau tahu, ada berapa bab di buku kurikulum? Atau, materi apa saja yang ada di bab 5? Bingung kan kalau harus baca semua halaman satu-satu? Nah, sekarang, kita bisa membuat **chatbot** yang bisa menjawab pertanyaan-pertanyaan itu secara otomatis!

Penjelasan Teori

Bayangkan kamu punya guru pribadi yang bisa ditanya kapan saja. Guru ini bisa menjawab semua pertanyaanmu tentang kurikulum sekolah. Seru kan? Nah, di proyek ini kita akan membuat chatbot pintar yang bisa melakukan hal itu.

Chatbot ini tidak sekadar hafal. Dia bisa **mencari, membaca, dan menjawab pertanyaan dari dokumen kurikulum**. Kita akan menggunakan *tool open source* seperti **Python, Pandas, Machine Learning, Keras, dan Neural Network**. Semua gratis, bisa dipakai siapa saja!

Kenapa Chatbot?

- Bayangkan kamu ingin tahu: “*Apa isi bab 5 di kurikulum Bahasa Indonesia?*”
 - Kalau manual, kamu harus buka buku, baca satu-satu.
 - Kalau pakai chatbot, tinggal tanya. Jawaban langsung keluar!
- Chatbot ini mirip teman yang super rajin, tidak pernah capek, dan selalu siap bantu kamu belajar.

Langkah Besar dalam Proyek Ini

1. **Kumpulkan Data**
 - Ambil dokumen kurikulum (bisa dalam bentuk PDF, teks, atau file).
 - Data ini seperti “bahan baku” yang nanti kita masak.
2. **Bersihkan Data dengan Python**
 - Hilangkan simbol aneh, tanda baca tidak penting, atau huruf besar/kecil.
 - Tujuannya agar teks rapi dan gampang diproses.
3. **Bangun Otak AI dengan RAG**
 - *Retriever*: Bagian ini yang mencari jawaban.
 - *Generator*: Bagian ini yang menyusun jawaban agar enak dibaca.
 - Gabungkan dengan Ollama + OpenWebUI biar bisa dijalankan lokal di laptop.
4. **Uji Coba Chatbot**
 - Coba tanyakan: “*Materi apa saja di bab 2?*”
 - Lihat apakah jawabannya sesuai.
5. **Evaluasi Hasil**
 - Cek apakah chatbot menjawab benar.
 - Kalau ada yang salah, perbaiki datanya atau atur ulang programnya.

Teori yang Harus Dipahami

- **Python**: bahasa pemrograman yang mudah, mirip bahasa sehari-hari.
- **Pandas**: tool Python untuk mengolah data, seperti Excel versi canggih.
- **Machine Learning**: cara komputer belajar dari data.
- **Neural Network**: model AI yang meniru cara kerja otak manusia.
- **RAG (Retriever-Augmented Generation)**: teknik untuk membuat chatbot pintar, gabungan antara *mesin pencari* dan *otak penjawab*.

Analogi Seru

Bikin chatbot ini mirip bikin game:

- **Tulis Skrip** → Ambil data kurikulum.
- **Rapikan Kode** → Bersihkan teks.
- **Rancang Level** → Instal RAG (Ollama + OpenWebUI).
- **Mainkan Game** → Uji chatbot, tanyakan sesuatu.
- **Rilis Game** → Bagikan hasil chatbot ke teman atau guru

Kasus Ilustrasi

Kasus 1: Chatbot untuk Menjawab Pertanyaan Bab Kurikulum

Bayangkan kamu ingin tahu isi bab 3 pada kurikulum Matematika. Daripada membuka buku dan membaca banyak halaman, chatbot bisa langsung menjawab.

Ilustrasi:

- Input: *"Apa isi Bab 3 Matematika?"*
- Output: *"Bab 3 membahas Persamaan Linear dan Contoh Soalnya."*

Pseudocode:

```
# Pseudocode sederhana
dokumen = load_kurikulum("matematika.pdf")
pertanyaan = "Apa isi Bab 3?"
jawaban = RAG(dokumen, pertanyaan)
print(jawaban)
```

Kasus 2: Chatbot Analisis Ulasan Film Sekolah

Misalnya ada acara nonton bareng film di sekolah. Semua siswa menulis ulasan. Chatbot bisa membaca ulasan itu dan memberi tahu apakah filmnya seru atau membosankan.

Ilustrasi:

- Input: *"Filmnya keren banget, saya suka alurnya!"*
- Output: *"Sentimen: Positif"*

Pseudocode:

```
# Analisis sentimen sederhana
ulasan = "Filmnya keren banget, saya suka alurnya!"
kata_positif = ["keren", "bagus", "seru"]
if any(kata in ulasan for kata in kata_positif):
    sentimen = "Positif"
else:
    sentimen = "Negatif"
print(sentimen)
```

Kasus 3: Chatbot Cek Materi Ulangan

Siswa bisa bertanya: *"Materi apa saja yang keluar untuk ulangan Fisika bab 4?"* Chatbot akan mencari di dokumen kurikulum, lalu menjawab dengan ringkasan materi.

Ilustrasi:

- Input: *"Materi ulangan bab 4 Fisika apa?"*

- Output: *“Bab 4 Fisika membahas Listrik Dinamis: hukum Ohm, rangkaian seri-paralel, dan energi listrik.”*

Pseudocode:

```
# Simulasi tanya jawab kurikulum
dokumen = {
    "fisika_bab4": "Listrik Dinamis: hukum Ohm, rangkaian
seri-paralel, energi listrik"
}
pertanyaan = "Materi bab 4 Fisika"
jawaban = dokumen.get("fisika_bab4", "Tidak ditemukan")
print(jawaban)
```

Kasus 4: Chatbot Monitoring Event Sekolah

Setelah ada event sekolah (misalnya lomba olahraga), siswa memberi komentar di media sosial. Chatbot bisa otomatis membaca komentar itu dan memberi tahu apakah event tersebut disukai atau tidak.

Ilustrasi:

- Komentar: *“Lomba futsal keren banget, panitianya oke!”*
- Output: *“Sentimen: Positif (Siswa puas dengan acara).”*

Pseudocode:

```
# Analisis komentar event
komentar = ["Lomba futsal keren banget", "Acaranya membosankan"]
for teks in komentar:
    if "keren" in teks or "oke" in teks:
        print(teks, "-> Positif")
    else:
        print(teks, "-> Negatif")
```

Kasus 5: Chatbot Sistem Rekomendasi Belajar

Chatbot bisa memberi saran materi tambahan. Misalnya siswa bingung di Bab 2, chatbot akan menyarankan untuk membaca Bab 1 terlebih dahulu sebagai dasar.

Ilustrasi:

- Input: *“Saya bingung dengan Bab 2 Kimia.”*
- Output: *“Coba pelajari Bab 1 dulu, tentang struktur atom, karena itu dasar Bab 2.”*

Pseudocode:

```
# Sistem rekomendasi materi
kesulitan = "kimia_bab2"
rekomendasi = {
    "kimia_bab2": "Pelajari bab1: Struktur Atom sebagai dasar"
```

```
}
print(rekomendasi[kesulitan])
```

Catatan

- Chatbot bisa dipakai untuk **jawab pertanyaan kurikulum**.
- Bisa juga dipakai untuk **analisis sentimen ulasan film atau event sekolah**.
- Bisa **cek materi ulangan, monitor komentar siswa**, dan bahkan **beri rekomendasi belajar**.
- Semua dilakukan dengan *tool open source* seperti Python dan Pandas.

Soal Latihan & Pembahasan

Soal 1 – Membersihkan Data Teks

Soal:

Kamu punya file `ulasan_film.csv` dengan kolom `ulasan`. Langkah pertama apa yang harus dilakukan sebelum data dipakai untuk analisis?

Jawaban:

Membersihkan teks. Hilangkan tanda baca, ubah huruf jadi kecil, buang simbol yang tidak perlu.

Pembahasan:

Data mentah biasanya kotor. Kalau tidak dibersihkan, komputer bingung membaca.

Pseudocode:

```
import pandas as pd
data = pd.read_csv("ulasan_film.csv")
data["ulasan"] = data["ulasan"].str.lower() # ubah ke huruf
kecil
data["ulasan"] = data["ulasan"].str.replace(r"[^a-zA-Z\s]", "",
regex=True)
print(data.head())
```

Soal 2 – Daftar Kata Positif

Soal:

Sebutkan 3 kata yang bisa dimasukkan ke daftar kata positif untuk analisis sentimen!

Jawaban:

“bagus”, “seru”, “keren”

Pembahasan:

Kata-kata ini sering muncul di komentar positif. Chatbot bisa memakai daftar ini untuk mengenali perasaan pengguna.

Pseudocode:

```
kata_positif = ["bagus", "seru", "keren"]
ulasan = "Filmnya keren sekali"
if any(k in ulasan for k in kata_positif):
    print("Sentimen: Positif")
```

Soal 3 – Variabel Sentimen

Soal:

Bagaimana membuat variabel **sentimen** yang berisi “positif” jika ulasan mengandung kata positif?

Jawaban:

Gunakan **if/else** untuk mengecek kata di dalam ulasan.

Pembahasan:

Komputer tidak tahu arti kata. Kita bantu dengan aturan sederhana: kalau ada kata positif → label positif.

Pseudocode:

```
ulasan = "Cerita film ini sangat bagus"
kata_positif = ["bagus", "seru", "keren"]
if any(k in ulasan for k in kata_positif):
    sentimen = "positif"
else:
    sentimen = "negatif"
print(sentimen)
```

Soal 4 – Menghitung Recall

Soal:

Kamu punya 50 ulasan positif. Modelmu berhasil memprediksi 45 dengan benar. Berapa nilai Recall?

Jawaban:

$45 / 50 = 90\%$

Pembahasan:

Recall = berapa banyak data positif yang berhasil ditemukan model. Kalau tinggi, artinya model tidak banyak “lupa” mendeteksi data positif.

Pseudocode:

```
benar = 45
total_positif = 50
recall = benar / total_positif
print("Recall:", recall * 100, "%")
```

Soal 5 – Visualisasi Hasil

Soal:

Mengapa visualisasi hasil analisis sentimen itu penting?

Jawaban:

Karena dengan grafik kita lebih mudah melihat tren. Misalnya, grafik batang bisa langsung menunjukkan berapa persen ulasan positif dan negatif.

Pembahasan:

Membaca angka banyak itu bikin pusing. Grafik bikin data lebih jelas dan menarik.

Pseudocode (contoh sederhana pakai matplotlib):

```
import matplotlib.pyplot as plt

labels = ["Positif", "Negatif"]
jumlah = [70, 30]

plt.bar(labels, jumlah)
plt.title("Hasil Analisis Sentimen")
plt.show()
```

Catatan

- **Soal 1** → Data harus dibersihkan dulu.
- **Soal 2** → Buat daftar kata positif.
- **Soal 3** → Gunakan *if/else* untuk menentukan sentimen.
- **Soal 4** → Recall = benar / total data positif.
- **Soal 5** → Visualisasi mempermudah membaca data.

Dengan latihan ini, kalian tidak hanya paham teori, tapi juga tahu bagaimana *pseudocode* dan Python membantu dalam proyek chatbot berbasis kurikulum.

Fun Facts!

- Proyek ini adalah versi mini dari pekerjaan seorang **Data Scientist**.
- Kalian bisa pakai **Ollama** dan **OpenWebUI** untuk bantu kalian bikin kode atau menjelaskan hasil.
- **RAG lokal** bisa kalian pakai buat **chatbot** yang bisa menjawab pertanyaan tentang proyek kalian.

Tips Belajar atau Menghafal Cepat

- **Kerjakan Bertahap**: Jangan langsung bikin semua model. Mulai dari satu per satu.
- **Banyak Bertanya**: Jangan ragu untuk bertanya pada guru atau teman jika ada yang bingung.
- **Simpan Kodenya**: Simpan semua kode kalian di satu tempat. Ini bakal jadi portofolio kalian.

Aktivitas Siswa

Aktivitas 1 – Mengumpulkan & Membersihkan Data

Tujuan: Belajar mengolah data teks agar rapi sebelum dipakai chatbot.

Langkah-langkah:

1. Buat file teks sederhana berisi minimal 20 ulasan (campur positif & negatif).
2. Impor file ke Python menggunakan *pandas*.
3. Bersihkan teks: hilangkan tanda baca, ubah huruf jadi kecil, dan buang simbol aneh.

Contoh Jawaban:

Data siap dipakai setelah semua teks seragam.

Pseudocode / Python Script:

```
import pandas as pd

# baca file
data = pd.read_csv("ulasan_film.csv")

# ubah huruf jadi kecil
data["ulasan"] = data["ulasan"].str.lower()

# hapus simbol/tanda baca
data["ulasan"] = data["ulasan"].str.replace(r"[^a-zA-Z\s]", "",
regex=True)

print(data.head())
```

Aktivitas 2 – Membuat Daftar Kata Positif & Negatif

Tujuan: Mengenalkan dasar *word list* untuk analisis sentimen.

Langkah-langkah:

1. Bagi kelas jadi beberapa kelompok.
2. Buat daftar 10 kata positif (misal: keren, bagus, hebat).
3. Buat daftar 10 kata negatif (misal: jelek, buruk, bosan).
4. Simpan daftar kata ini di Python sebagai *list*.

Contoh Jawaban:

```
Kata positif: ["keren", "bagus", "seru"]
Kata negatif: ["jelek", "buruk", "bosan"]
```

Pseudocode / Python Script:

```
kata_positif = ["keren", "bagus", "seru"]
kata_negatif = ["jelek", "buruk", "bosan"]

print("Kata positif:", kata_positif)
print("Kata negatif:", kata_negatif)
```

Aktivitas 3 – Analisis Sentimen Sederhana

Tujuan: Membuat program Python untuk membaca ulasan dan memberi label.

Langkah-langkah:

1. Ambil satu ulasan dari data.
2. Cek apakah ulasan mengandung kata positif atau negatif.
3. Beri label **positif**, **negatif**, atau **netral**.

Contoh Jawaban:

Ulasan *"Film ini keren banget"* → Label: **positif**

Pseudocode / Python Script:

```
ulasan = "Film ini keren banget"
if any(k in ulasan for k in kata_positif):
    label = "positif"
elif any(k in ulasan for k in kata_negatif):
    label = "negatif"
else:
    label = "netral"

print("Ulasan:", ulasan)
print("Label Sentimen:", label)
```

Aktivitas 4 – Menghitung Akurasi Model

Tujuan: Mengecek apakah program bekerja dengan baik.

Langkah-langkah:

1. Tentukan label manual (kebenaran asli) dari 10 ulasan.
2. Jalankan program analisis sentimen kalian.
3. Bandingkan hasil program dengan label manual.
4. Hitung akurasi = (jawaban benar / total ulasan) × 100%.

Contoh Jawaban:

Jika ada 8 ulasan benar dari 10 → Akurasi = 80%

Pseudocode / Python Script:

```
label_asli = ["positif", "negatif", "positif", "netral", "positif"]
label_prediksi =
["positif", "negatif", "netral", "netral", "positif"]
```

```

benar = sum(1 for a, p in zip(label_asli, label_prediksi) if a
== p)
akurasi = benar / len(label_asli) * 100

print("Akurasi:", akurasi, "%")

```

Aktivitas 5 – Visualisasi Hasil

Tujuan: Menampilkan hasil analisis dalam bentuk grafik agar mudah dibaca.

Langkah-langkah:

1. Hitung jumlah ulasan positif, negatif, netral.
2. Gunakan *matplotlib* untuk membuat grafik batang.
3. Tampilkan grafik di layar.

Contoh Jawaban:

Jika ada 7 positif, 2 negatif, 1 netral → Grafik batang dengan 3 kategori.

Pseudocode / Python Script:

```

import matplotlib.pyplot as plt

labels = ["Positif", "Negatif", "Netral"]
jumlah = [7, 2, 1]

plt.bar(labels, jumlah)
plt.title("Hasil Analisis Sentimen")
plt.show()

```

Catatan

- **Aktivitas 1** → Belajar membersihkan data.
- **Aktivitas 2** → Membuat daftar kata positif & negatif.
- **Aktivitas 3** → Analisis sentimen sederhana pakai *if/else*.
- **Aktivitas 4** → Menghitung akurasi model.
- **Aktivitas 5** → Menampilkan hasil dengan grafik.

Dengan 5 aktivitas ini, kalian sudah menyelesaikan alur kerja *mini data science project*: dari data mentah → program Python → evaluasi → visualisasi. Semua pakai **tool open source** yang bisa dijalankan di laptop kalian.

Rangkuman

Dalam proyek ini, kalian sudah belajar bagaimana **data mentah bisa diubah menjadi pengetahuan**. Mulai dari **mengumpulkan data, membersihkannya dengan Python dan**

Pandas, sampai membuat **chatbot pintar berbasis RAG (Retriever-Augmented Generation)**. Semua ini dilakukan dengan tool *open source* seperti *Python*, *Pandas*, *Machine Learning*, *Keras*, dan *Neural Network*. Proses ini membuktikan bahwa **teknologi canggih bisa dipelajari dan dipakai oleh siswa SMK** hanya dengan laptop sederhana.

Kalian juga sudah melihat bagaimana **analisis sentimen** bekerja. Dengan membuat daftar kata positif dan negatif, kalian bisa melatih program sederhana untuk membaca ulasan film, komentar event sekolah, bahkan memfilter *komentar negatif*. Hasilnya tidak hanya berupa teks, tapi juga bisa divisualisasikan dengan grafik menggunakan *matplotlib* atau *Orange Data Mining*. **Visualisasi membuat data lebih mudah dipahami** dan lebih menarik untuk dipresentasikan.

Akhirnya, kalian sudah membuktikan bahwa **menjadi ilmuwan data mini itu mungkin!** Kalian tidak hanya belajar teori, tapi juga praktek nyata: dari *pseudocode* sederhana hingga script Python yang bisa dijalankan. Proyek chatbot kurikulum ini adalah contoh nyata bagaimana **AI bisa membantu kegiatan belajar di sekolah**. Ingatlah, **keterampilan ini adalah bekal berharga di masa depan**. Teruslah bereksperimen, jangan takut gagal, dan selalu gunakan *open source* agar kalian bisa berbagi ilmu dengan semua orang.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Chatbot bisa dibuat untuk menjawab pertanyaan dari dokumen kurikulum.
Jawaban: Benar
2. *RAG (Retriever-Augmented Generation)* hanya bisa dijalankan kalau ada internet.
Jawaban: Salah
3. Python dan Pandas adalah contoh teknologi *open source*.
Jawaban: Benar
4. Visualisasi hasil analisis hanya bisa dilakukan dengan aplikasi berbayar.
Jawaban: Salah
5. Analisis sentimen bisa membantu memisahkan ulasan positif dan negatif.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Apa tujuan utama proyek chatbot berbasis dokumen kurikulum?
 - a. Bermain game

- b. Menjawab pertanyaan dari dokumen
- c. Membuat animasi
- d. Menggambar grafik

Jawaban: b

2. Langkah pertama sebelum membuat chatbot adalah...
- a. Membersihkan data
 - b. Membeli server
 - c. Menginstal game
 - d. Membuat presentasi

Jawaban: a

3. tool yang dipakai untuk membersihkan data teks adalah...
- a. Photoshop
 - b. Python
 - c. CorelDraw
 - d. Excel versi bajakan

Jawaban: b

4. *Pandas* di Python biasanya dipakai untuk...
- a. Menggambar
 - b. Olah data
 - c. Main musik
 - d. Bikin presentasi

Jawaban: b

5. Dalam *analisis sentimen*, daftar kata positif digunakan untuk...
- a. Menandai komentar lucu
 - b. Menentukan sentimen positif
 - c. Membuat tabel kosong
 - d. Menghapus dataset

Jawaban: b

6. Recall dihitung dengan rumus...
- a. Benar / Total data positif
 - b. Salah / Total data negatif
 - c. Total data / Waktu komputasi
 - d. Hasil prediksi / Jumlah bab

Jawaban: a

7. Visualisasi data dapat membantu...
- a. Membuat data terlihat indah dan jelas
 - b. Menghilangkan data lama
 - c. Menyembunyikan data salah
 - d. Membuat data jadi game

Jawaban: a

8. Ollama dan OpenWebUI digunakan untuk...
- a. Menyimpan dokumen
 - b. Menjalankan model LLM lokal
 - c. Membuat musik digital
 - d. Menghapus file CSV

Jawaban: b

9. Neural Network meniru cara kerja...
- a. Mesin ketik
 - b. Otak manusia
 - c. Printer 3D
 - d. Kamera digital

Jawaban: b

10. Hasil akhir dari proyek chatbot biasanya berupa...
- a. File gambar
 - b. Laporan analisis dan chatbot berjalan
 - c. Suara musik
 - d. Film animasi

Jawaban: b

C. Isian Singkat (5 Soal)

1. Sebutkan satu contoh teknologi *open source* yang dipakai dalam proyek ini.

Jawaban: Python

2. Tuliskan tiga contoh kata positif untuk analisis sentimen.

Jawaban: keren, bagus, seru

3. Jika sebuah model berhasil memprediksi 45 dari 50 ulasan positif dengan benar, recall-nya adalah...

Jawaban: 90%

4. Library Python yang dipakai untuk menampilkan grafik batang adalah...

Jawaban: matplotlib

5. RAG adalah singkatan dari...

Jawaban: Retriever-Augmented Generation

D. Soal Eksplorasi (3 Soal)

Soal 1:

Eksperimen Dataset Ujian Sekolah

Bayangkan kamu punya dataset nilai ujian matematika. Buat program Python sederhana yang bisa menjawab pertanyaan: “Siapa siswa dengan nilai tertinggi?”

Contoh Jawaban:

- Buat list nama & nilai
- Cari nilai maksimum
- Tampilkan nama siswa

```
siswa = {"Andi":80,"Budi":95,"Citra":90}
nama = max(siswa, key=siswa.get)
print("Nilai tertinggi:", nama, siswa[nama])
```

Soal 2:

Chatbot Rekomendasi Bab Belajar

Kamu bingung di Bab 3 Fisika. Buat program sederhana yang merekomendasikan bab sebelumnya sebagai dasar belajar.

Contoh Jawaban:

- Buat dictionary hubungan antar bab
- Jika input Bab 3 → Rekomendasi Bab 2

```
rekom = {"Bab 3":"Pelajari Bab 2 dulu","Bab 2":"Pelajari Bab 1 dulu"}
print(rekom.get("Bab 3"))
```

Soal 3:

Analisis Komentar Media Sosial

Kamu ingin chatbot mendeteksi komentar negatif sederhana. Buat program Python mini dengan daftar kata negatif (contoh: “jelek”, “buruk”).

Contoh Jawaban:

- Buat list kata negatif
- Cek apakah ada kata negatif dalam komentar
- Jika ada → “Komentar negatif”

```
neg = ["jelek","buruk"]
komen = "Film ini jelek banget"
if any(k in komen for k in neg):
    print("Komentar negatif")
else:
    print("Komentar aman")
```

Dengan soal ini, siswa SMK bisa berlatih coding dasar, mengenal *open source tools*, dan memahami konsep AI secara sederhana tapi tetap menantang.

Tugas Proyek

Proyek 1 – Chatbot Absensi Kelas

Soal: Buat program Python sederhana untuk mencatat absensi siswa (hadir/tidak hadir) dan biarkan chatbot menjawab pertanyaan: *“Siapa saja yang tidak hadir hari ini?”*

Kunci Jawaban:

- Langkah 1: Buat dataset sederhana berisi nama siswa dan status hadir/tidak hadir.
- Langkah 2: Simpan data dalam list/dictionary Python.
- Langkah 3: Buat fungsi untuk mencari siswa yang absen.
- Langkah 4: Chatbot mengembalikan daftar nama siswa absen.

Pseudocode / Python Script:

```
absensi = {
    "Andi": "hadir",
    "Budi": "tidak hadir",
    "Citra": "hadir",
    "Dewi": "tidak hadir"
}

def cari_absen(data):
    return [nama for nama, status in data.items() if status ==
"tidak hadir"]

print("Siswa absen:", cari_absen(absensi))
```

Proyek 2 – Chatbot Kamus Istilah Kurikulum

Soal: Buat chatbot mini yang bisa menjelaskan arti istilah kurikulum, misalnya: *“Apa itu kompetensi dasar?”*

Kunci Jawaban:

- Langkah 1: Buat dictionary Python dengan istilah dan penjelasannya.
- Langkah 2: Buat fungsi untuk mencari arti istilah.
- Langkah 3: Chatbot menjawab berdasarkan dictionary.

Pseudocode / Python Script:

```
kamus = {
    "kompetensi dasar": "Kemampuan minimal yang harus dikuasai
siswa.",
    "silabus": "Rencana pembelajaran untuk satu mata
pelajaran.",
    "capaian pembelajaran": "Hasil belajar yang diharapkan."
}
```

```
def tanya_istilah(istilah):
    return kamus.get(istilah.lower(), "Istilah tidak ditemukan")

print(tanya_istilah("Kompetensi Dasar"))
```

Proyek 3 – Analisis Nilai Siswa

Soal: Buat program Python yang membaca nilai siswa dari file CSV, lalu chatbot bisa menjawab pertanyaan: *“Siapa siswa dengan nilai tertinggi?”*

Kunci Jawaban:

- Langkah 1: Siapkan file **nilai.csv** berisi nama dan nilai.
- Langkah 2: Baca file dengan *pandas*.
- Langkah 3: Cari siswa dengan nilai tertinggi.
- Langkah 4: Tampilkan hasil.

Pseudocode / Python Script:

```
import pandas as pd

data = pd.DataFrame({
    "nama": ["Andi", "Budi", "Citra", "Dewi"],
    "nilai": [85, 92, 88, 75]
})

siswa_top = data.loc[data["nilai"].idxmax()]
print("Siswa nilai tertinggi:", siswa_top["nama"], "(",
siswa_top["nilai"], ")")
```

Proyek 4 – Chatbot Pengecek Plagiarisme Sederhana

Soal: Buat chatbot yang bisa mendeteksi apakah sebuah kalimat sama dengan kalimat lain di dataset.

Kunci Jawaban:

- Langkah 1: Siapkan list berisi kalimat referensi.
- Langkah 2: Buat fungsi untuk mengecek apakah kalimat input ada di dataset.
- Langkah 3: Chatbot jawab *“Plagiarisme terdeteksi”* jika sama persis.

Pseudocode / Python Script:

```
kalimat_ref = [
    "Belajar Python itu menyenangkan",
    "Data science sangat berguna di masa depan"
]

def cek_plagiat(kalimat):
    if kalimat in kalimat_ref:
```

```

        return "Plagiarisme terdeteksi"
    else:
        return "Kalimat aman"

print(cek_plagiat("Belajar Python itu menyenangkan"))

```

Proyek 5 – Chatbot Rekomendasi Belajar

Soal: Buat chatbot sederhana yang merekomendasikan bab yang harus dipelajari, berdasarkan bab yang sedang sulit dipahami siswa.

Kunci Jawaban:

- Langkah 1: Buat dictionary berisi hubungan antar bab.
- Langkah 2: Jika siswa kesulitan di Bab X, rekomendasikan Bab sebelumnya sebagai dasar.
- Langkah 3: Tampilkan rekomendasi.

Pseudocode / Python Script:

```

rekomendasi = {
    "Bab 2": "Pelajari dulu Bab 1: Pengantar",
    "Bab 3": "Pelajari dulu Bab 2: Konsep Dasar",
    "Bab 4": "Pelajari dulu Bab 3: Studi Kasus"
}

def saran_belajar(bab):
    return rekomendasi.get(bab, "Tidak ada rekomendasi")

print(saran_belajar("Bab 3"))

```

Catatan

- **Proyek 1:** Absensi siswa otomatis.
- **Proyek 2:** Kamus istilah kurikulum.
- **Proyek 3:** Analisis nilai siswa.
- **Proyek 4:** Cek plagiarisme sederhana.
- **Proyek 5:** Rekomendasi belajar bab kurikulum.

Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.

Para penulis adalah **dosen aktif di Institut Teknologi Tangerang Selatan (ITTS)**, sebuah **institusi pendidikan tinggi berbasis teknologi** yang berperan aktif dalam pengembangan **solusi digital** dan *kecerdasan buatan (AI)*. Tidak hanya mengajar, mereka juga merupakan **praktisi dan inovator** yang menghasilkan **karya-karya nyata di bidang teknologi mandiri**.

Berlokasi di **Komplek Komersial BSD, Serpong Utara, Tangerang Selatan**, ITTS hadir sebagai **pusat inovasi** yang mengintegrasikan **pembelajaran, penelitian, dan pengembangan teknologi secara menyeluruh**. Fokus utama ITTS mencakup bidang strategis seperti *AI, Machine Learning, Deep Learning*, jaringan, *cloud*, keamanan siber, pemrograman, sistem informasi, dan multimedia.

ITTS menawarkan lingkungan akademik yang dinamis dan kolaboratif. Setiap minggu, diselenggarakan berbagai *workshop, demo teknologi*, seminar, hingga *webinar*—mayoritas **gratis dan tersedia secara terbuka di YouTube**. Ini menciptakan **kultur belajar yang aktif dan relevan dengan perkembangan industri**.

Fasilitas ITTS menunjang kegiatan riset dan inovasi secara optimal. Lab komputer dilengkapi *GPU RTX 4060* untuk eksperimen *AI*, serta **laboratorium teknologi dengan puluhan server, perangkat IoT, sistem komunikasi radio, dan infrastruktur jaringan jarak jauh**. Dari ekosistem ini lahir **produk-produk mandiri** seperti **ChatGPT versi lokal** untuk institusi pendidikan dan **sistem perpustakaan digital yang dapat diakses tanpa koneksi internet**.

Daya saing ITTS juga tercermin dari karakter dosen dan mahasiswa yang tidak hanya akademik, tetapi juga **produktif secara praktis**. Mahasiswa didorong aktif menjadi **narasumber, penulis, dan peneliti** sejak dini. Banyak diantaranya telah menghasilkan **karya buku, artikel ilmiah, serta terlibat langsung dalam proyek-proyek teknologi**.

Kolaborasi strategis dengan berbagai mitra industri—seperti *IDCloudHost, Ubuntu, XecureIT, Dinas KOMINFO Tangerang Selatan*, berbagai *ISP*, dan komunitas IT nasional—menjadi **jembatan nyata antara kampus dan dunia profesional**. Hasilnya, **lebih dari 75% mahasiswa ITTS telah bekerja bahkan sebelum lulus, dan hampir seluruh lulusan langsung terserap industri**.

ITTS bukan hanya tempat belajar, tapi tempat mencipta. Dosen sebagai **penulis dan praktisi**, mahasiswa sebagai **peneliti dan kontributor**—semuanya berperan **membangun ekosistem teknologi yang mandiri dan berdaya saing tinggi**.

Institut Teknologi Tangerang Selatan (ITTS)

Komplek Komersial BSD

Jl. Raya Serpong No. Kav. 9, Lengkong Karya, Serpong Utara

Kota Tangerang Selatan, Banten 15331

☎ (+62) 877-7277-1775

✉ info@itts.ac.id

🌐 www.itts.ac.id